

RAPPORT DE PROJET DE FIN D'ETUDES

GRENOBLE INP Esisar 2025/2026

Titre du projet

Control Implementation for a Safe and Secure Self-Driving Car

Nom et adresse de l'entreprise

LCIS Laboratory

50 rue Barthelemy de Laffemas, CS 10054, 26902 Valence Cedex 9

Nom et prénom de l'étudiant

Quang Thinh BUI

Dates du stage	09/03/2026 – 25/09/2026
Spécialité	ISC
Tuteur Entreprise	Ionela PRODAN
Tuteur ESISAR	Laurent LEFEVRE

RAPPORT DE PROJET DE FIN D'ETUDES

GRENOBLE INP Esisar 2025/2026

Mots clés : Conduite autonome, Model Predictive Control (MPC), MPC non linéaire (NMPC) , Flatness-Based MPC (FLMPC), Differential flatness, évitement d'obstacles, Quanser QCar2, Quanser Interactive Labs, ROS2.

Résumé : Ce projet de fin d'études porte sur la mise en œuvre et la validation de stratégies de commande prédictive pour la navigation autonome du Quanser QCar2. Le travail comprend la génération de trajectoires de Bézier basée sur la platitude différentielle, le suivi de trajectoire et l'évitement d'obstacles. Deux contrôleurs ont été étudiés : un NMPC utilisant directement le modèle cinématique non linéaire du véhicule et un FLMPC exploitant la platitude différentielle afin d'obtenir un modèle de prédiction linéaire. Les deux approches ont été évaluées en simulation puis implémentées sur le QCar2 réel avec ROS 2. Les résultats montrent un avantage de calcul du FLMPC en simulation, tandis que le NMPC offre une meilleure précision de suivi lors des essais expérimentaux. Le FLMPC a également été étendu à l'évitement d'obstacles par des contraintes mixtes-entiers, avec une stratégie d'activation permettant de réduire la complexité du problème d'optimisation.

Keywords : Autonomous driving, Model Predictive Control (MPC), Nonlinear MPC (NMPC), Flatness-Based MPC (FLMPC), Differential flatness, Obstacle avoidance, Quanser QCar2, Quanser Interactive Labs, ROS2.

Abstract : This graduation project focuses on the implementation and validation of Model Predictive Control strategies for the autonomous navigation of the Quanser QCar2. The work includes flatness-based Bézier trajectory generation, trajectory tracking, and obstacle avoidance. Two controllers were investigated: an NMPC based directly on the nonlinear kinematic vehicle model and an FLMPC using differential flatness to obtain a linear prediction model. Both approaches were evaluated in simulation and then implemented on the physical QCar2 using ROS2. The results show a computational advantage for the FLMPC in simulation, while the NMPC achieves better tracking accuracy in the experimental comparison. The FLMPC was also extended to obstacle avoidance using mixed-integer constraints. An obstacle-activation strategy was introduced to reduce the optimization complexity by limiting the number of obstacles considered at each control step.

Contents

1	Introduction	6
1.1	Motivation	6
1.2	Challenges	7
1.3	Project Methodology and Progression	8
1.4	Project Cost Overview	8
2	Project Context and Specifications	9
2.1	Project Context and Strategic Issues	9
2.2	Specifications	9
3	QCar2 Platform and Tool Selection	10
3.1	Technological Watch and Tool Selection	10
3.2	Hardware Description	10
3.3	Software Description	12
3.3.1	Robot Operating System 2	12
3.3.2	Quanser Interactive Labs	12
4	Theoretical Tools	13
4.1	Differential Flatness	13
4.2	Model Predictive Control	14
4.3	QCar2 Kinematic Model	15
4.4	Experimental Environment Mapping	16
5	Reference Trajectory Generation	17
5.1	Flat Output Selection	17
5.2	Boundary Conditions	18
5.3	Flatness-Based Bézier Trajectory Generation	19
5.4	Automatic Boundary Computation	20
5.4.1	Heading Angle Computation	20
5.4.2	Path Curvature and Steering Angle Computation	21
5.4.3	Longitudinal Velocity Computation	22
5.4.4	Longitudinal Acceleration Computation	23
5.4.5	Construction of Flat Boundary Conditions	23
6	Controller Design	24
6.1	NMPC Controller	24
6.2	FLMPC Controller	25
6.3	Obstacle Avoidance Constraints	27
6.4	Worst-Case Virtual Input Constraint Set	28
7	Simulation Results	31
7.1	Reference Trajectory Tracking	31
7.1.1	NMPC	31
7.1.2	FLMPC	32
7.1.3	NMPC and FLMPC Comparison	33

7.2	Obstacle Avoidance	34
7.2.1	OAMPC	34
7.2.2	Obstacle Activation	34
8	Experimental Results	37
8.1	Reference Trajectory Tracking	37
8.1.1	NMPC	37
8.1.2	FLMPC	39
8.1.3	NMPC and FLMPC Comparison	41
9	Conclusion and Future Directions	43
9.1	Conclusion	43
9.2	Future Directions	44
	Bibliography	45
	Annexes	46

List of Tables

6.1	Worst-case virtual-input radius for fixed longitudinal velocities	29
7.1	NMPC and FLMPC simulation performance comparison	33
8.1	NMPC experimental performance over four runs	38
8.2	FLMPC experimental performance over four runs	41
8.3	NMPC and FLMPC experimental performance comparison	42

List of Figures

1.1	Quanser QCar2	6
3.1	NVIDIA Jetson AGX Orin Onboard Computer	11
3.2	Visual Perception Sensors	11
4.1	Kinematic single-track model	15
4.2	Previously scanned experimental environment used for QCar2 experiments.	16
6.1	Physical input constraint set $\mathcal{U}$ of the QCar2.	29
6.2	State-dependent virtual-input constraint sets for different longitudinal velocities.	30
7.1	Closed-loop NMPC simulation architecture in Quanser Interactive Labs.	31
7.2	Closed-loop FL MPC simulation architecture in Quanser Interactive Labs.	32
8.1	Closed-loop NMPC experimental architecture on the QCar2.	37
8.2	Experimental setup with the ground workstation and the QCar2.	37
8.3	Closed-loop FL MPC experimental architecture on the QCar2.	39
A.1	NMPC simulation trajectory tracking.	46
A.2	NMPC simulation state tracking.	47
A.3	NMPC simulation control inputs.	48
A.4	NMPC simulation computation time.	48
A.5	FL MPC simulation trajectory tracking.	49
A.6	FL MPC simulation state tracking.	50
A.7	FL MPC simulation control inputs.	51
A.8	FL MPC simulation computation time.	51
A.9	NMPC and FL MPC simulation state-tracking comparison.	52
A.10	NMPC and FL MPC simulation tracking error comparison.	53
A.11	NMPC and FL MPC simulation control input comparison.	54
A.12	NMPC and FL MPC simulation computation time comparison.	54
A.13	OAMPC simulation trajectory tracking with obstacle avoidance.	55
A.14	OAMPC simulation state trajectories.	56
A.15	OAMPC simulation control inputs.	57
A.16	OAMPC simulation computation time.	57
A.17	OAMPC simulation trajectory tracking with obstacle activation.	58
A.18	Number of active obstacles during the OAMPC simulation.	58
A.19	OAMPC simulation vehicle states with obstacle activation.	59
A.20	OAMPC simulation control inputs with obstacle activation.	60
A.21	OAMPC simulation computation time with obstacle activation.	60
A.22	NMPC experimental trajectory-tracking results over four runs.	61
A.23	NMPC experimental state-tracking results over four runs.	62
A.24	NMPC experimental control-input results over four runs.	62
A.25	NMPC experimental computation-time results over four runs.	63
A.26	NMPC Mk2 experimental trajectory tracking.	63

A.27 NMPC Mk2 experimental state tracking.	64
A.28 NMPC Mk2 experimental control inputs.	64
A.29 NMPC Mk2 experimental computation time.	65
A.30 FL MPC experimental trajectory-tracking results over four runs.	65
A.31 FL MPC experimental state-tracking results over four runs.	66
A.32 FL MPC experimental control-input results over four runs.	66
A.33 FL MPC experimental computation-time results over four runs.	67
A.34 FL MPC Mk2 experimental trajectory tracking.	67
A.35 FL MPC Mk2 experimental state tracking.	68
A.36 FL MPC Mk2 experimental control inputs.	68
A.37 FL MPC Mk2 experimental computation time.	69
A.38 NMPC and FL MPC experimental trajectory-tracking comparison.	69
A.39 NMPC and FL MPC experimental tracking-error comparison.	70
A.40 NMPC and FL MPC experimental state-tracking comparison.	71
A.41 NMPC and FL MPC experimental control-input comparison.	72
A.42 NMPC and FL MPC experimental computation time comparison.	72

1. Introduction

This graduation project focuses on the implementation and validation of optimization-based control strategies for the safe autonomous navigation of the Quanser QCar2. The work combines trajectory generation, model predictive control, simulation, and experimental validation, with particular attention to trajectory-tracking performance, physical constraints, computational efficiency, and obstacle avoidance.

1.1. Motivation

Self-driving cars represent an important technological direction with the potential to improve mobility, reduce human driving errors, and decrease traffic-related accidents. However, their safety and reliability must be carefully studied before large-scale deployment. Autonomous vehicles operate in environments shared with human drivers, pedestrians, cyclists, and other robots, and their control systems must therefore respond appropriately while respecting physical limitations.

Security is also an important aspect of autonomous driving systems because their operation depends on sensors, embedded computers, communication networks, and software modules. Failures or compromised components may lead to unsafe behavior, which motivates the development of reliable constrained control strategies suitable for real-time operation.

The Quanser QCar2 is an open-architecture 1/10th-scale autonomous vehicle platform designed for research and education in self-driving technologies. Its embedded computing and sensing capabilities make it suitable for testing navigation and control algorithms in both simulation and real experimental conditions. The QCar2 platform used in this project is shown in Figure 1.1.



Figure 1.1: Quanser QCar2

This project builds upon the results in [1] and aims to implement optimization-based control strategies for the safe navigation of the QCar2. The control system must track

reference trajectories, respect the physical limits of the vehicle, and remain suitable for real-time implementation.

Model Predictive Control (MPC) is selected because it provides a systematic framework for trajectory tracking while explicitly considering state and input constraints. By solving an optimization problem at each control step, MPC predicts the future vehicle behavior and computes control actions adapted to the current state.

Several MPC-based approaches are investigated. First, **Nonlinear Model Predictive Control** (NMPC) is used for trajectory tracking based on the nonlinear kinematic vehicle model. Then, **Flatness-Based Model Predictive Control** (FLMPC) is studied to simplify the prediction model and improve computational efficiency. Finally, based on the mixed-integer obstacle-avoidance formulation presented in [1], polyhedral obstacle constraints are incorporated into the flatness-based optimization problem to enable collision avoidance while investigating the computational challenges of mixed-integer optimization.

From the laboratory perspective, this project provides LCIS with a reusable simulation and experimental framework for the development and validation of MPC-based control strategies on the QCar2. The implemented NMPC and FLMPC architectures, together with the trajectory-generation and obstacle-avoidance framework, can support future research on autonomous navigation and real-time optimization-based control.

1.2. Challenges

QLabs provided a convenient framework for validating the software architecture and vehicle behavior before real experiments. The main difficulties therefore appeared during the transition from simulation to real-time execution on the QCar2.

The most important difficulty was integrating the MPC solver into the control loop of the vehicle while keeping the computation time low enough for real-time operation. Since an Optimal Control Problem (OCP) has to be solved at each sampling instant, the choice of the prediction horizon, the constraints, and the solver configuration became critical, especially for the obstacle avoidance problem.

Another challenge concerned communication through ROS2 [8]. The amount of data exchanged among the vehicle, sensors, and control nodes had to be managed to avoid unnecessary bandwidth usage and maintain a stable control frequency.

These difficulties required a deeper understanding of the QCar2 hardware architecture, its communication protocols, and the Quanser ROS2 source code. Several adaptations were therefore needed to make the software structure compatible with the objectives of the project and with the practical limitations of the experimental platform.

For a quick overview of the obtained results, video demonstrations of the simulations and experiments are available below:

Simulation results:

NMPC, FLMPC, OAMPC, and OAMPC with obstacle activation are demonstrated in Quanser QLab: <https://youtu.be/w3ytkV00ATY>

Experimental results:

- NMPC trajectory tracking: https://youtu.be/I_gmxMKuoCM
- FLMPC trajectory tracking: <https://youtu.be/9vCbPOEIag4>

1.3. Project Methodology and Progression

The project was conducted using an iterative task-based methodology rather than a fixed long-term schedule. The objectives were progressively defined according to the results obtained at each stage. For each task, the corresponding method was first studied and implemented, then validated in simulation or on the experimental platform. The obtained results were subsequently analyzed and discussed before proceeding to the next development stage.

The work progressed through several main phases: familiarization with the QCar2 platform and its software environment, reference trajectory generation, NMPC trajectory tracking, FLMPc development and comparison with the NMPC, mixed-integer obstacle avoidance and obstacle activation in simulation, and finally the experimental implementation and validation of the NMPC and FLMPc controllers on the physical QCar2.

1.4. Project Cost Overview

The main costs associated with this project are related to the experimental hardware, the motion-capture infrastructure, and the internship duration. The software tools used in the project are mainly open-source, which limits additional licensing costs.

The principal cost components are:

- Crazyflie 2.1 nanodrones: approximately €225 per drone;
- Raspberry Pi 4B boards: approximately €120 per board, depending on the configuration and accessories;
- Crazyradio PA USB dongles: approximately €30 per unit;
- Qualisys motion-capture system: approximately €200,000;
- Quanser QCar2: approximately €17,000;
- Internship gratification: approximately €600 per month.

The use of low-cost embedded computing units such as Raspberry Pi boards is important for the project because it allows the high-level control algorithms to be tested in a physically distributed configuration without relying only on a powerful central computer. This makes the experimental setup more representative of future multi-agent robotic systems, where each agent or group of agents may rely on limited onboard or nearby embedded computation.

Overall, the project cost is dominated by the existing laboratory infrastructure, especially the motion-capture system and the Crazyflie experimental platform. The additional cost directly associated with the proposed distributed embedded architecture remains moderate, since it mainly requires Raspberry Pi boards and open-source software tools.

2. Project Context and Specifications

2.1. Project Context and Strategic Issues

This graduation project was carried out at the LCIS Laboratory of Grenoble INP in Valence and is associated with the Region PlanMAV project. It focuses on the implementation and validation of optimization-based control strategies for autonomous car-like vehicles using the Quanser QCar2 platform.

The project addresses the need for safe and reliable autonomous navigation in environments shared with human drivers, pedestrians, and cyclists. In this context, the control system must track reference trajectories accurately, respect the physical limitations of the vehicle, and remain suitable for real-time operation.

A major technical issue is the computational cost of Model Predictive Control, since an optimization problem must be solved at each sampling instant. The work therefore compares Nonlinear Model Predictive Control and Flatness-Based Model Predictive Control in terms of tracking performance and computational efficiency, before extending the flatness-based formulation to mixed-integer obstacle avoidance.

From a social-responsibility perspective, the project is related to the improvement of mobility and the reduction of traffic-related accidents, while safety and security remain essential for the acceptance of autonomous-driving technologies. These considerations are reflected in the use of constrained control methods, simulation-based validation before experiments, and the attention given to real-time feasibility and safe trajectory tracking.

2.2. Specifications

The project specifications are defined around the implementation and validation of MPC-based control strategies for the Quanser QCar2 platform.

The main specifications of the project are:

- validate the QCar2 software and hardware environment in simulation and on the physical platform;
- generate smooth reference trajectories compatible with the kinematic vehicle model;
- implement an NMPC controller for trajectory tracking while respecting the physical state and input constraints;
- develop a Flatness-Based MPC formulation and compare its tracking performance and computational efficiency with the NMPC;
- extend the flatness-based formulation to obstacle avoidance using mixed-integer constraints and investigate its computational scalability;
- evaluate the real-time feasibility of the developed controllers through simulation and experimental tests.

The experimental validation presented in this report focuses on trajectory tracking with the NMPC and FLMPc controllers, while the obstacle-avoidance formulation is evaluated in simulation.

3. QCar2 Platform and Tool Selection

3.1. Technological Watch and Tool Selection

The initial project objectives proposed CasADi as the optimization framework. During the development of the project, the software tools were selected according to the requirements of the simulation and experimental implementations.

Quanser Interactive Labs was retained as the main simulation environment, while ROS2 provides the communication and integration framework for the physical QCar2 [5, 8]. For the simulation studies, Gurobi was selected because it supports the optimization problems required by the NMPC, FLMP, and mixed-integer obstacle-avoidance formulations [10].

For the experimental implementation, acados was selected for the NMPC and FLMP controllers because of its suitability for efficient real-time optimal control [9]. The controllers were integrated into the ROS2 C++ framework and executed onboard the QCar2. This separation between simulation and experimental tools provides flexibility during controller development while maintaining low computation times for the physical implementation.

3.2. Hardware Description

The Quanser QCar2 is the feature vehicle of the Self-Driving Car Studio, an open-architecture 1/10th-scale vehicle designed for academic self-driving applications [3]. It is equipped with a set of embedded sensors and onboard computational resources that make it suitable for testing perception, localization, navigation, and control algorithms. In this project, the QCar2 is used as the main experimental platform for validating autonomous navigation strategies, including trajectory tracking and obstacle avoidance.

The vehicle has a compact car-like structure and provides the necessary hardware interface for low-level control, sensor acquisition, and real-time experimentation. Its embedded architecture allows the implementation of algorithms directly on the vehicle, while keeping communication with external computers through ROS2 when needed.

The QCar2 used in this project is powered by an onboard NVIDIA Jetson AGX Orin Developer Kit 32GB, shown in Figure 3.1. This embedded computer provides the computational resources required for real-time processing of sensor data, ROS2 communication, and the execution of control algorithms. In practice, this onboard computer is important for deploying navigation and MPC-based algorithms close to the vehicle hardware.

Sensors:

- **LiDAR:** For environmental perception, the QCar2 is equipped with a 2D LiDAR sensor, shown in Figure 3.2(a). It is used to measure distances around the vehicle and can provide useful information for obstacle detection, mapping, and navigation tasks [7].
- **Intel RealSense D435 Camera:** The vehicle also includes an Intel RealSense D435 RGB-D camera, shown in Figure 3.2(b). This camera provides both visual and depth information, which can be used for perception and scene understanding.

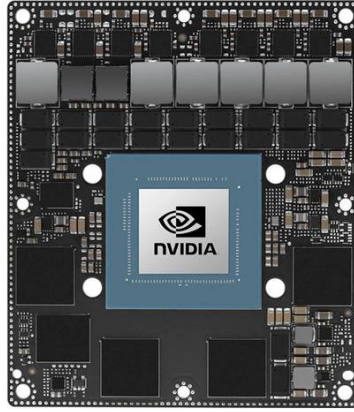


Figure 3.1: NVIDIA Jetson AGX Orin Onboard Computer

- **360° CSI Camera Suite:** The QCar2 includes a set of CSI cameras, shown in Figure 3.2(c), providing visual information around the vehicle. This camera suite is useful for perception tasks and for extending the field of view of the platform.
- **Inertial Measurement Unit:** The vehicle includes an inertial measurement unit used to obtain motion-related information such as angular velocity and acceleration. These measurements are useful for odometry, localization, and state estimation.
- **Encoder:** The QCar2 also includes motor encoder measurements, which can be used to estimate the wheel motion and support the computation of the vehicle state.



(a) LiDAR



(b) D435 Camera



(c) CSI Camera



(d) 360° camera coverage with blind spots in white

Figure 3.2: Visual Perception Sensors

3.3. Software Description

The QCar2 platform supports several software frameworks for simulation, communication, and experimental deployment. In this project, the main software tools are ROS2 for communication and experimental integration, and Quanser Interactive Labs for simulation.

3.3.1. Robot Operating System 2

ROS2 provides the main communication framework used to connect the QCar2, its sensors, and the control algorithms [8]. It is based on a distributed architecture in which software components are implemented as nodes and exchange data through topics using a publisher-subscriber communication mechanism.

In this project, ROS2 is used to receive vehicle and sensor information, publish control commands, and integrate navigation modules. It also provides the framework required for mapping, localization, and autonomous navigation using SLAM and Nav2.

The control algorithms are implemented mainly in Python and C++. At each control step, the current vehicle state is obtained through the corresponding software interface, the optimization problem is solved, and the computed control command is transmitted to the vehicle. This modular structure allows the different components to be developed and tested separately before their integration into the complete control loop.

3.3.2. Quanser Interactive Labs

Quanser Interactive Labs is used as the main simulation environment for the QCar2 [5]. It provides virtual models of the vehicle, sensors, and surrounding environment, allowing the developed algorithms to be tested before their deployment on the physical platform.

Communication with QLab is performed through the Quanser Python API. The control program reads the simulated vehicle state, while the computed velocity and steering commands are sent back to the virtual QCar2. This environment is used to validate trajectory generation, trajectory tracking, and obstacle avoidance under repeatable simulation conditions.

QLabs also reduces the risk of hardware damage during the initial development stages and facilitates analysis of controller behavior across different driving scenarios.

4. Theoretical Tools

In this chapter, the theoretical tools required for the controller design are introduced, including differential flatness, Model Predictive Control, and the kinematic model of the QCar2.

4.1. Differential Flatness

Differential flatness provides a useful framework in control theory for analyzing and controlling nonlinear systems. By transforming the nonlinear system into a representation based on flat outputs, the trajectory generation and control problems can be simplified.

A system that possesses the flatness property is referred to as a flat system. In such systems, there exists a flat output through which all system states and inputs can be explicitly expressed as functions of this output and a finite number of its time derivatives. An important class of systems for which trajectory generation is particularly convenient is the so-called differentially flat systems. The components of the flat output are differentially independent, meaning that there are no algebraic relations among the outputs and their derivatives.

Given the model of the system, where $q(t)$ stands for the states at the moment of time t , $u(t)$ for the inputs at the same moment, and f represents the nonlinear continuous-time definition of the system:

$$\dot{q}(t) = f(q(t), u(t)). \quad (4.1)$$

The states and the inputs can be represented as:

$$q = \phi(z, \dot{z}, \ddot{z}, \dots, z^{(s)}). \quad (4.2)$$

$$u = \varphi(z, \dot{z}, \ddot{z}, \dots, z^{(s+1)}). \quad (4.3)$$

where z is the flat output and s and l are finite nonnegative integers. The flat output can also be expressed as

$$z = h(q, u, \dot{u}, \ddot{u}, \dots, u^{(l)}). \quad (4.4)$$

Differentially flat systems are well suited to problems requiring trajectory planning, because the trajectory generation problem can be reduced to finding a trajectory of the flat outputs. For this reason, it is possible to define the desired motion directly in the flat output space, and then recover the corresponding states and inputs from the flatness relations.

For this project, the flatness property is important because it allows the car-like model to be rewritten in a more convenient form for control design. This will be used later to build the Flatness-Based Model Predictive Controller (FLMPC), where the optimization problem is formulated with respect to the flat output and the virtual input.

4.2. Model Predictive Control

Model Predictive Control (MPC) is an advanced control strategy that uses numerical optimization to achieve optimal performance. It works by predicting future control inputs and system responses using a mathematical model and then continuously optimizing these inputs at regular intervals based on a defined performance criterion. Its theoretical foundation is well established, with the properties of stability, optimality, and robustness being widely studied. In what follows, we outline the framework of MPC in discrete time.

After discretizing the continuous-time system in (4.1) with sampling time T_s , the discrete-time model can be written in the generic form

$$q_{k+1} = f_d(q_k, u_k). \quad (4.5)$$

where q_k and u_k denote the state and input vectors at sampling instant k , respectively. In this work, a fourth-order Runge–Kutta integration scheme is used to obtain the discrete-time prediction of the nonlinear vehicle dynamics.

Given the continuous-time model $\dot{q} = f(q, u)$ and assuming that the control input is constant over one sampling interval T_s , the RK4 integration is defined as

$$\begin{aligned} k_1 &= f(q_k, u_k), \\ k_2 &= f\left(q_k + \frac{T_s}{2}k_1, u_k\right), \\ k_3 &= f\left(q_k + \frac{T_s}{2}k_2, u_k\right), \\ k_4 &= f(q_k + T_s k_3, u_k). \end{aligned} \quad (4.6)$$

The next state is then computed as

$$q_{k+1} = q_k + \frac{T_s}{6} (k_1 + 2k_2 + 2k_3 + k_4). \quad (4.7)$$

At each computation step, MPC determines the optimal control input by solving an optimization problem. The algorithm evaluates possible control inputs and selects the one that minimizes a predefined cost function. This cost usually represents a trade-off between performance objectives, such as minimizing the tracking error, and the control effort required to reach the desired behavior.

The cost function can be written as

$$J_k = \sum_{i=0}^{N-1} \left(\|q_{i|k} - q_{\text{ref},i|k}\|_Q^2 + \|u_{i|k} - u_{\text{ref},i|k}\|_R^2 \right). \quad (4.8)$$

where N is the prediction horizon and $i|k$ denotes the prediction at time $k+i$, computed at the current sampling instant k . The matrices $Q = Q^\top \succeq 0$ and $R = R^\top \succ 0$ weight the state-tracking error and input-tracking error, respectively.

The complete finite-horizon optimization problem additionally includes a terminal state-tracking cost. The optimal control sequence can therefore be obtained from

$$\begin{aligned}
\mathbf{U}_k^* = \arg \min_{\mathbf{U}_k} \quad & J_k + \|q_{N|k} - q_{\text{ref},N|k}\|_P^2 \\
\text{s.t.} \quad & q_{0|k} = q_k, \\
& q_{i+1|k} = f_d(q_{i|k}, u_{i|k}), \quad i = 0, \dots, N-1, \\
& q_{i|k} \in \mathcal{Q}, \quad i = 0, \dots, N, \\
& u_{i|k} \in \mathcal{U}, \quad i = 0, \dots, N-1, \\
& q_{N|k} \in \mathcal{Q}_f.
\end{aligned} \tag{4.9}$$

where q_k denotes the current measured state and $\mathbf{U}_k = (u_{0|k}, u_{1|k}, \dots, u_{N-1|k})$ denotes the predicted control-input sequence. In this formulation, $\mathcal{Q}$ and $\mathcal{U}$ are the state and input constraint sets, respectively, while $\mathcal{Q}_f$ represents the terminal constraint set. The matrix $P = P^\top \succeq 0$ weights the terminal state-tracking error.

For nonlinear plants, the dynamical constraints are nonlinear and may lead to non-convex optimization, increasing computational time and complicating real-time implementation. Hence, linear or linearized models are often preferred due to their simpler mathematical properties and reduced computational burden. In this work, the nonlinear vehicle model is used directly in the NMPC, while its flatness-based linear representation is used in the FLMPC. The two formulations are later compared in terms of trajectory-tracking performance and computational efficiency.

4.3. QCar2 Kinematic Model

For trajectory planning and controller design, the QCar2 is approximated by a kinematic single-track model, illustrated in Figure 4.1. This model provides a simpler representation than the full dynamic model, while still capturing the main motion behavior of a car-like vehicle. It is therefore suitable for trajectory tracking and obstacle avoidance problems where the objective is to control the position, heading angle, and longitudinal velocity of the vehicle.

Let X and Y denote the Cartesian coordinates of the vehicle in the inertial frame, ψ the vehicle heading (rad), and v_x the longitudinal velocity (m/s). The control inputs are the front-wheel steering angle δ (rad) and the longitudinal acceleration a_x (m/s²). The parameter L_{wb} (m) represents the wheelbase of the vehicle.

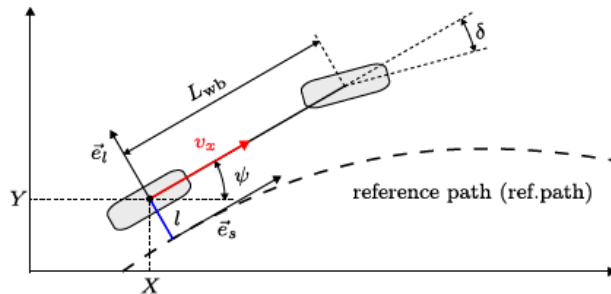


Figure 4.1: Kinematic single-track model

The state and input vectors are defined as

$$\mathbf{x} = [X \ Y \ \psi \ v_x]^\top, \quad \mathbf{u} = [\delta \ a_x]^\top. \tag{4.10}$$

Under the pure-rolling and no-slip assumptions, the continuous-time kinematic single-track model is given by [1]

$$\dot{X} = v_x \cos \psi, \quad (4.11a)$$

$$\dot{Y} = v_x \sin \psi, \quad (4.11b)$$

$$\dot{\psi} = \frac{v_x}{L_{wb}} \tan \delta, \quad (4.11c)$$

$$\dot{v}_x = a_x. \quad (4.11d)$$

For compactness, the model can be written as

$$\dot{\mathbf{x}}(t) = f_{\text{car}}(\mathbf{x}(t), \mathbf{u}(t)). \quad (4.12)$$

4.4. Experimental Environment Mapping

The experimental environment is treated as previously known. Before deploying the control algorithms on the physical QCar2, a map of the operating area is therefore generated and later reused for localization and visualization.

Quanser provides ROS2 resources for the QCar2 with a Cartographer-based mapping configuration. Rather than developing a SLAM algorithm as part of this project, the existing tools are used to construct the map from the vehicle's 2D LiDAR measurements.

During mapping, the QCar2 is manually driven through the experimental area while Cartographer processes the LiDAR data and builds a two-dimensional occupancy-grid map. Once the required area has been scanned, the map is saved and reused for the subsequent experiments.

In this project, SLAM is therefore used only as a preliminary mapping step. The resulting map, shown in Figure 4.2, provides the reference frame required by the localization system, allowing the QCar2 pose to be used by the trajectory-generation and control algorithms during the experiments.

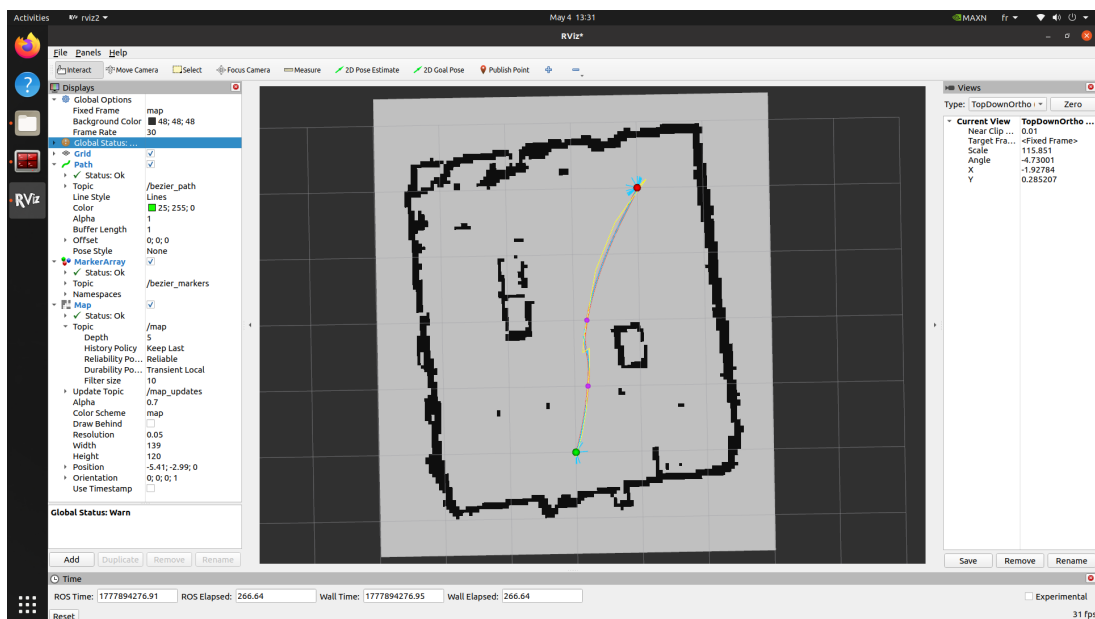


Figure 4.2: Previously scanned experimental environment used for QCar2 experiments.

5. Reference Trajectory Generation

5.1. Flat Output Selection

The objective of this chapter is to generate a smooth reference trajectory for the QCar2 before using it in the MPC controllers. Since the kinematic single-track model describes the vehicle, the trajectory can be generated using its differential flatness property. The main idea is to define the desired motion in a simpler output space and then recover the corresponding vehicle states and inputs.

For this model, the flat output is chosen as the Cartesian position of the vehicle:

$$\mathbf{z} = \begin{bmatrix} z_1 \\ z_2 \end{bmatrix} = \begin{bmatrix} X \\ Y \end{bmatrix}. \quad (5.1)$$

Using (4.11a) and (4.11b), the first derivative of the flat output is

$$\dot{\mathbf{z}} = \begin{bmatrix} \dot{z}_1 \\ \dot{z}_2 \end{bmatrix} = \begin{bmatrix} v_x \cos \psi \\ v_x \sin \psi \end{bmatrix}. \quad (5.2)$$

Therefore, the longitudinal velocity and the heading angle can be recovered from the first derivative of the flat output:

$$v_x = \sqrt{\dot{z}_1^2 + \dot{z}_2^2}, \quad \psi = \text{atan2}(\dot{z}_2, \dot{z}_1). \quad (5.3)$$

The steering angle can also be obtained from the flat output derivatives. By differentiating the heading angle in (5.3), one obtains

$$\dot{\psi} = \frac{\dot{z}_1 \ddot{z}_2 - \dot{z}_2 \ddot{z}_1}{\dot{z}_1^2 + \dot{z}_2^2}. \quad (5.4)$$

Using the heading dynamics in (4.11c), the steering angle becomes

$$\delta = \arctan \left(\frac{L_{wb} (\dot{z}_1 \ddot{z}_2 - \dot{z}_2 \ddot{z}_1)}{(\dot{z}_1^2 + \dot{z}_2^2)^{3/2}} \right). \quad (5.5)$$

The longitudinal acceleration is recovered from the derivative of the longitudinal velocity:

$$a_x = \frac{\dot{z}_1 \ddot{z}_1 + \dot{z}_2 \ddot{z}_2}{\sqrt{\dot{z}_1^2 + \dot{z}_2^2}}. \quad (5.6)$$

Consequently, all states and inputs of the QCar2 model can be expressed as functions of the flat output and its derivatives:

$$\begin{aligned} X &= z_1, & Y &= z_2, \\ \psi &= \text{atan2}(\dot{z}_2, \dot{z}_1), & v_x &= \sqrt{\dot{z}_1^2 + \dot{z}_2^2}, \\ \delta &= \arctan \left(\frac{L_{wb} (\dot{z}_1 \ddot{z}_2 - \dot{z}_2 \ddot{z}_1)}{(\dot{z}_1^2 + \dot{z}_2^2)^{3/2}} \right), & a_x &= \frac{\dot{z}_1 \ddot{z}_1 + \dot{z}_2 \ddot{z}_2}{\sqrt{\dot{z}_1^2 + \dot{z}_2^2}}. \end{aligned} \quad (5.7)$$

The reconstruction relations for ψ , δ , and a_x require $\dot{z}_1^2 + \dot{z}_2^2 > 0$, which is equivalent to $v_x > 0$. Therefore, the flatness-based reconstruction becomes singular when the longitudinal velocity is zero. In the trajectory-generation stage, this case is avoided by prescribing nonzero waypoint velocities or by applying an appropriate regularization constant.

This result shows that the trajectory can be generated directly in terms of the Cartesian position X and Y and their time derivatives. Once the flat output trajectory and its derivatives are known, the complete reference trajectory for the vehicle states and inputs can be reconstructed. This property is used in the following sections to impose boundary conditions and generate smooth Bézier trajectories.

5.2. Boundary Conditions

After selecting the Cartesian position as the flat output, the next step is to define the boundary conditions required for trajectory generation. These conditions must be expressed in the flat-output space because the Bézier trajectory will be constructed directly from the flat output and its derivatives.

At waypoint i , the vehicle state and input variables are described by

$$X_i, \quad Y_i, \quad \psi_i, \quad v_{x,i}, \quad \delta_i, \quad a_{x,i}.$$

The corresponding flat output and its first derivative are obtained directly from (5.1) and (5.2), respectively.

Differentiating (5.2) gives

$$\ddot{z}_1 = \frac{d}{dt} (v_x \cos \psi) = \dot{v}_x \cos \psi - v_x \dot{\psi} \sin \psi. \quad (5.8)$$

$$\ddot{z}_2 = \frac{d}{dt} (v_x \sin \psi) = \dot{v}_x \sin \psi + v_x \dot{\psi} \cos \psi. \quad (5.9)$$

Using the longitudinal velocity dynamics in (4.11d) and the heading dynamics in (4.11c), the second derivative of the flat output becomes

$$\ddot{\mathbf{z}} = \begin{bmatrix} a_x \cos \psi - \frac{v_x^2}{L_{wb}} \tan \delta \sin \psi \\ a_x \sin \psi + \frac{v_x^2}{L_{wb}} \tan \delta \cos \psi \end{bmatrix}. \quad (5.10)$$

Therefore, each complete vehicle waypoint can be converted into a complete flat boundary condition according to

$$(X_i, Y_i, \psi_i, v_{x,i}, \delta_i, a_{x,i}) \longrightarrow (\mathbf{z}_i, \dot{\mathbf{z}}_i, \ddot{\mathbf{z}}_i). \quad (5.11)$$

Hence, the boundary information required for trajectory generation is completely determined by the vehicle position, heading angle, longitudinal velocity, steering angle, and longitudinal acceleration at each waypoint. These flat boundary conditions will be used in Section 5.3 to construct the Bézier trajectory.

5.3. Flatness-Based Bézier Trajectory Generation

Once the flat boundary conditions have been determined, each trajectory segment can be constructed directly in the flat-output space. The trajectory is generated using the flat output defined in (5.1), with boundary conditions imposed up to the second derivative:

$$\mathbf{z}(0), \dot{\mathbf{z}}(0), \ddot{\mathbf{z}}(0), \mathbf{z}(T), \dot{\mathbf{z}}(T), \ddot{\mathbf{z}}(T). \quad (5.12)$$

Since the highest derivative order imposed at the boundaries is $r = 2$, the degree of the Bézier polynomial is selected such that

$$n + 1 = 2(r + 1) = 6, \quad n = 5. \quad (5.13)$$

Therefore, a degree-five Bézier polynomial is used for each trajectory segment. It is defined as

$$\mathbf{z}(s) = \sum_{i=0}^5 \mathbf{P}_i B_i^5(s), \quad s \in [0, 1], \quad (5.14)$$

where $\mathbf{P}_i \in \mathbb{R}^2$ are the Bézier control points and

$$B_i^5(s) = \binom{5}{i} (1-s)^{5-i} s^i \quad (5.15)$$

is the Bernstein basis polynomial of degree five.

The normalized time variable is introduced as

$$s = \frac{t}{T}, \quad (5.16)$$

where T denotes the traveling time of the trajectory segment. Hence, $s = 0$ and $s = 1$ correspond to the physical times $t = 0$ and $t = T$, respectively.

Using the normalized time variable, the first derivative of the Bézier trajectory with respect to time is

$$\dot{\mathbf{z}}(s) = \frac{5}{T} \sum_{i=0}^4 (\mathbf{P}_{i+1} - \mathbf{P}_i) B_i^4(s). \quad (5.17)$$

The second derivative is

$$\ddot{\mathbf{z}}(s) = \frac{20}{T^2} \sum_{i=0}^3 (\mathbf{P}_{i+2} - 2\mathbf{P}_{i+1} + \mathbf{P}_i) B_i^3(s). \quad (5.18)$$

The six Bézier control points are obtained directly from the initial and final flat boundary conditions.

At $s = 0$, the first three control points are

$$\mathbf{P}_0 = \mathbf{z}(0), \quad (5.19a)$$

$$\mathbf{P}_1 = \mathbf{z}(0) + \frac{T}{5} \dot{\mathbf{z}}(0), \quad (5.19b)$$

$$\mathbf{P}_2 = \mathbf{z}(0) + \frac{2T}{5} \dot{\mathbf{z}}(0) + \frac{T^2}{20} \ddot{\mathbf{z}}(0). \quad (5.19c)$$

At $s = 1$, the final three control points are

$$\mathbf{P}_5 = \mathbf{z}(1), \quad (5.20a)$$

$$\mathbf{P}_4 = \mathbf{z}(1) - \frac{T}{5}\dot{\mathbf{z}}(1), \quad (5.20b)$$

$$\mathbf{P}_3 = \mathbf{z}(1) - \frac{2T}{5}\dot{\mathbf{z}}(1) + \frac{T^2}{20}\ddot{\mathbf{z}}(1). \quad (5.20c)$$

Therefore, each trajectory segment is completely determined by its initial and final flat boundary conditions through (5.19) and (5.20).

5.4. Automatic Boundary Computation

In practice, the user does not specify all vehicle states and inputs at the waypoints. Instead, the user specifies the Cartesian positions of all waypoints, the heading angles at the first and last waypoints, and the traveling time for each segment.

For a sequence of N_w waypoints, the Cartesian position of waypoint i is defined as

$$\mathbf{p}_i = \begin{bmatrix} X_i \\ Y_i \end{bmatrix}, \quad i = 0, \dots, N_w - 1. \quad (5.21)$$

The prescribed boundary headings are ψ_0 and ψ_{N_w-1} , while the segment traveling times are denoted by T_i , $i = 0, \dots, N_w - 2$, where T_i is the traveling time from waypoint i to waypoint $i + 1$.

The remaining quantities required by the flatness-based trajectory generation, namely the intermediate heading angles, the longitudinal velocities, the steering angles, and the longitudinal accelerations, are automatically computed from the waypoint geometry and the segment times.

Thus, the initial user-defined data $\mathbf{p}_i$, ψ_0 , ψ_{N_w-1} , and T_i are converted into complete boundary waypoints of the form

$$\mathbf{q}_i = [X_i \ Y_i \ \psi_i \ v_{x,i} \ \delta_i \ a_{x,i}]^\top. \quad (5.22)$$

Using the mapping introduced in (5.11), these complete boundary waypoints are converted into the flat boundary conditions $\mathbf{z}_i$, $\dot{\mathbf{z}}_i$, and $\ddot{\mathbf{z}}_i$, which are required to construct the Bézier trajectory.

5.4.1. Heading Angle Computation

The heading angles at the first and last waypoints are directly imposed by the user:

$$\psi_0 = \psi_{\text{start}}, \quad \psi_{N_w-1} = \psi_{\text{goal}}. \quad (5.23)$$

The intermediate heading angles are computed from the local geometry of the waypoint sequence. For an intermediate waypoint i , the normalized incoming and outgoing direction vectors are

$$\mathbf{d}_i^- = \frac{\mathbf{p}_i - \mathbf{p}_{i-1}}{\|\mathbf{p}_i - \mathbf{p}_{i-1}\|}, \quad \mathbf{d}_i^+ = \frac{\mathbf{p}_{i+1} - \mathbf{p}_i}{\|\mathbf{p}_{i+1} - \mathbf{p}_i\|}. \quad (5.24)$$

The local tangent direction is approximated by the sum of the incoming and outgoing directions:

$$\tilde{\mathbf{h}}_i = \mathbf{d}_i^- + \mathbf{d}_i^+. \quad (5.25)$$

The corresponding unit heading direction is obtained by normalization:

$$\hat{\mathbf{h}}_i = \frac{\tilde{\mathbf{h}}_i}{\|\tilde{\mathbf{h}}_i\|} = \begin{bmatrix} \cos \psi_i \\ \sin \psi_i \end{bmatrix}, \quad i = 1, \dots, N_w - 2. \quad (5.26)$$

Therefore, the intermediate heading angle is computed as

$$\psi_i = \text{atan2}(\hat{h}_{i,y}, \hat{h}_{i,x}), \quad i = 1, \dots, N_w - 2. \quad (5.27)$$

The function atan2 is used to preserve the correct quadrant of the heading angle.

5.4.2. Path Curvature and Steering Angle Computation

The steering angle is computed from the local path curvature. From the kinematic single-track model, the heading dynamics are given by (4.11c).

For a planar path, the curvature is defined as the variation of the heading angle with respect to the traveled distance:

$$\kappa = \frac{d\psi}{ds}. \quad (5.28)$$

Since the traveled distance satisfies

$$\frac{d\ell}{dt} = v_x, \quad (5.29)$$

The curvature can also be written as

$$\kappa = \frac{d\psi/dt}{d\ell/dt} = \frac{\dot{\psi}}{v_x}. \quad (5.30)$$

Substituting the heading dynamics from (4.11c) into (5.30) gives

$$\kappa = \frac{\tan \delta}{L_{wb}}. \quad (5.31)$$

Therefore, if the path curvature is known, the steering angle can be computed as

$$\delta = \arctan(L_{wb}\kappa). \quad (5.32)$$

However, at this stage, the curvature cannot be computed directly from $\kappa = \dot{\psi}/v_x$. Although the heading angles are known at the waypoints, they are only discrete boundary orientations and do not yet define a continuous heading profile $\psi(t)$ along the complete path.

For this reason, the curvature used for the steering-angle computation is estimated directly from the geometry of three consecutive waypoints $\mathbf{p}_{i-1}$, $\mathbf{p}_i$, and $\mathbf{p}_{i+1}$. This provides a purely geometric approximation of the local path bending before the continuous Bézier trajectory is constructed.

The distances between the three consecutive waypoints are defined as

$$d_{i-1,i} = \sqrt{(X_i - X_{i-1})^2 + (Y_i - Y_{i-1})^2}, \quad (5.33)$$

$$d_{i,i+1} = \sqrt{(X_{i+1} - X_i)^2 + (Y_{i+1} - Y_i)^2}, \quad (5.34)$$

$$d_{i-1,i+1} = \sqrt{(X_{i+1} - X_{i-1})^2 + (Y_{i+1} - Y_{i-1})^2}. \quad (5.35)$$

Using these distances, the signed curvature at an intermediate waypoint is approximated as

$$\kappa_i = \frac{2[(X_i - X_{i-1})(Y_{i+1} - Y_i) - (Y_i - Y_{i-1})(X_{i+1} - X_i)]}{d_{i-1,i} d_{i,i+1} d_{i-1,i+1}}. \quad (5.36)$$

The numerator corresponds to the signed two-dimensional cross product between two consecutive path segments. Its sign indicates the turning direction of the path. A positive value corresponds to a left turn, while a negative value corresponds to a right turn. If the three points are aligned, the numerator is zero, and the curvature is also zero.

As a boundary assumption, the curvature at the first and last waypoints is set to zero:

$$\kappa_0 = 0, \quad \kappa_{N_w-1} = 0. \quad (5.37)$$

Finally, the steering angle at each waypoint is obtained from the geometric curvature as

$$\delta_i = \arctan(L_{wb}\kappa_i). \quad (5.38)$$

5.4.3. Longitudinal Velocity Computation

The longitudinal velocity is computed from the segment times. The straight-line distance between consecutive waypoints i and $i + 1$ is

$$d_i = \|\mathbf{p}_{i+1} - \mathbf{p}_i\|, \quad i = 0, \dots, N_w - 2. \quad (5.39)$$

Since the segment time T_i is prescribed, a nominal longitudinal velocity for this segment is estimated as

$$\bar{v}_{x,i} = \frac{d_i}{T_i}, \quad i = 0, \dots, N_w - 2. \quad (5.40)$$

However, the flatness-based boundary conditions require a longitudinal velocity at each waypoint. Therefore, the waypoint velocities are obtained from the adjacent segment velocities.

At the first waypoint,

$$v_{x,0} = \bar{v}_{x,0}. \quad (5.41)$$

At the last waypoint,

$$v_{x,N_w-1} = \bar{v}_{x,N_w-2}. \quad (5.42)$$

For an intermediate waypoint, the velocity is chosen as the average of the two neighboring segment velocities:

$$v_{x,i} = \frac{\bar{v}_{x,i-1} + \bar{v}_{x,i}}{2}, \quad i = 1, \dots, N_w - 2. \quad (5.43)$$

5.4.4. Longitudinal Acceleration Computation

For the kinematic single-track model used in this work, the longitudinal acceleration a_x is an input of the system. Therefore, after the waypoint velocities have been computed, the longitudinal acceleration is approximated from the variation of velocity over the segment times.

For segment i , the average longitudinal acceleration is

$$\bar{a}_{x,i} = \frac{v_{x,i+1} - v_{x,i}}{T_i}, \quad i = 0, \dots, N_w - 2. \quad (5.44)$$

The acceleration at the first waypoint is taken from the first segment:

$$a_{x,0} = \bar{a}_{x,0}. \quad (5.45)$$

The acceleration at the last waypoint is taken from the last segment:

$$a_{x,N_w-1} = \bar{a}_{x,N_w-2}. \quad (5.46)$$

For an intermediate waypoint, a centered approximation is used:

$$a_{x,i} = \frac{\bar{a}_{x,i-1} + \bar{a}_{x,i}}{2}, \quad i = 1, \dots, N_w - 2. \quad (5.47)$$

Thus, the automatically computed acceleration is consistent with the variation of the longitudinal velocity along the waypoint sequence.

5.4.5. Construction of Flat Boundary Conditions

After computing ψ_i , $v_{x,i}$, δ_i , and $a_{x,i}$, each waypoint is written as the complete boundary waypoint defined in (5.22).

Using the mapping in (5.11), the corresponding flat output and its derivatives are obtained from (5.1), (5.2), and (5.10).

Thus, the initial user-defined data are converted into complete flat boundary conditions of the form

$$(\mathbf{z}_i, \dot{\mathbf{z}}_i, \ddot{\mathbf{z}}_i).$$

For the segment connecting waypoint i to waypoint $i + 1$, the initial boundary conditions are

$$\mathbf{z}(0) = \mathbf{z}_i, \quad \dot{\mathbf{z}}(0) = \dot{\mathbf{z}}_i, \quad \ddot{\mathbf{z}}(0) = \ddot{\mathbf{z}}_i. \quad (5.48)$$

The final boundary conditions are

$$\mathbf{z}(1) = \mathbf{z}_{i+1}, \quad \dot{\mathbf{z}}(1) = \dot{\mathbf{z}}_{i+1}, \quad \ddot{\mathbf{z}}(1) = \ddot{\mathbf{z}}_{i+1}. \quad (5.49)$$

These boundary conditions are substituted into (5.19) and (5.20) to compute the six control points of the Bézier curve for each segment.

Finally, the generated reference trajectory is checked against the physical limits of the QCar2, such as

$$|\delta(t)| \leq \delta_{\max}, \quad |a_x(t)| \leq a_{x,\max}. \quad (5.50)$$

6. Controller Design

6.1. NMPC Controller

In this section, the trajectory-tracking problem is formulated as a Nonlinear Model Predictive Control problem over a finite prediction horizon. The state vector $\mathbf{x}$, the physical input vector $\mathbf{u}$, and the nonlinear kinematic single-track model of the QCar2 were previously defined in (4.10) and (4.11).

The reference trajectory generated in Chapter 5 is represented by the reference state and input sequences

$$\mathbf{x}_{\text{ref},i} = \begin{bmatrix} X_{\text{ref},i} \\ Y_{\text{ref},i} \\ \psi_{\text{ref},i} \\ v_{x,\text{ref},i} \end{bmatrix}, \quad \mathbf{u}_{\text{ref},i} = \begin{bmatrix} \delta_{\text{ref},i} \\ a_{x,\text{ref},i} \end{bmatrix}. \quad (6.1)$$

The longitudinal velocity constraint is

$$\mathcal{X} = \{\mathbf{x} \in \mathbb{R}^4 : v_{x,\min} \leq v_x \leq v_{x,\max}\}. \quad (6.2)$$

The steering-angle and longitudinal-acceleration constraints are

$$\mathcal{U} = \{\mathbf{u} \in \mathbb{R}^2 : \delta_{\min} \leq \delta \leq \delta_{\max}, \quad a_{x,\min} \leq a_x \leq a_{x,\max}\}. \quad (6.3)$$

Given a prediction horizon $N > 0$, the nonlinear optimal control problem solved at each sampling instant is written as

$$\begin{aligned} \min_{\substack{\mathbf{x}_0, \dots, \mathbf{x}_N \\ \mathbf{u}_0, \dots, \mathbf{u}_{N-1}}} & \sum_{i=0}^{N-1} \left[(\mathbf{x}_i - \mathbf{x}_{\text{ref},i})^\top Q (\mathbf{x}_i - \mathbf{x}_{\text{ref},i}) + (\mathbf{u}_i - \mathbf{u}_{\text{ref},i})^\top R (\mathbf{u}_i - \mathbf{u}_{\text{ref},i}) \right] \\ & + (\mathbf{x}_N - \mathbf{x}_{\text{ref},N})^\top P (\mathbf{x}_N - \mathbf{x}_{\text{ref},N}) \\ \text{s.t.} \quad & \mathbf{x}_0 = \mathbf{x}_{\text{meas}}, \\ & \mathbf{x}_{i+1} = f_d(\mathbf{x}_i, \mathbf{u}_i), \quad i = 0, \dots, N-1, \\ & \mathbf{x}_i \in \mathcal{X}, \quad i = 0, \dots, N, \\ & \mathbf{u}_i \in \mathcal{U}, \quad i = 0, \dots, N-1. \end{aligned} \quad (6.4)$$

Here, $Q = Q^\top \succeq 0$, $R = R^\top \succ 0$, and $P = P^\top \succeq 0$ are the weighting matrices for the state-tracking error, input-tracking error, and terminal state-tracking error, respectively. The function f_d denotes the discrete-time nonlinear prediction model obtained from the continuous-time vehicle dynamics. Its numerical implementation depends on the simulation or experimental control framework and is described in the corresponding results chapters.

At each sampling instant, the measured vehicle state is imposed as the initial condition. The reference state and input sequences

$$\mathbf{x}_{\text{ref},0}, \mathbf{x}_{\text{ref},1}, \dots, \mathbf{x}_{\text{ref},N}, \quad \mathbf{u}_{\text{ref},0}, \mathbf{u}_{\text{ref},1}, \dots, \mathbf{u}_{\text{ref},N-1} \quad (6.5)$$

are updated from the generated trajectory. The nonlinear optimal control problem is then solved, and only the first optimal input is applied to the vehicle:

$$\mathbf{u}_{\text{applied}} = \mathbf{u}_0^*. \quad (6.6)$$

At the next sampling instant, the vehicle state is measured again, the reference horizon is advanced, and the optimization problem is solved again. This procedure is repeated in a receding-horizon manner.

6.2. FLMPC Controller

In Chapter 5, the Cartesian position was selected as the flat output for reference trajectory generation. To follow the controller notation used in [1], the flat output is denoted in this section by $\mathbf{y}$, while $\mathbf{z}$ denotes the corresponding flat state.

For the Cartesian formulation, the flat output is written as

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} X \\ Y \end{bmatrix}. \quad (6.7)$$

Using the first derivative of the Cartesian position previously obtained in (5.2), the flat state vector is defined as

$$\mathbf{z} = \begin{bmatrix} y_1 \\ \dot{y}_1 \\ y_2 \\ \dot{y}_2 \end{bmatrix} = \begin{bmatrix} X \\ v_x \cos \psi \\ Y \\ v_x \sin \psi \end{bmatrix} = t(\mathbf{x}). \quad (6.8)$$

The original vehicle state can be recovered from the flat state through the inverse transformation

$$\mathbf{x} = \begin{bmatrix} X \\ Y \\ \psi \\ v_x \end{bmatrix} = \begin{bmatrix} z_1 \\ z_3 \\ \text{atan2}(z_4, z_2) \\ \sqrt{z_2^2 + z_4^2} \end{bmatrix} = t^{-1}(\mathbf{z}). \quad (6.9)$$

The virtual input vector is defined by the second derivatives of the flat outputs:

$$\boldsymbol{\nu} = \begin{bmatrix} \nu_1 \\ \nu_2 \end{bmatrix} = \begin{bmatrix} \ddot{y}_1 \\ \ddot{y}_2 \end{bmatrix}. \quad (6.10)$$

Using the flat state and virtual input, the vehicle dynamics become two decoupled double integrators:

$$\dot{\mathbf{z}} = A_c \mathbf{z} + B_c \boldsymbol{\nu}, \quad (6.11)$$

where

$$A_c = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad B_c = \begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 0 & 0 \\ 0 & 1 \end{bmatrix}. \quad (6.12)$$

The flat representation therefore allows the nonlinear vehicle model to be replaced by a linear time-invariant prediction model. With sampling time T_s , the continuous-time flat model is discretized under a zero-order hold assumption as

$$\mathbf{z}_{k+1} = A_d \mathbf{z}_k + B_d \boldsymbol{\nu}_k, \quad (6.13)$$

where

$$A_d = \begin{bmatrix} 1 & T_s & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & T_s \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad B_d = \begin{bmatrix} \frac{T_s^2}{2} & 0 \\ T_s & 0 \\ 0 & \frac{T_s^2}{2} \\ 0 & T_s \end{bmatrix}. \quad (6.14)$$

At each sampling instant, the measured physical state is transformed into the initial flat state according to

$$\mathbf{z}_k = t(\mathbf{x}_{\text{meas},k}). \quad (6.15)$$

For a prediction horizon N , the predicted flat-state and virtual-input sequences are denoted by

$$\mathbf{Z}_k = (\mathbf{z}_{k+1}, \dots, \mathbf{z}_{k+N}), \quad \mathbf{V}_k = (\boldsymbol{\nu}_k, \dots, \boldsymbol{\nu}_{k+N-1}). \quad (6.16)$$

For the Cartesian flat-output formulation, the tracking error is defined from the predicted and reference Cartesian positions:

$$\mathbf{e}_{k+j} = \begin{bmatrix} e_X \\ e_Y \end{bmatrix}_{k+j} = \begin{bmatrix} X_{k+j} \\ Y_{k+j} \end{bmatrix} - \begin{bmatrix} X_{\text{ref},k+j} \\ Y_{\text{ref},k+j} \end{bmatrix}. \quad (6.17)$$

At each sampling instant, the FLMPC solves the following optimization problem over the prediction horizon N :

$$\begin{aligned} \min_{\mathbf{Z}_k, \mathbf{V}_k} \quad & \sum_{j=0}^{N-1} (\mathbf{e}_{k+j+1}^\top Q \mathbf{e}_{k+j+1} + \boldsymbol{\nu}_{k+j}^\top R \boldsymbol{\nu}_{k+j}) \\ \text{s.t.} \quad & \mathbf{z}_{k+j+1} = A_d \mathbf{z}_{k+j} + B_d \boldsymbol{\nu}_{k+j}, \quad j = 0, \dots, N-1, \\ & \boldsymbol{\nu}_{k+j} \in W, \quad j = 0, \dots, N-1. \end{aligned} \quad (6.18)$$

Here, $\mathbf{Z}_k$ and $\mathbf{V}_k$ denote the predicted flat-state and virtual-input sequences, respectively. The matrices $Q = Q^\top \succeq 0$ and $R = R^\top \succ 0$ weight the Cartesian position-tracking error and the virtual control input, respectively.

The physical input constraints are originally defined as

$$\mathcal{U} = \{\mathbf{u} : \mathbf{u}_{\min} \leq \mathbf{u} \leq \mathbf{u}_{\max}\}, \quad \mathbf{u} = \begin{bmatrix} \delta \\ a_x \end{bmatrix}. \quad (6.19)$$

Since the optimization problem is solved in the flat coordinates, the physical input constraints must be transformed into constraints on the virtual input. From the physical-to-virtual input transformation, one obtains

$$\underbrace{\begin{bmatrix} -\frac{L_{wb}}{v_x^2} \sin \psi & \frac{L_{wb}}{v_x^2} \cos \psi \\ \cos \psi & \sin \psi \end{bmatrix}}_{M(\psi, v_x)} \begin{bmatrix} \nu_1 \\ \nu_2 \end{bmatrix} = \begin{bmatrix} \tan \delta \\ a_x \end{bmatrix} = \tilde{\mathbf{u}}. \quad (6.20)$$

Therefore, the virtual-input set is written as

$$W = \{\boldsymbol{\nu} : \tilde{\mathbf{u}}_{\min} \leq M(\psi, v_x) \boldsymbol{\nu} \leq \tilde{\mathbf{u}}_{\max}\}, \quad (6.21)$$

where

$$\tilde{\mathbf{u}}_{\min} = \begin{bmatrix} \tan \delta_{\min} \\ a_{x, \min} \end{bmatrix}, \quad \tilde{\mathbf{u}}_{\max} = \begin{bmatrix} \tan \delta_{\max} \\ a_{x, \max} \end{bmatrix}. \quad (6.22)$$

After solving the FL MPC problem, the first optimal virtual input $\boldsymbol{\nu}_k^*$ is converted back into the physical input using the current vehicle state:

$$\mathbf{u}_k^* = \begin{bmatrix} \delta_k^* \\ a_{x,k}^* \end{bmatrix} = \begin{bmatrix} \arctan \left(\frac{L_{wb}}{v_{x,k}^2} (-\nu_{1,k}^* \sin \psi_k + \nu_{2,k}^* \cos \psi_k) \right) \\ \nu_{1,k}^* \cos \psi_k + \nu_{2,k}^* \sin \psi_k \end{bmatrix}. \quad (6.23)$$

Remark on low-velocity sensitivity. As shown in (6.23), the steering-angle reconstruction contains the coefficient L_{wb}/v_x^2 . In the practical implementation, this denominator is regularized as $v_x^2 + \epsilon$, with $\epsilon > 0$, to avoid numerical singularity when v_x approaches zero.

For $L_{wb} = 0.25725$ m and $\epsilon = 10^{-3}$,

$$\frac{0.25725}{0.5^2 + 10^{-3}} \approx 1.025, \quad \frac{0.25725}{0.1^2 + 10^{-3}} \approx 23.386.$$

Thus, when the longitudinal velocity decreases from 0.5 m/s to 0.1 m/s, this coefficient increases by a factor of approximately 22.8. Consequently, small variations in the virtual input or heading angle can produce much larger steering commands at low velocity. This sensitivity can therefore contribute to the steering angle reaching its saturation limit during low-speed operation.

6.3. Obstacle Avoidance Constraints

The FL MPC formulation introduced in Section 6.2 is extended to include obstacle-avoidance constraints in the flat-output space. For the Cartesian formulation, the flat output corresponds directly to the global vehicle position (X, Y) .

A bounded convex polygonal obstacle region Q with n_e edges is represented by a set of half-space inequalities:

$$Q = \left\{ \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} : G \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} \leq \mathbf{f} \right\}, \quad (6.24)$$

where $G \in \mathbb{R}^{n_e \times 2}$ and $\mathbf{f} \in \mathbb{R}^{n_e}$.

To avoid the obstacle, the predicted vehicle position must lie outside Q . This condition is non-convex because at least one of the obstacle half-space inequalities must be violated. The Big- M method is used to represent this disjunctive condition through the following constraints:

$$\mathbf{f} - G \begin{bmatrix} y_{1,k+j+1} \\ y_{2,k+j+1} \end{bmatrix} + (\gamma - s)\mathbf{1} \leq M(\mathbf{1} - \boldsymbol{\alpha}_{k+j+1}), \quad (6.25a)$$

$$\mathbf{1}^\top \boldsymbol{\alpha}_{k+j+1} \geq 1, \quad (6.25b)$$

$$\boldsymbol{\alpha}_{k+j+1} \in \{0, 1\}^{n_e}, \quad (6.25c)$$

$$s \geq 0. \quad (6.25d)$$

for $j = 0, \dots, N-1$.

Here, M is a sufficiently large positive constant, γ is a safety buffer used to expand the obstacle region and account for the finite dimensions of the QCar2, and s is a scalar slack variable used to preserve feasibility. A positive value of s relaxes the safety buffer and is therefore strongly penalized in the objective function by the weight r_s .

For n_{obs} obstacles, obstacle m is represented by G_m and $\mathbf{f}_m$, and has $n_{e,m}$ edges. The variable $\boldsymbol{\alpha}$ collects the binary variables $\boldsymbol{\alpha}_{m,k+j+1}$ for all obstacles and prediction steps. The addition of the binary variables converts the FLMPC problem into a Mixed-Integer Quadratic Programming problem. The complete OAMPC problem is written as

$$\begin{aligned} \min_{\mathbf{z}_k, \mathbf{v}_k, \boldsymbol{\alpha}, s} \quad & \sum_{j=0}^{N-1} (\mathbf{e}_{k+j+1}^\top Q \mathbf{e}_{k+j+1} + \boldsymbol{\nu}_{k+j}^\top R \boldsymbol{\nu}_{k+j}) + r_s s \\ \text{s.t.} \quad & \mathbf{z}_{k+j+1} = A_d \mathbf{z}_{k+j} + B_d \boldsymbol{\nu}_{k+j}, \quad j = 0, \dots, N-1, \\ & \boldsymbol{\nu}_{k+j} \in W, \quad j = 0, \dots, N-1, \\ & \mathbf{f}_m - G_m \begin{bmatrix} y_{1,k+j+1} \\ y_{2,k+j+1} \end{bmatrix} + (\gamma - s)\mathbf{1} \leq M(\mathbf{1} - \boldsymbol{\alpha}_{m,k+j+1}), \\ & \mathbf{1}^\top \boldsymbol{\alpha}_{m,k+j+1} \geq 1, \\ & \boldsymbol{\alpha}_{m,k+j+1} \in \{0, 1\}^{n_{e,m}}, \\ & s \geq 0, \quad m=1, \dots, n_{\text{obs}}; j=0, \dots, N-1. \end{aligned} \quad (6.26)$$

6.4. Worst-Case Virtual Input Constraint Set

The virtual-input constraint set W defined in (6.21) depends on the longitudinal velocity v_x and the heading angle ψ . This section studies its variation and derives a worst-case approximation inspired by [2].

From the symmetric physical input limits in (6.19) and the input transformation in (6.20), the virtual input satisfies

$$|-\nu_1 \sin \psi + \nu_2 \cos \psi| \leq \frac{v_x^2}{L_{\text{wb}}} \tan \delta_{\text{max}}, \quad (6.27)$$

$$|\nu_1 \cos \psi + \nu_2 \sin \psi| \leq a_{x,\text{max}}, \quad (6.28)$$

where

$$\beta(v_x) = \frac{v_x^2}{L_{\text{wb}}} \tan \delta_{\text{max}}. \quad (6.29)$$

For a fixed $v_x > 0$, these inequalities define a rectangle in the (ν_1, ν_2) -plane. The common admissible region over all heading angles is a circular set.

$$\widehat{W}(v_x) = \bigcap_{\psi \in [0, 2\pi)} W(v_x, \psi) = \{\boldsymbol{\nu} \in \mathbb{R}^2 : \nu_1^2 + \nu_2^2 \leq r^2(v_x)\}. \quad (6.30)$$

Its radius is given by

$$r(v_x) = \min(a_{x,\max}, \beta(v_x)). \quad (6.31)$$

The critical longitudinal velocity is obtained when the two limits in (6.31) are equal:

$$\tilde{v}_x = \sqrt{\frac{a_{x,\max} L_{wb}}{\tan \delta_{\max}}}. \quad (6.32)$$

Therefore, the radius can be written as

$$r(v_x) = \begin{cases} \beta(v_x), & 0 < v_x < \tilde{v}_x, \\ a_{x,\max}, & v_x \geq \tilde{v}_x. \end{cases} \quad (6.33)$$

Using the physical limits applied to this QCar2, namely $a_{x,\max} = 1.0 \text{ m/s}^2$, $\delta_{\max} = 0.5236 \text{ rad}$, and $L_{wb} = 0.25725 \text{ m}$, the critical velocity is $\tilde{v}_x \approx 0.668 \text{ m/s}$, and the corresponding worst-case radii are reported in Table 6.1.

Table 6.1: Worst-case virtual-input radius for fixed longitudinal velocities

$v_x \text{ [m/s]}$	$\beta(v_x) \text{ [m/s}^2\text{]}$	$r(v_x) \text{ [m/s}^2\text{]}$	Limiting constraint
0.4	0.359	0.359	Steering angle
0.6	0.808	0.808	Steering angle
0.8	1.436	1.000	Acceleration
1.0	2.244	1.000	Acceleration

The physical input constraint set is illustrated in Figure 6.1. For each fixed velocity, $N_\psi = 30$ uniformly distributed heading angles are used to illustrate the rotation of $W(v_x, \psi)$. The yellow region represents the numerical intersection of the sampled rectangles and approaches the circular set defined in (6.30).

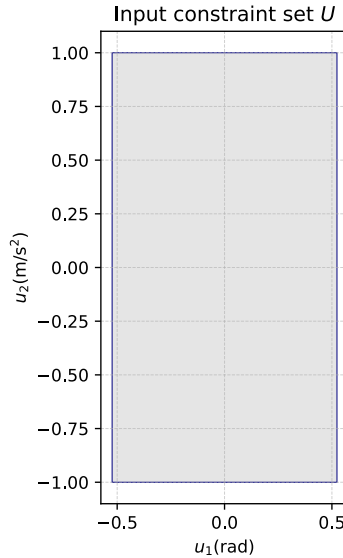


Figure 6.1: Physical input constraint set $\mathcal{U}$ of the QCar2.

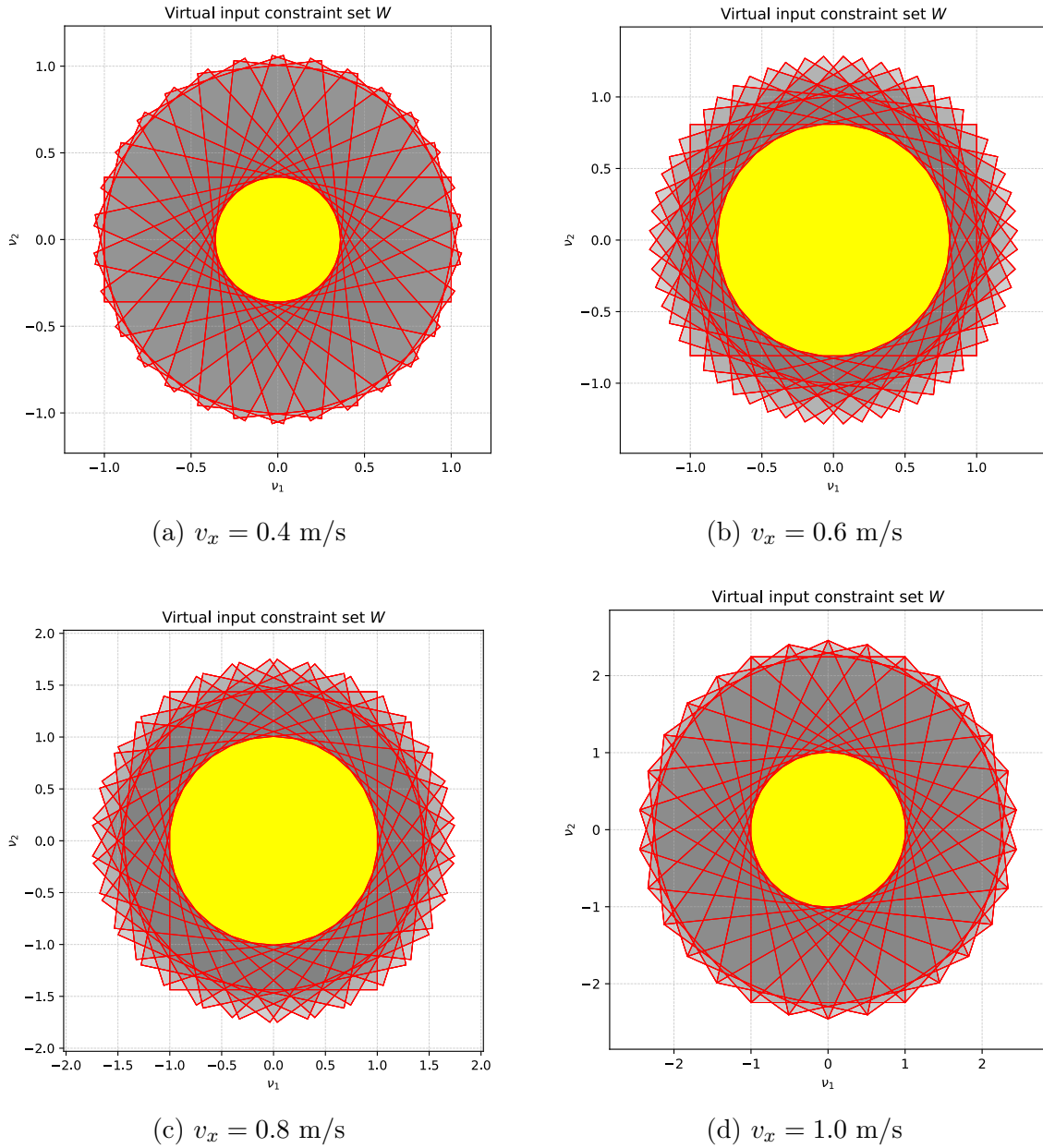


Figure 6.2: State-dependent virtual-input constraint sets for different longitudinal velocities.

As shown in Figure 6.2, the radius increases with v_x^2 below the critical velocity and remains equal to $a_{x,\max}$ above this value.

$$\lim_{v_x \rightarrow 0^+} r(v_x) = 0.$$

Hence, if arbitrarily low velocities are included, the global worst-case set collapses to the single point $\{(0,0)\}$. Such a constraint would be excessively conservative. For this reason, the state-dependent set $W(v_x, \psi)$ is applied directly in the FLMPC formulation instead of using one constant worst-case set for all operating velocities.

7. Simulation Results

It is essential to evaluate the control algorithms in simulation before deploying them on the experimental platform. In this work, the controllers were implemented in Python and evaluated using the QCar2 model in Quanser Interactive Labs through the Quanser Python API.

7.1. Reference Trajectory Tracking

7.1.1. NMPC

The NMPC optimization problem was formulated and solved using Gurobi Optimizer through its Python interface, GurobiPy. Gurobi is a general-purpose mathematical optimization solver supporting continuous, quadratic, mixed-integer, and nonlinear optimization problems.

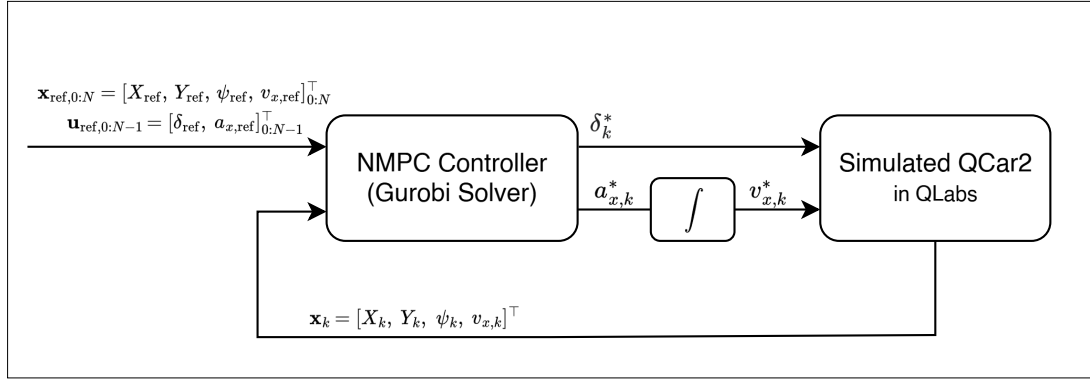


Figure 7.1: Closed-loop NMPC simulation architecture in Quanser Interactive Labs.

The closed-loop NMPC simulation architecture is shown in Figure 7.1. In the closed-loop simulation, the current vehicle state and the reference state and input trajectories over the prediction horizon are supplied to the NMPC at each sampling instant. The continuous-time kinematic single-track model is discretized using the fourth-order Runge–Kutta scheme introduced in Section 4.2. The optimization problem is then solved, and only the first optimal control input, consisting of the steering angle δ_k^* and longitudinal acceleration $a_{x,k}^*$, is used according to the receding-horizon principle.

The simulated QCar2 plant receives the steering angle δ and longitudinal velocity v_x as control commands. Therefore, the optimal acceleration $a_{x,k}^*$ is integrated to obtain the corresponding longitudinal velocity $v_{x,k}^*$. The resulting velocity and steering commands are then sent to QLab's, which updates the vehicle state and returns it to the NMPC for the next control step.

To improve the numerical initialization of consecutive optimization problems, a warm-start strategy is applied. For the first optimization, the reference state and input trajectories over the prediction horizon are used as the initial guess, with the first predicted state replaced by the current vehicle state. After a successful optimization, the complete optimal state and input sequences are stored. At the following sampling instant, these sequences are shifted forward by one prediction step, while their final elements are repeated

to preserve the horizon length. The first state of the shifted sequence is then replaced by the updated vehicle state, and the resulting sequences are used as the initial guess for the new optimization problem.

The parameters used for the NMPC simulation are

$$N = 13, \quad T_s = 0.07 \text{ s}, \quad L_{wb} = 0.25725 \text{ m}.$$

The weighting matrices are

$$Q = \text{diag}(160, 160, 80, 200), \quad R = \text{diag}(10, 500), \quad P = \text{diag}(320, 320, 160, 400).$$

The constraints are

$$-2.0 \leq v_x \leq 2.0 \text{ m/s}, \quad -0.5236 \leq \delta \leq 0.5236 \text{ rad}, \quad -1.0 \leq a_x \leq 1.0 \text{ m/s}^2.$$

After several simulation tests, the effective QLab closed-loop update period was observed to be approximately 0.0667 s. Therefore, the NMPC sampling time was selected as $T_s = 0.07 \text{ s}$. With $N = 13$, this provides a prediction interval comparable to that used in the experimental implementation.

The simulation is not terminated solely according to the prescribed trajectory duration. Once the end of the reference trajectory is reached, the final reference pose is maintained until the vehicle satisfies the prescribed goal-position and goal-heading tolerances

$$e_{p,\text{goal}} \leq 0.10 \text{ m}, \quad |e_{\psi,\text{goal}}| \leq 0.175 \text{ rad}.$$

With the selected OCP weights, these simulation results show that the NMPC controller successfully follows the reference while remaining close to its state references and also maintaining the control inputs within the prescribed physical constraints. On the other hand, the computational time stays below the selected sampling time $T_s = 0.07 \text{ s}$. Among the tested parameter configurations, these OCP weights provided the best overall simulation performance and were therefore selected for the experimental implementation.

The simulation results are provided in Figures A.1–A.4 in the Annexes.

7.1.2. FLMPC

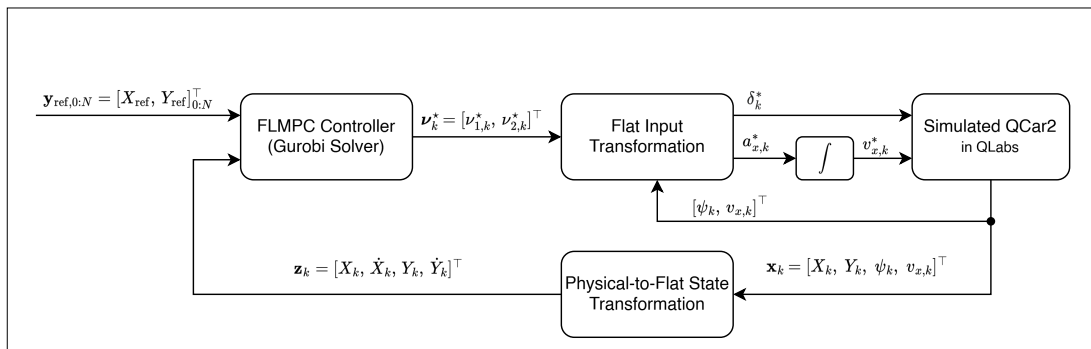


Figure 7.2: Closed-loop FLMPC simulation architecture in Quanser Interactive Labs.

The closed-loop FLMPC simulation architecture is shown in Figure 7.2. Unlike the NMPC, the FLMPC optimization problem is formulated in the flat coordinates introduced in Section 6.2. At each control step, the current physical state $\mathbf{x}_k = [X_k, Y_k, \psi_k, v_{x,k}]^\top$ is transformed into the flat state $\mathbf{z}_k = [X_k, \dot{X}_k, Y_k, \dot{Y}_k]^\top$.

The FL MPC tracking error is defined only from the Cartesian position as

$$\mathbf{e}_k = [X_k - X_{\text{ref},k}, Y_k - Y_{\text{ref},k}]^\top.$$

The optimization returns the first optimal virtual input $\boldsymbol{\nu}_k^* = [\nu_{1,k}^*, \nu_{2,k}^*]^\top$, which is transformed using the current vehicle state to obtain the physical inputs δ_k^* and $a_{x,k}^*$. The virtual input is constrained by the state-dependent set $W(v_x, \psi)$ defined in (6.21) and further analyzed in Section 6.4. This constraint set is updated according to the current longitudinal velocity and heading angle.

A similar warm-start strategy is applied to the predicted flat-state and virtual-input sequences.

The parameters specific to the FL MPC simulation are

$$N = 13, \quad T_s = 0.07 \text{ s}, \quad \epsilon = 10^{-3} \quad \text{and} \quad Q = \text{diag}(120, 120), \quad R = \text{diag}(5.0, 5.0).$$

The physical limits and the goal-position tolerance are retained from the NMPC simulation in Section 7.1.1, while no goal-heading tolerance is imposed.

The simulation results show that the FL MPC can follow the reference trajectory well even though the cost function only uses the Cartesian position error. The steering angle and longitudinal acceleration obtained from the virtual inputs remain within the physical constraints. The computation time is also low during the simulation. Among the tested parameter configurations, these weights gave the best overall results and were therefore selected for the experimental implementation.

The simulation results are provided in Figures A.5–A.8 in the Annexes.

7.1.3. NMPC and FL MPC Comparison

The NMPC and FL MPC are compared under the same reference trajectory, sampling time, prediction horizon, and physical input constraints. The comparison focuses on their tracking performance and computational efficiency.

The simulation results are provided in Figures A.9–A.12 in the Annexes.

Table 7.1: NMPC and FL MPC simulation performance comparison

Controller	RMSE position [m]	Maximum position error [m]	Mean solve time [s]
NMPC	0.0434	0.0910	0.0069
FL MPC	0.0345	0.1340	0.0013

The results show that both controllers are able to follow the reference trajectory while respecting the physical input constraints, as illustrated in Figures A.9 and A.11. In terms of average position-tracking accuracy, the FL MPC achieves a lower RMSE than the NMPC. As shown in Figure A.10 and Table 7.1, the position RMSE is reduced from 0.0434 m to 0.0345 m, while the maximum position error increases from 0.0910 m to 0.1340 m.

The FL MPC shows a clear computational advantage, with a mean value more than 5.2 times lower than that of the NMPC, as shown in Table 7.1. These results show that the FL MPC achieves higher computational efficiency while maintaining nearly comparable position-tracking accuracy to the NMPC in this simulation, which is encouraging since the addition of obstacle-avoidance constraints will further increase the complexity of the optimization problem.

7.2. Obstacle Avoidance

7.2.1. OAMPC

The obstacle-avoidance MPC considered in this section is built directly on the FL MPC formulation presented in Section 7.1.2. The linear flat-space prediction model, virtual-input formulation, and physical input constraints are retained, while the mixed-integer obstacle-avoidance constraints introduced in Section 6.3 are added to the optimization problem. The resulting controller therefore solves the OAMPC problem given in (6.26).

In this simulation, the positions and dimensions of all static obstacles are known in advance and included in the optimization problem at every control step. Two square obstacles are placed directly along the straight reference trajectory. The FL MPC parameters are retained with $N = 13$ and $T_s = 0.07$ s, while the obstacle avoidance formulation uses a safety buffer of $\gamma = 0.35$ m, $M = 100$, and a slack weight of $r_s = 10^8$. Since a total of eight half-space edges represent the two obstacles, the prediction horizon of $N = 13$ results in 104 binary variables in each OAMPC optimization problem.

We slightly adjusted the OCP weights in this simulation compared with those used in the FL MPC to ensure successful obstacle avoidance. All other FL MPC parameters and physical input constraints were retained as before.

$$Q = \text{diag}(160, 160), \quad R = \text{diag}(2.0, 2.0).$$

The simulation results show that the OAMPC successfully avoids both obstacles while following the reference trajectory. The mean solve time is 0.0127 s, while it increases to a maximum of 0.0521 s near the obstacles, approximately 4.1 times higher than the mean value.

This increase in computation time becomes more critical as the number of obstacles grows. With $N = 13$, each additional four-edge obstacle introduces 52 binary variables, so 40 and 50 obstacles would require 2080 and 2600 binary variables, respectively. Therefore, directly including all obstacles in the optimization problem can quickly become computationally demanding.

These results show that the complete OAMPC formulation is suitable for obstacle avoidance in a sparse environment, but directly considering every obstacle at every control step does not scale efficiently to more complex environments. This motivates the obstacle-activation strategy introduced in the following section, in which only obstacles relevant to the current vehicle motion are included in the optimization problem.

The simulation results are provided in Figures A.13–A.16 in the Annexes.

7.2.2. Obstacle Activation

We consider the complete set of static obstacles in the environment, whose positions and geometries are known in advance:

$$\mathcal{O} = \{\mathcal{Q}_1, \mathcal{Q}_2, \dots, \mathcal{Q}_{n_{\text{obs}}}\}.$$

To select the relevant obstacles, the maximum number of simultaneously active obstacles is limited by $N_{\text{act,max}}$. The activation region is defined using the activation radius r_a , the field of view ϕ_{FOV} , and the current vehicle pose. Let

$$\mathbf{p}_k = \begin{bmatrix} X_k \\ Y_k \end{bmatrix}, \quad \psi_k$$

denote the current tracking point and heading angle of the vehicle. The corresponding activation region is defined as

$$\mathcal{A}_k = \left\{ \begin{bmatrix} X \\ Y \end{bmatrix} \in \mathbb{R}^2 : \begin{aligned} &(X - X_k)^2 + (Y - Y_k)^2 \leq r_a^2, \\ &(X - X_k) \cos \psi_k + (Y - Y_k) \sin \psi_k \\ &\geq \sqrt{(X - X_k)^2 + (Y - Y_k)^2} \cos \left(\frac{\phi_{\text{FOV}}}{2} \right) \end{aligned} \right\}. \quad (7.1)$$

The complete obstacle footprint is considered for activation. For each obstacle $\mathcal{Q}_i \in \mathcal{O}$, its footprint is the entire convex polygon defined by its vertices:

$$\mathcal{Q}_i = \text{conv} \{ \mathbf{q}_{i,1}, \mathbf{q}_{i,2}, \dots, \mathbf{q}_{i,n_{v,i}} \}, \quad \mathbf{q}_{i,l} = \begin{bmatrix} X_{i,l} \\ Y_{i,l} \end{bmatrix}, \quad l = 1, \dots, n_{v,i}, \quad (7.2)$$

where $n_{v,i}$ is the number of vertices of obstacle $\mathcal{Q}_i$. An obstacle satisfies the activation condition when its footprint intersects the activation region:

$$\mathcal{Q}_i \cap \mathcal{A}_k \neq \emptyset. \quad (7.3)$$

If more than $N_{\text{act,max}}$ obstacles satisfy this condition, priority is given to the closest obstacles. The minimum distance from the current tracking point to the footprint of obstacle $\mathcal{Q}_i$ is defined as

$$d_i^{\min} = \text{dist}(\mathbf{p}_k, \mathcal{Q}_i) = \min_{\mathbf{q} \in \mathcal{Q}_i} \|\mathbf{q} - \mathbf{p}_k\|_2.$$

The selected obstacles form the active subset

$$\overline{\mathcal{O}} = \{ \mathcal{Q}_{i_1}, \mathcal{Q}_{i_2}, \dots, \mathcal{Q}_{i_{N_{\text{act}}}} \} \subseteq \mathcal{O}, \quad N_{\text{act}} \leq N_{\text{act,max}}. \quad (7.4)$$

Once activated, an obstacle remains in $\overline{\mathcal{O}}$ until its complete footprint has passed behind the vehicle by at least the predefined distance. The deactivation condition can be expressed as

$$\max_{\mathbf{q} \in \mathcal{Q}_i} [(X - X_k) \cos \psi_k + (Y - Y_k) \sin \psi_k] \leq -\gamma, \quad \mathbf{q} = \begin{bmatrix} X \\ Y \end{bmatrix}. \quad (7.5)$$

The mixed-integer obstacle-avoidance constraints are instantiated only for obstacles belonging to the active set $\overline{\mathcal{O}}$. Therefore, for each active obstacle $\mathcal{Q}_i \in \overline{\mathcal{O}}$,

$$\boldsymbol{\alpha}_{i,k+j+1} \in \{0, 1\}^{n_{e,i}}, \quad j = 0, \dots, N-1. \quad (7.6)$$

Obstacles outside the active set do not participate in the OAMPC constraints and therefore introduce no binary decision variables at the current control step.

For the following simulation, the obstacle-activation parameters are selected as

$$r_a = 2.0 \text{ m}, \quad \phi_{\text{FOV}} = 180^\circ, \quad N_{\text{act,max}} = 4.$$

Compared with the OAMPC simulation in Section 7.2.1, the prediction horizon, sampling time, and the velocity constraint are adjusted as

$$N = 30, \quad T_s = 0.02 \text{ s}, \quad 0 \leq v_x \leq 2.0 \text{ m/s}.$$

All other OAMPC parameters and input constraints are retained from Section 7.2.1.

About the obstacle configuration, 100 obstacles are distributed along the reference trajectory using different arrangements. With four edges per obstacle and $N = 30$, considering all obstacles simultaneously would require 12000 binary variables. The proposed activation strategy limits this number to at most 480 binary variables by allowing only $N_{\text{act,max}} = 4$ obstacles to be active simultaneously.

The simulation results are provided in Figures A.17–A.21 in the Annexes.

Before applying the proposed obstacle-activation strategy, the OAMPC formulation from Section 7.2.1 was tested on the same reference trajectory and obstacle arrangement. In this configuration, when the number of obstacles exceeded 8, the vehicle was no longer able to reliably track the reference trajectory while avoiding all obstacles.

The simulation results show that the OAMPC successfully follows the reference trajectory while avoiding the obstacles distributed along the path. As shown in Figure A.17, the vehicle deviates from the reference trajectory when necessary to avoid nearby obstacles and returns toward the reference after passing them.

Figure A.18 confirms the operation of the proposed activation strategy. Although the environment contains 100 obstacles, the number of simultaneously active obstacles never exceeds the prescribed limit $N_{\text{act,max}} = 4$. The mean number of active obstacles during the simulation is approximately 3, showing that only a small subset of locally relevant obstacles is considered by the OAMPC at each control step.

The vehicle states and control inputs are shown in Figures A.19 and A.20. The longitudinal velocity remains non-negative, while the steering angle and longitudinal acceleration remain within their prescribed limits. Larger control actions occur mainly when the vehicle performs avoidance maneuvers around the obstacles.

The computational time is shown in Figure A.21. The mean solve time is 0.0265 s, while the maximum solve time reaches 0.1311 s. The solver successfully finds an optimal solution at every control step. In general, the computational effort increases when more obstacles are simultaneously active, since a larger number of binary variables participate in the mixed-integer optimization.

8. Experimental Results

8.1. Reference Trajectory Tracking

8.1.1. NMPC

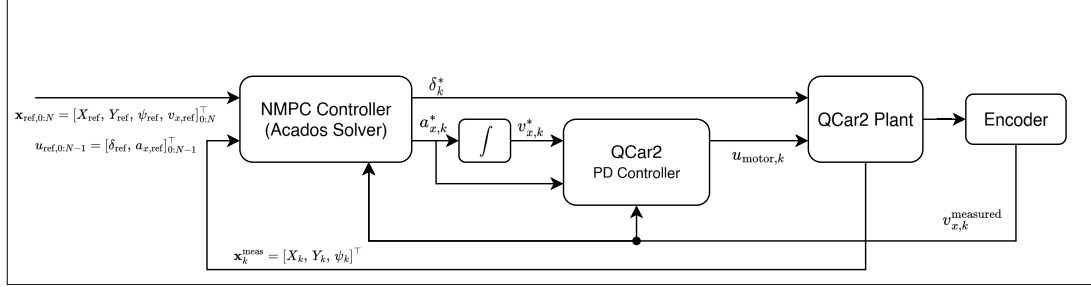


Figure 8.1: Closed-loop NMPC experimental architecture on the QCar2.

Following the NMPC implementation presented in Section 7.1.1, the controller was deployed on the physical QCar2 according to the closed-loop experimental architecture shown in Figure 8.1. At each control step, the measured vehicle state

$$\mathbf{x}_k^{\text{meas}} = [X_k \ Y_k \ \psi_k \ v_{x,k}]^\top$$

and the reference state and input trajectories over the prediction horizon are supplied to the NMPC. The vehicle position and heading angle are obtained from the localization transform between the `map` and `base_link` frames, while the longitudinal velocity is obtained from the encoder feedback.

After solving the OCP, the first optimal steering angle δ_k^* is applied to the steering actuator. The optimal longitudinal acceleration $a_{x,k}^*$ is integrated at each control step to update the desired longitudinal velocity $v_{x,k}^{\text{cmd}}$. The desired velocity and acceleration are then provided to the QCar2 PD speed controller, which uses the measured longitudinal velocity to generate the motor command applied to the vehicle.

The experimental setup consists of the physical QCar2 and a ground workstation running Linux. The workstation is used to visualize the vehicle and the reference trajectory in RViz2 using a previously scanned map. The control algorithm itself is executed onboard the QCar2, while the ground workstation is used to access and supervise the experimental platform.

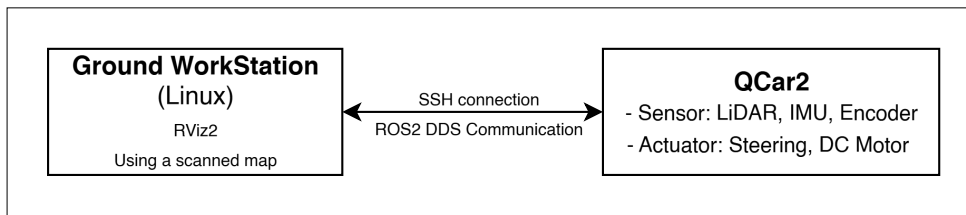


Figure 8.2: Experimental setup with the ground workstation and the QCar2.

For the experimental implementation, the NMPC OCP was generated using *acados* and integrated directly into the ROS2 C++ controller. *acados* is an optimization framework designed for efficient numerical optimal control and is suitable for real-time MPC implementation. It was therefore selected for the experimental controller to achieve a shorter computation time on the onboard computer.

The parameters used for the experimental NMPC are

$$N = 30, \quad T_s = 0.03 \text{ s}, \quad L_{wb} = 0.25725 \text{ m}.$$

Compared with the simulation parameters $N = 13$ and $T_s = 0.07 \text{ s}$, the experimental NMPC uses $N = 30$ and $T_s = 0.03 \text{ s}$, resulting in a prediction interval of 0.90 s , which is nearly identical to the 0.91 s used in simulation. This increases the control frequency while preserving a similar prediction window. The onboard PD speed controller operates with a sampling time of 0.015 s , which is twice as fast as the NMPC.

The OCP weighting matrices, physical constraints, and goal-position and goal-heading tolerances are retained from the NMPC simulation in Section 7.1.1.

Two experimental configurations are considered for the NMPC implementation. In the first configuration, denoted Mk1, a new reference trajectory is generated from the current QCar2 pose to a user-defined goal pose for each experimental run. In the second configuration, denoted Mk2, a predefined reference trajectory is used directly by the controller.

For the Mk1 implementation, the current QCar2 pose is used as the initial pose, while the desired final pose is selected in RViz2 using the 2D Goal tool. The flatness-based Bézier trajectory-generation method introduced in Chapter 5 is then used to construct the reference trajectory between the initial and final poses.

Experimental results with online-generated trajectories (NMPC Mk1):

The experimental results are provided in Figures A.22–A.25 in the Annexes.

Table 8.1: NMPC experimental performance over four runs

Run	Position RMSE [m]	Maximum position error [m]	Mean solve time [s]	Maximum solve time [s]
1	0.0909	0.2078	0.0011	0.0024
2	0.1207	0.2822	0.0014	0.0042
3	0.0813	0.1947	0.0014	0.0022
4	0.0809	0.1579	0.0016	0.0034

As summarized in Table 8.1, the experimental results show that the NMPC successfully tracks the generated reference trajectories and reaches the desired final pose in all four runs, including both forward and reverse motion, without solver failures. The position RMSE ranges from 0.0809 m to 0.1207 m , with an average value of approximately 0.0934 m , while the maximum position error remains between 0.1579 m and 0.2822 m . The tracking errors are higher than in simulation, which is expected when moving from the simulated model to the physical platform. Nevertheless, considering the physical dimensions of the QCar2 and the observed experimental behavior, these tracking errors remain acceptable for the current implementation.

The control-input results show that the steering angle and longitudinal acceleration remain within the prescribed physical constraints. Both inputs reach their saturation limits at some parts of the trajectories when larger corrections are required, but the saturation is not maintained continuously and does not prevent the vehicle from completing the tracking task.

The average solve time over the four runs is approximately 0.0014 s, with a maximum of 0.0042 s, remaining well below the sampling time $T_s = 0.03$ s. Overall, these results validate the real-time implementation of the NMPC on the physical QCar2 and show that the complete trajectory-generation and control framework can track different point-to-point trajectories while respecting the physical input constraints.

Experimental results with predefined circular trajectory (NMPC Mk2):

The experimental results are provided in Figures A.26–A.29 in the Annexes.

For the predefined circular trajectory, the NMPC achieves a position RMSE of 0.0691 m and a maximum position error of 0.2433 m. After the initial transition to the circular path, the vehicle maintains stable tracking and reaches the prescribed goal pose within the goal-position and goal-heading tolerances.

The control inputs remain within their physical limits, with larger corrections mainly occurring during the initial transition and curved motion. The mean and maximum solve times are 0.00143 s and 0.00366 s, respectively, both well below $T_s = 0.03$ s, with no solver failures observed.

8.1.2. FLMPC

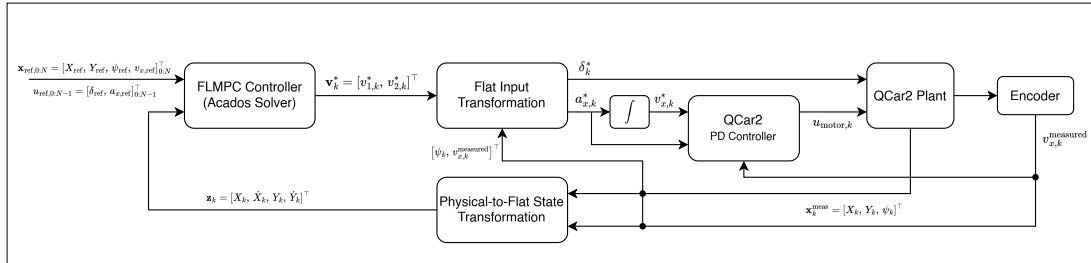


Figure 8.3: Closed-loop FLMPC experimental architecture on the QCar2.

Following the FLMPC formulation presented in Section 7.1.2, the controller was deployed on the physical QCar2 according to the closed-loop control scheme shown in Figure 8.3. At each control step, the measured vehicle pose

$$\mathbf{x}_k^{\text{meas}} = [X_k \ Y_k \ \psi_k]^\top$$

is obtained from the QCar2 plant, while the measured longitudinal velocity $v_{x,k}^{\text{measured}}$ is provided by the encoder. These measurements are supplied to the physical-to-flat state transformation to construct the current flat state

$$\mathbf{z}_k = [X_k \ \dot{X}_k \ Y_k \ \dot{Y}_k]^\top,$$

where

$$\dot{X}_k = v_{x,k}^{\text{measured}} \cos \psi_k, \quad \dot{Y}_k = v_{x,k}^{\text{measured}} \sin \psi_k.$$

The current flat state and the Cartesian reference trajectory over the prediction horizon are supplied to the FLMPC. After solving the OCP, the controller returns the first optimal virtual input

$$\boldsymbol{\nu}_k^* = [\nu_{1,k}^* \quad \nu_{2,k}^*]^\top.$$

The optimal virtual input, together with the current heading angle ψ_k and the measured longitudinal velocity $v_{x,k}^{\text{measured}}$, is supplied to the flat input transformation. This transformation recovers the corresponding optimal physical inputs δ_k^* and $a_{x,k}^*$.

The reconstructed steering angle and longitudinal acceleration are limited to their prescribed physical bounds. The steering command is then applied to the QCar2, while the longitudinal acceleration is integrated to obtain the desired velocity $v_{x,k}^{\text{cmd}}$. The desired velocity and acceleration, together with the measured longitudinal velocity from the encoder, are supplied to the QCar2 PD speed controller. The resulting motor command $u_{\text{motor},k}$ closes the longitudinal control loop.

The physical experimental setup is identical to that used for the NMPC and is shown in Figure 8.2.

For the experimental implementation, the FLMPC OCP was generated using *acados* and integrated into the ROS2 controller running onboard the QCar2. The parameters used for the experimental FLMPC are

$$N = 30, \quad T_s = 0.03 \text{ s}, \quad L_{\text{wb}} = 0.25725 \text{ m}, \quad \epsilon = 10^{-3}.$$

The weighting matrices are

$$Q = \text{diag}(120, 120), \quad R = \text{diag}(15, 15).$$

The physical limits used in the experimental implementation are

$$-2.0 \leq v_x \leq 2.0 \text{ m/s}, \quad -0.5236 \leq \delta \leq 0.5236 \text{ rad}, \quad -1.0 \leq a_x \leq 1.0 \text{ m/s}^2.$$

The steering-angle and longitudinal-acceleration limits are enforced in the FLMPC through the state-dependent virtual-input set $W(v_x, \psi)$, while the reconstructed inputs are additionally saturated before being applied to the vehicle. The desired longitudinal velocity is also limited after integrating the optimal acceleration. The goal-position tolerance is 0.10 m, with no goal-heading tolerance imposed.

The experimental OCP parameters were initialized from the FLMPC simulation configuration in Section 7.1.2 and subsequently refined through repeated experimental trials. In particular, the virtual-input weighting matrix was increased from $R = \text{diag}(5, 5)$ in simulation to $R = \text{diag}(15, 15)$, which provided the best overall behavior among the tested experimental configurations.

To maintain a consistent comparison with the NMPC, the FLMPC is evaluated using the same two trajectory configurations. In Mk1, a new reference trajectory is generated online from the current QCar2 pose to a user-defined goal pose for each experimental run, whereas Mk2 uses a predefined reference trajectory directly.

For the Mk1 implementation, the current QCar2 pose defines the initial pose, while the desired final pose is specified in RViz2 using the 2D Goal tool. The flatness-based Bézier trajectory-generation method introduced in Chapter 5 is then used to generate the reference trajectory between the initial and final poses.

Experimental results with online-generated trajectories (FLMPC Mk1):

The experimental results are provided in Figures A.30–A.33 in the Annexes.

Table 8.2: FL MPC experimental performance over four runs

Run	Position RMSE [m]	Maximum position error [m]	Mean solve time [s]	Maximum solve time [s]
1	0.1119	0.2000	0.0012	0.0027
2	0.2223	0.4294	0.0014	0.0035
3	0.0417	0.0930	0.0013	0.0027
4	0.0529	0.1542	0.0013	0.0046

As summarized in Table 8.2, the FL MPC successfully reaches the prescribed final position in all four experimental runs without solver failures. The position RMSE ranges from 0.0417 m to 0.2223 m, with an average value of approximately 0.1072 m. Run 2 presents the largest tracking error, while Runs 3 and 4 achieve considerably smaller position errors.

The steering angle and longitudinal acceleration remain within their prescribed physical limits. The steering angle reaches its saturation limit during some parts of the trajectories, whereas the longitudinal acceleration remains below its imposed bound in all four runs.

The mean solve time remains between approximately 0.0012 s and 0.0014 s, with a maximum recorded value of 0.0046 s. These values remain well below the sampling time $T_s = 0.03$ s, confirming that the FL MPC can be executed in real time on the experimental platform.

Experimental results with predefined circular trajectory (FL MPC Mk2):

For the Mk2 implementation, the RViz2 2D Goal tool is used only as a trigger, while the FL MPC tracks a predefined circular trajectory.

The experimental results are provided in Figures A.34–A.37 in the Annexes.

For this predefined trajectory, the FL MPC achieves a position RMSE of 0.0980 m and a maximum position error of 0.2673 m. The vehicle completes the trajectory and reaches the prescribed final position within the goal-position tolerance without solver failures.

The steering angle remains within its prescribed limit and reaches the saturation value of 0.5236 rad during some curved portions of the trajectory. The maximum absolute longitudinal acceleration is 0.5157 m/s², remaining below its imposed limit.

The mean and maximum solve times are approximately 0.00128 s and 0.00356 s, respectively. Both values remain well below the sampling time $T_s = 0.03$ s, confirming that the FL MPC maintains real-time computational performance when tracking the predefined trajectory.

8.1.3. NMPC and FL MPC Comparison

Because the Mk1 experiments use independently generated reference trajectories for each run, they do not provide a consistent reference for a direct controller comparison. Therefore, we use the predefined circular trajectory from the NMPC Mk2 and FL MPC Mk2 experiments for the following comparison.

The experimental results are provided in Figures A.38–A.42 in the Annexes.

Table 8.3: NMPC and FL MPC experimental performance comparison

Controller	RMSE position [m]	Maximum position error [m]	Mean solve time [s]
NMPC	0.0691	0.2433	0.00143
FL MPC	0.0980	0.2673	0.00128

As summarized in Table 8.3, both controllers successfully track the same predefined circular trajectory while respecting the physical input constraints. The NMPC achieves better position-tracking accuracy, with a position RMSE of 0.0691 m compared with 0.0980 m for the FL MPC. The maximum position errors are 0.2433 m and 0.2673 m, respectively. Both controllers reach the steering-angle limit during the experiment, while the FL MPC uses a smaller maximum longitudinal acceleration.

In terms of computational performance, the FL MPC achieves a lower mean solve time of 0.00128 s, compared with 0.00143 s for the NMPC. The difference is smaller than that observed in simulation, but both controllers remain well below the sampling time $T_s = 0.03$ s. Overall, the FL MPC achieves slightly lower tracking accuracy than the NMPC in this experiment, while maintaining comparable real-time performance. However, the obtained tracking performance remains satisfactory and provides a promising basis for extending the FL MPC framework toward experimental obstacle avoidance.

9. Conclusion and Future Directions

9.1. Conclusion

This project addressed the main specifications defined in Section 2.2 for the development and validation of optimization-based control strategies on the Quanser QCar2. The QCar2 software and hardware environments were used in both simulation and experimental conditions, and smooth reference trajectories were generated using a flatness-based Bézier formulation. An NMPC controller was implemented for trajectory tracking, followed by the development of an FLMPC formulation and its comparison with the NMPC. The flatness-based approach was then extended to mixed-integer obstacle avoidance and its computational scalability was investigated. The experimental validation focused on the NMPC and FLMPC trajectory-tracking controllers. Experimental implementation of the OAMPC was not completed because the mixed-integer formulation remained computationally demanding for real-time operation and the remaining internship period did not allow sufficient time for further optimization and reliable experimental validation.

In simulation, both the NMPC and FLMPC successfully tracked the reference trajectories while respecting the imposed physical constraints. The comparison showed that the flatness-based formulation can provide satisfactory tracking performance while using a simpler linear prediction model. This formulation also expresses the vehicle motion directly in Cartesian flat-output coordinates, which facilitates the introduction of geometric constraints for obstacle avoidance.

Based on this formulation, the FLMPC was extended to the OAMPC by introducing mixed-integer constraints for static-obstacle avoidance. The controller successfully combined trajectory tracking and obstacle avoidance in simulation. However, its computational complexity increased as more obstacles and binary variables were included in the optimization problem. The proposed obstacle-activation strategy reduced this complexity by considering only the obstacles that were locally relevant to the vehicle. These results demonstrated the potential of the approach while also identifying computational scalability as the main limitation for its real-time experimental deployment.

The experimental implementation of the NMPC and FLMPC on the physical QCar2 confirmed that both controllers can be deployed within the developed ROS2 and acados control architecture. Both approaches successfully completed the trajectory-tracking experiments while respecting the vehicle constraints. The experimental comparison showed that the NMPC provided better tracking accuracy, whereas the FLMPC retained the advantages of a simpler prediction model and a formulation that can be naturally extended toward obstacle avoidance.

Overall, the results obtained during the project are consistent with the initial specifications. The work provides LCIS with a complete simulation and experimental basis for studying MPC-based trajectory tracking on the QCar2, as well as a simulation framework for further investigation of optimization-based obstacle avoidance. The main remaining challenges concern the behavior of the flatness-based formulation at low velocity and the computational complexity of mixed-integer obstacle avoidance for real-time implementation. These limitations define the main directions for future development presented in the following section.

9.2. Future Directions

A first limitation of the flatness-based formulation concerns low-speed operation. As discussed in Section 6.2, the reconstruction of the steering input becomes increasingly sensitive when the longitudinal velocity approaches zero. Although regularization avoids numerical singularity, large steering corrections and saturation can still occur at low velocity.

The main challenge for future obstacle-avoidance experiments is the computational burden of the MIQP. Even with obstacle activation, the simulation reached a mean solve time of 0.0265 s and a maximum of 0.1311 s. Preliminary tests on the physical QCar2 showed that the vehicle could initiate an avoidance maneuver, but the prediction horizon had to remain too short to reliably complete the maneuver around the obstacle. Increasing the horizon would also increase the number of mixed-integer decisions and therefore the online computation time.

A possible direction for reducing this online complexity is the Control Lyapunov Function (CLF) approach proposed in [11]. In this method, an MPC controller is used offline to generate admissible state-control trajectories, which are then exploited to construct a CLF through linear programming. Once the CLF has been obtained, a lower-complexity controller can be implemented online while retaining stability information obtained from the original MPC controller.

This idea is further developed in [12] through the Sub-Control Lyapunov Function (SCLF). A high-performance but computationally expensive controller is retained during the offline synthesis, while the resulting SCLF is used to derive a simpler feedback law for online implementation. The method therefore shifts a significant part of the computational effort from the real-time control loop to the offline design stage while providing stability and performance guarantees.

For the QCar2, future work could investigate using the developed OAMPC as the offline reference controller, where longer prediction horizons could be used without being limited by the real-time sampling period. The resulting trajectories and control actions could then support the synthesis of a CLF or SCLF for a computationally lighter online controller. Since the methods in [11, 12] were not developed specifically for mixed-integer obstacle avoidance, an important next step would be to determine how collision-avoidance requirements can be preserved in this reduced-complexity formulation before experimental validation on the QCar2.

Bibliography

- [1] A. L. Gratzner, M. M. Broger, A. Schirrer, and S. Jakubek, “Flatness-Based Mixed-Integer Obstacle Avoidance MPC for Collision-Safe Automated Urban Driving,” in *2023 9th International Conference on Control, Decision and Information Technologies (CoDIT)*, Rome, Italy, 2023, pp. 1844–1849. doi: <https://doi.org/10.1109/CoDIT58514.2023.10284415>.
- [2] C. Tiriolo and W. Lucia, “A Feedback Linearized Model Predictive Control Strategy for Input-Constrained Self-Driving Cars,” arXiv preprint arXiv:2405.01753, 2024. Available: <https://arxiv.org/abs/2405.01753>.
- [3] Quanser, “QCar,” 2025. Available: <https://www.quanser.com/products/qcar-2/>.
- [4] Quanser, “QUARC Real-Time Control Software.” Available: <https://www.quanser.com/products/quarc-real-time-control-software/>.
- [5] Quanser, “QLabs Core Documentation.” Available: https://qlabs.quanserdocs.com/en/latest/System/qlabs_core.html.
- [6] Quanser, “Quanser Academic Resources.” Available: https://github.com/quanser/Quanser_Academic_Resources.git.
- [7] SLAMTEC, “RPLidar A2.” Available: <https://gargantua.ai/product/gargantua-triangulation-lidar-rplidar-a2/>.
- [8] Open Robotics, “Robot Operating System.” Available: <https://www.ros.org/>.
- [9] acados developers, “acados Documentation.” Available: <https://docs.acados.org/index.html>.
- [10] Gurobi Optimization, LLC, “Gurobi Optimizer Reference Manual.” Available: <https://docs.gurobi.com/projects/optimizer/en/current/>.
- [11] H.-T. Do, F. Blanchini, S. Miani, and I. Prodan, “LP-Generated Control Lyapunov Functions With Application to Multicopter Control,” *IEEE Transactions on Control Systems Technology*, vol. 32, no. 6, pp. 2090–2101, 2024. doi: <https://doi.org/10.1109/TCST.2024.3396704>.
- [12] H.-T. Do, F. Blanchini, S. Miani, and I. Prodan, “Reducing Real-Time Complexity via Sub-Control Lyapunov Functions: From Theory to Experiments,” arXiv preprint arXiv:2501.09143, 2025. Available: <https://arxiv.org/abs/2501.09143>.

Annexes

Simulation Results

NMPC Reference Trajectory Tracking

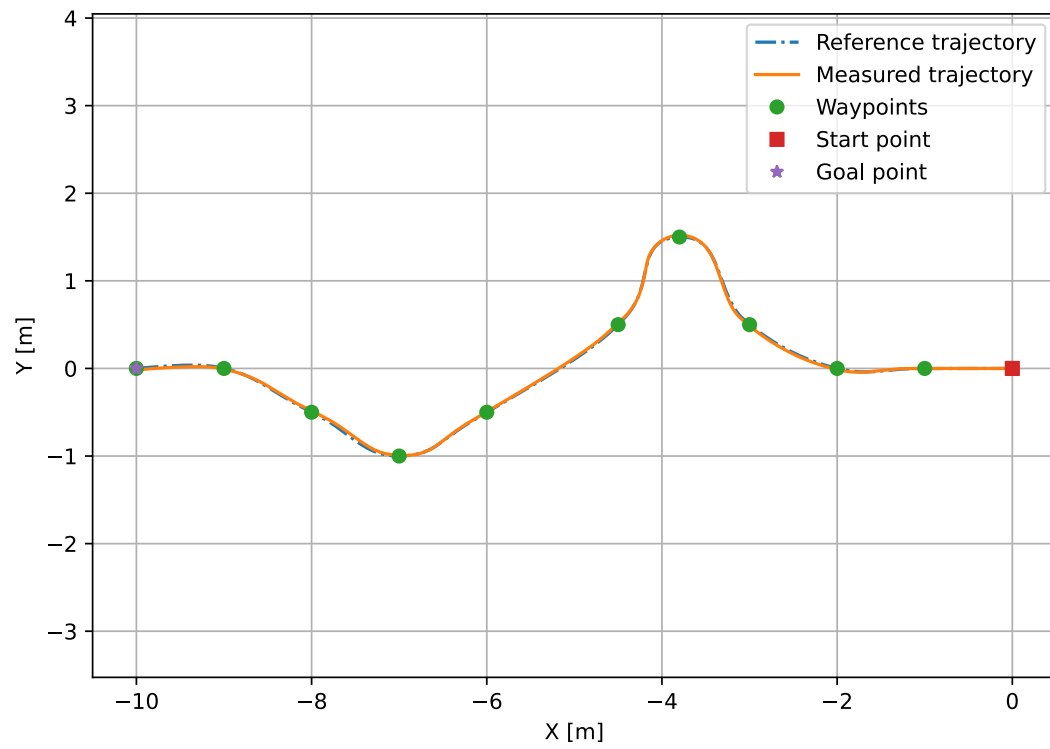


Figure A.1: NMPC simulation trajectory tracking.

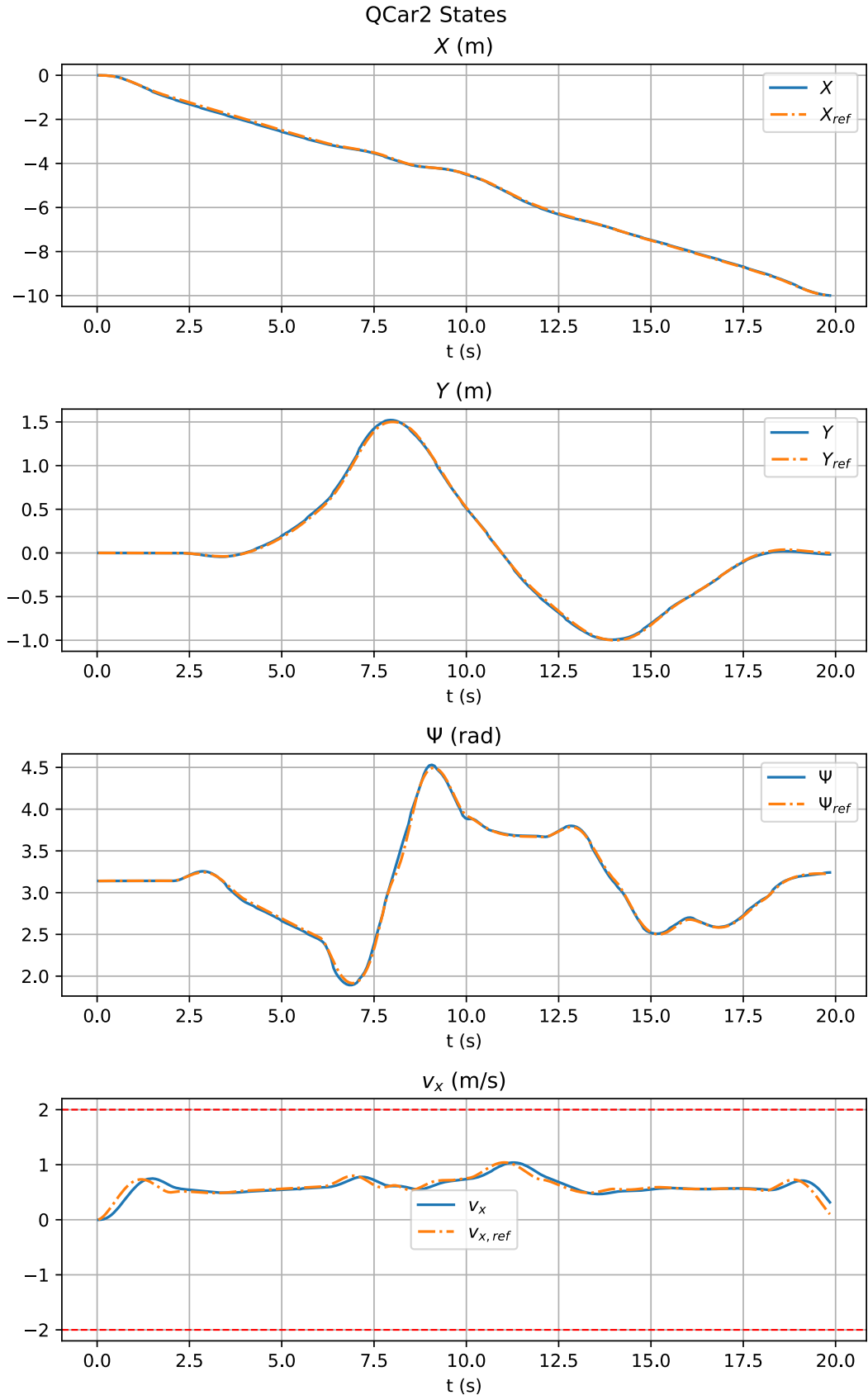


Figure A.2: NMPC simulation state tracking.

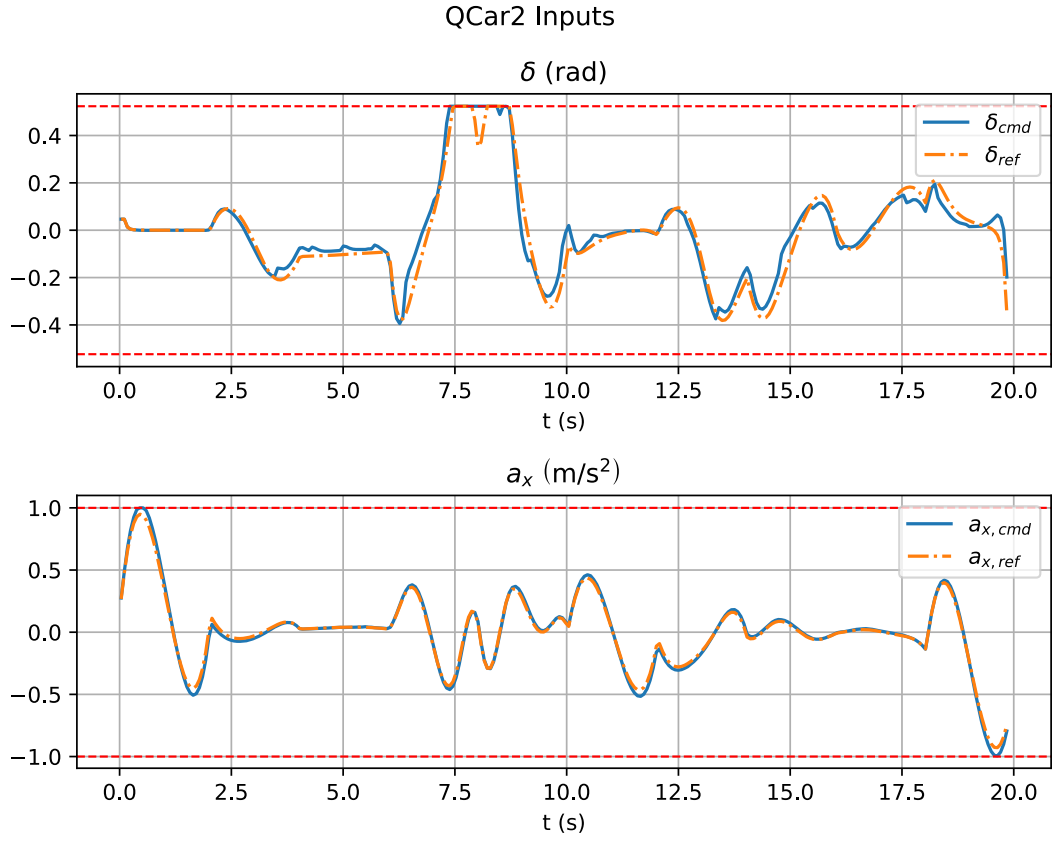


Figure A.3: NMPC simulation control inputs.

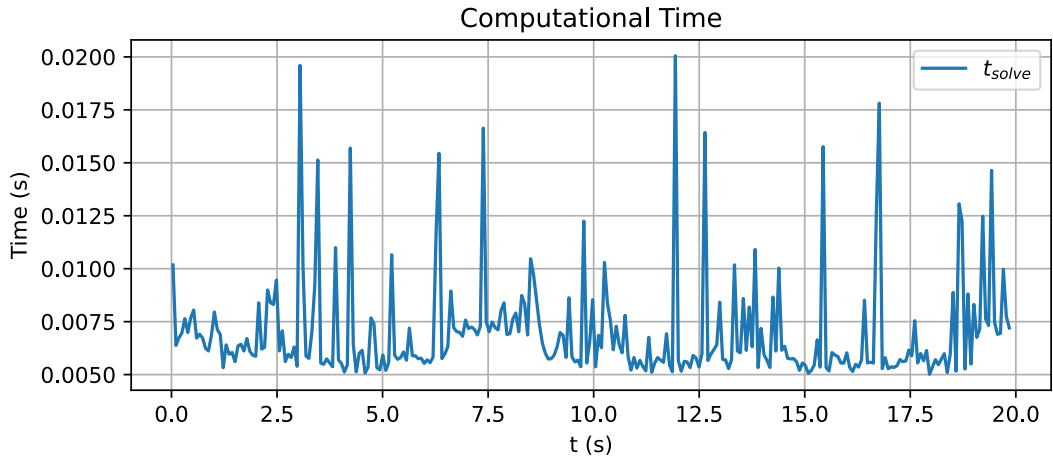


Figure A.4: NMPC simulation computation time.

FLMPC Reference Trajectory Tracking

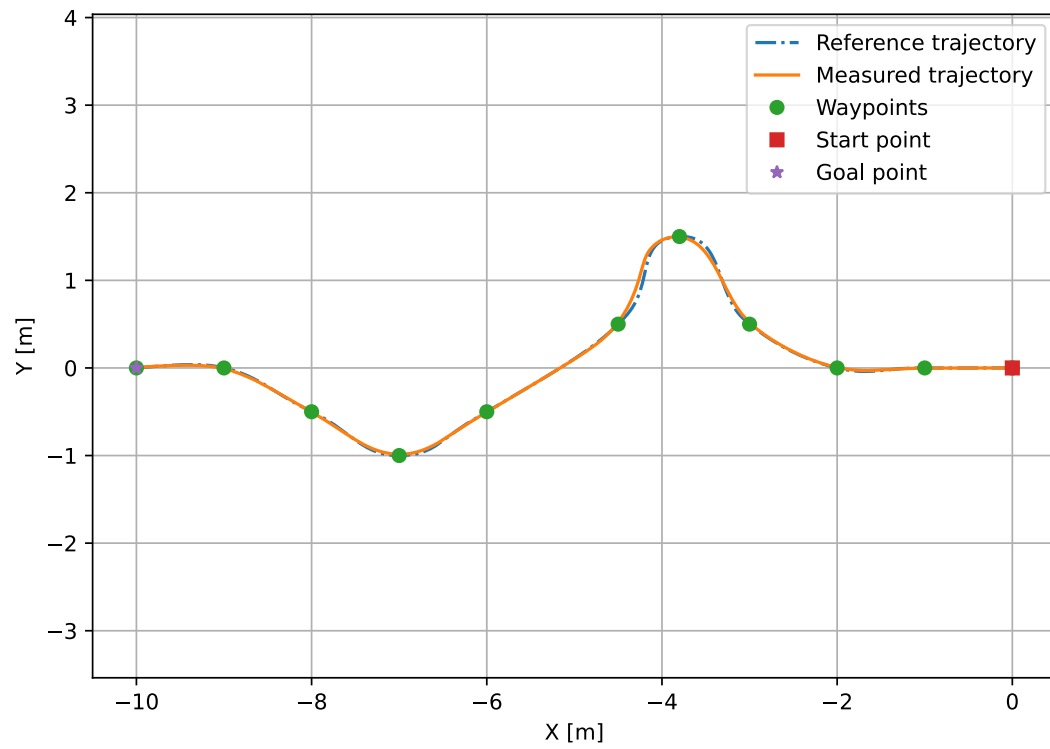


Figure A.5: FLMPC simulation trajectory tracking.

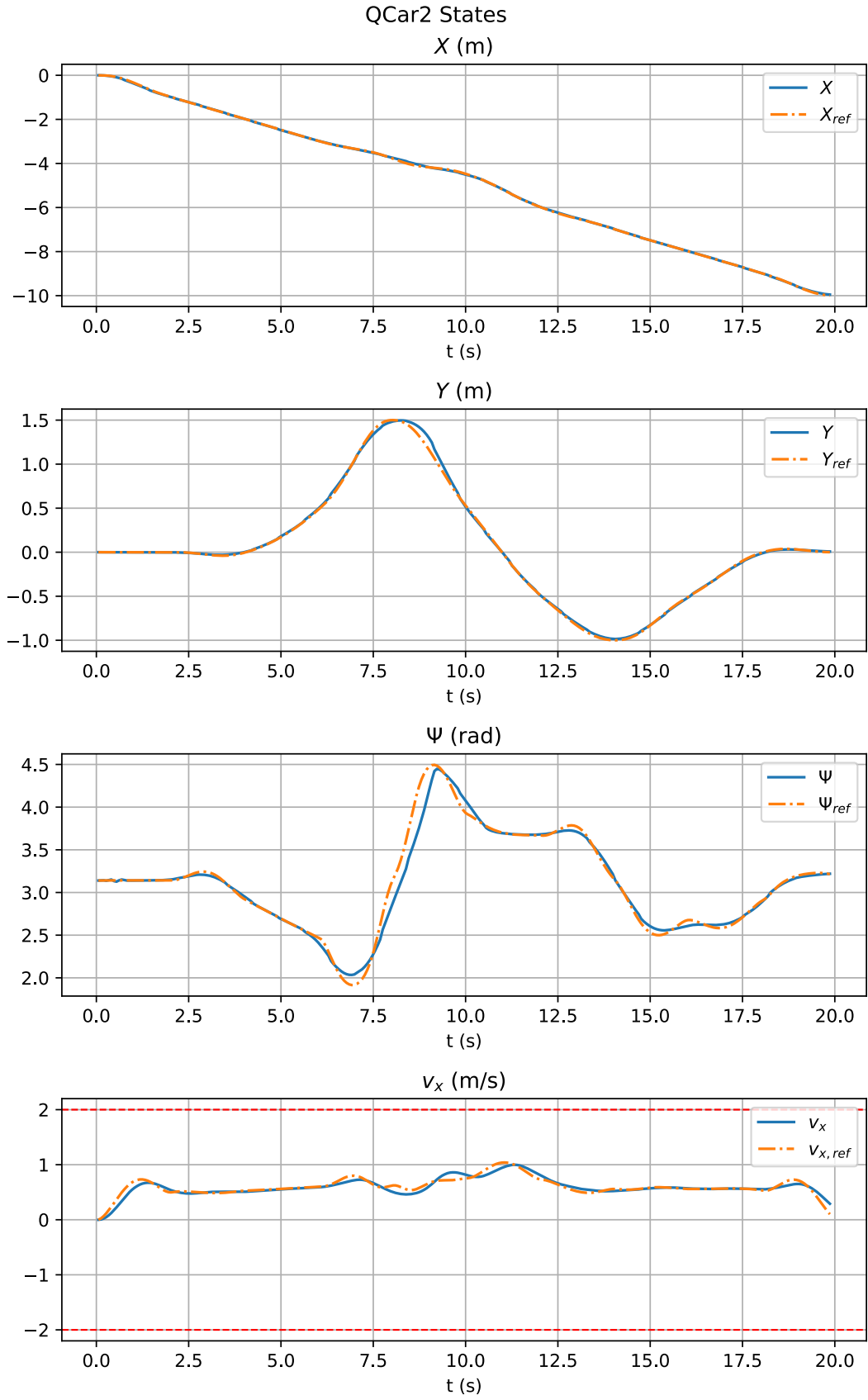


Figure A.6: FLMPC simulation state tracking.

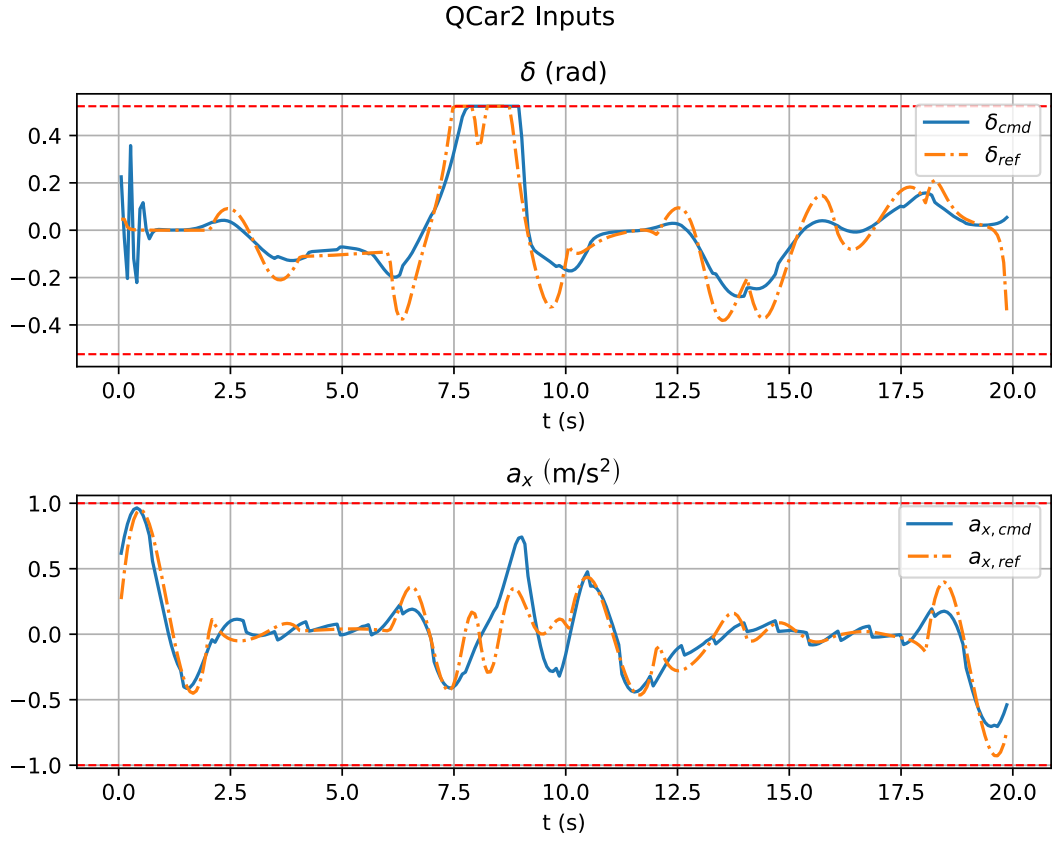


Figure A.7: FLMPc simulation control inputs.

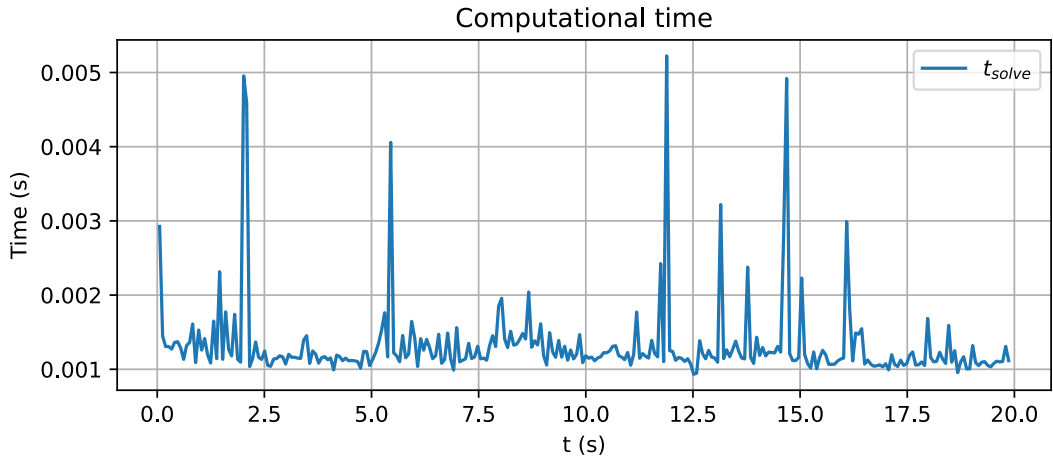


Figure A.8: FLMPc simulation computation time.

NMPC and FLMPC Comparison

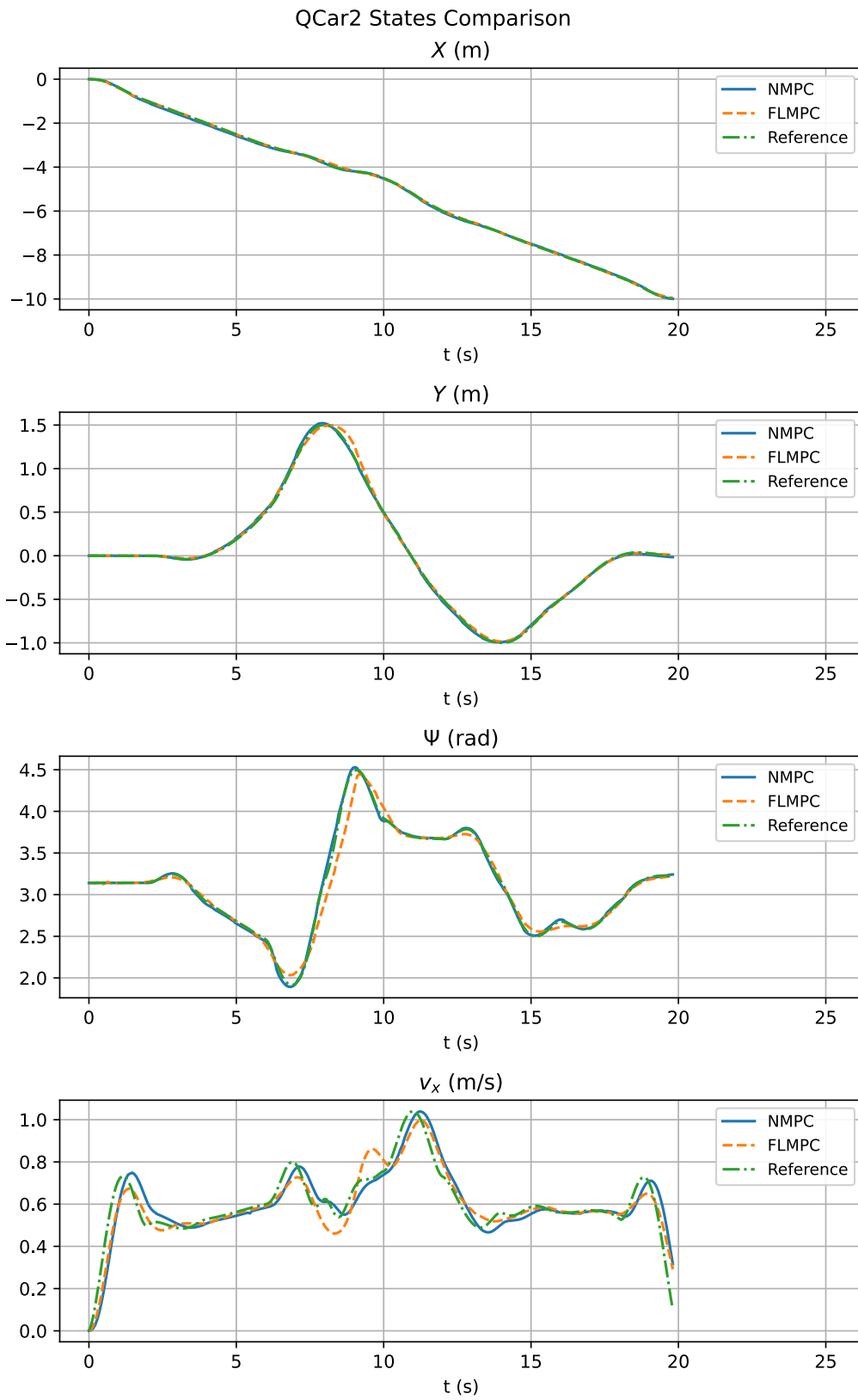


Figure A.9: NMPC and FLMPC simulation state-tracking comparison.

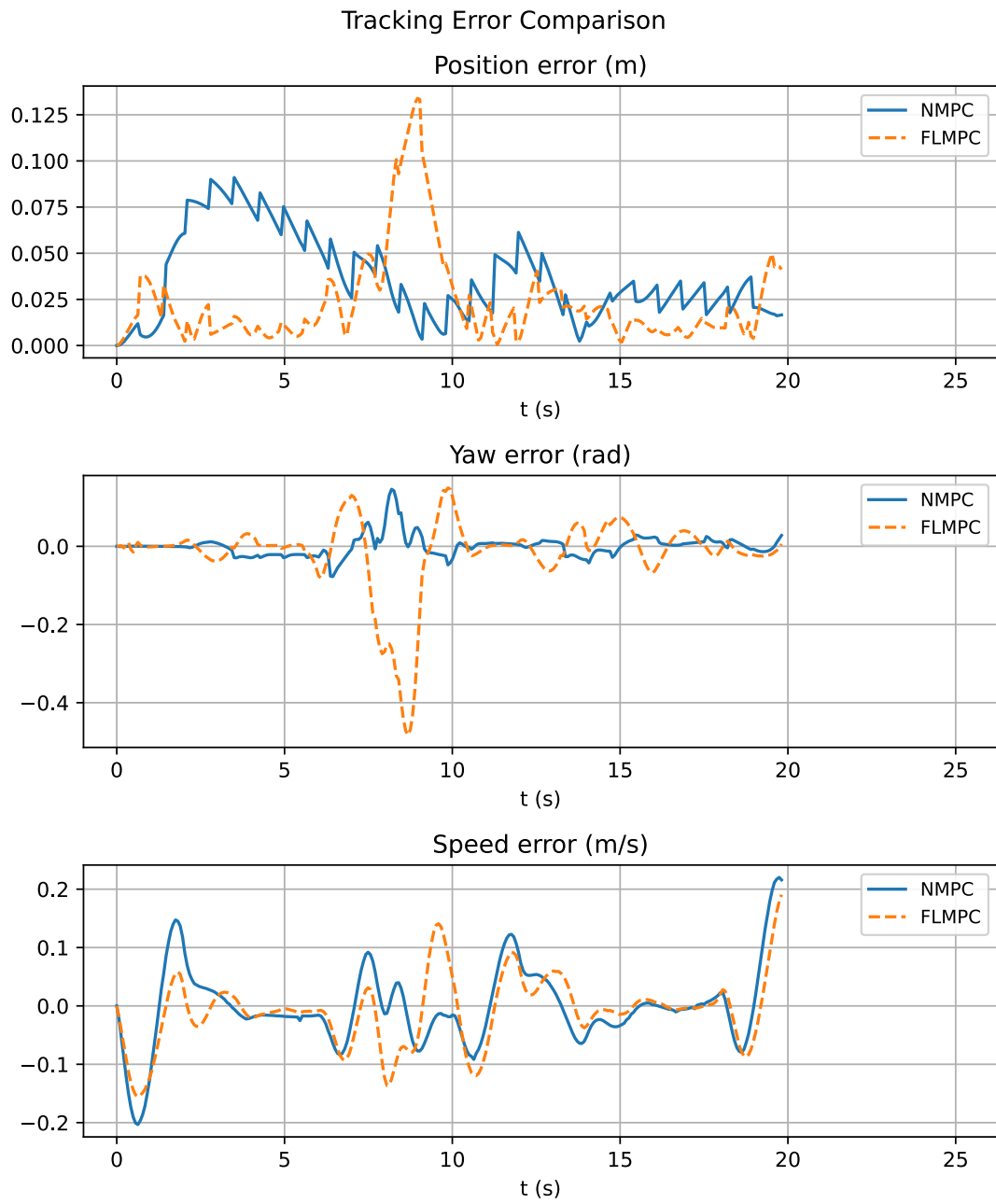


Figure A.10: NMPC and FLMP simulation tracking error comparison.

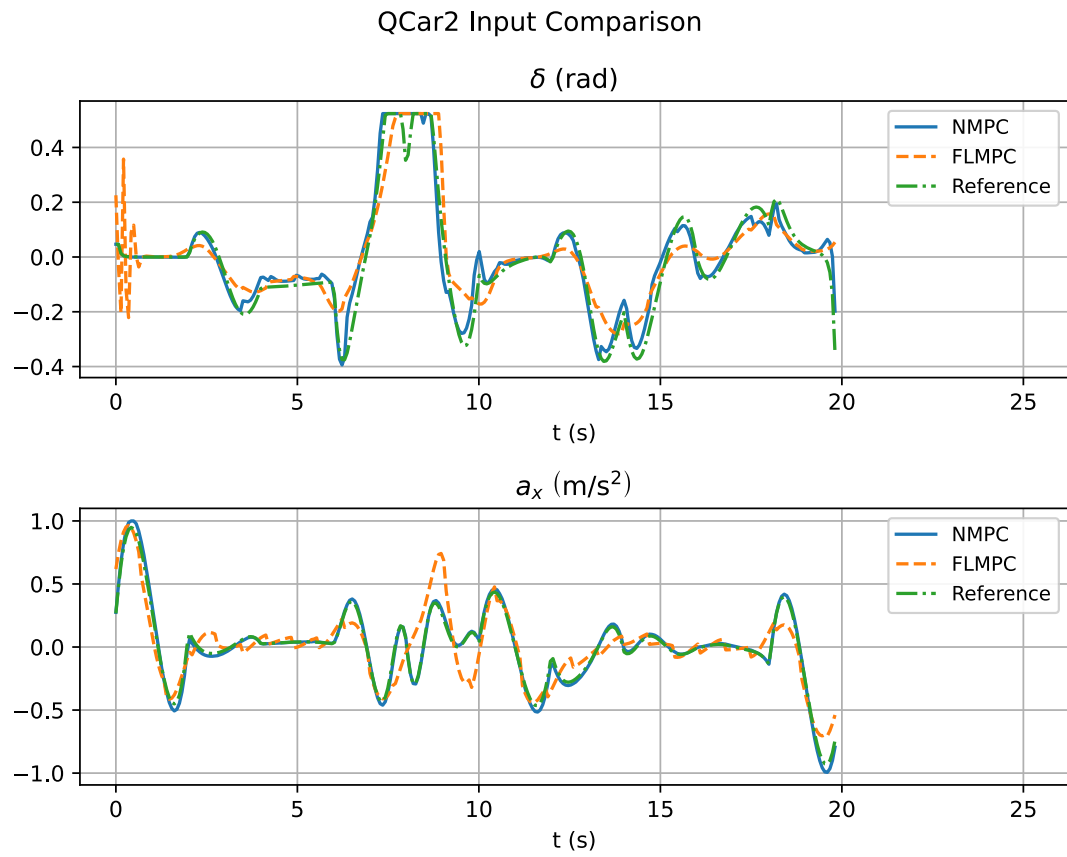


Figure A.11: NMPC and FLMPC simulation control input comparison.

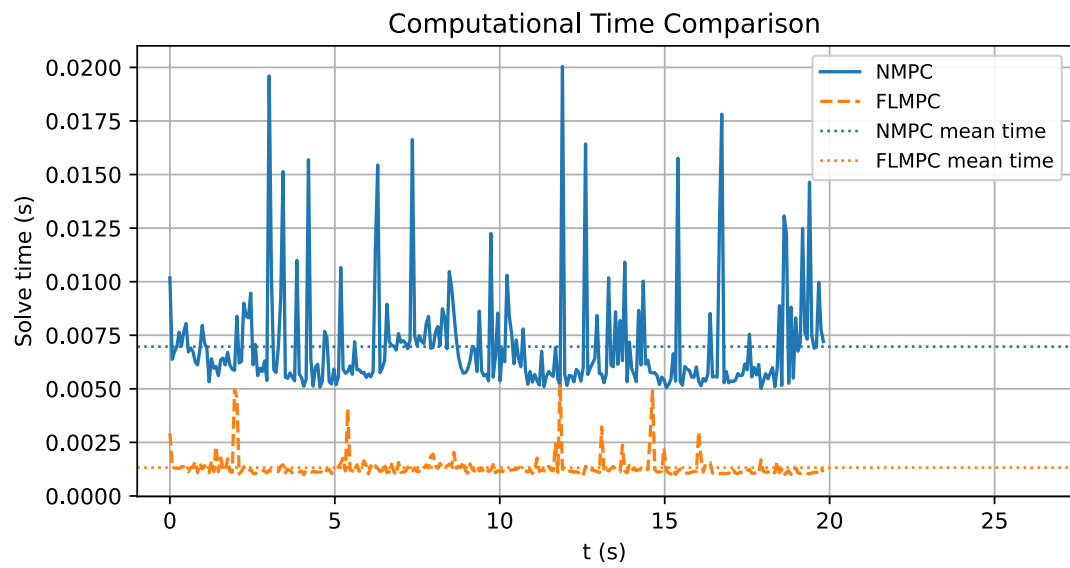


Figure A.12: NMPC and FLMPC simulation computation time comparison.

OAMPC Obstacle Avoidance

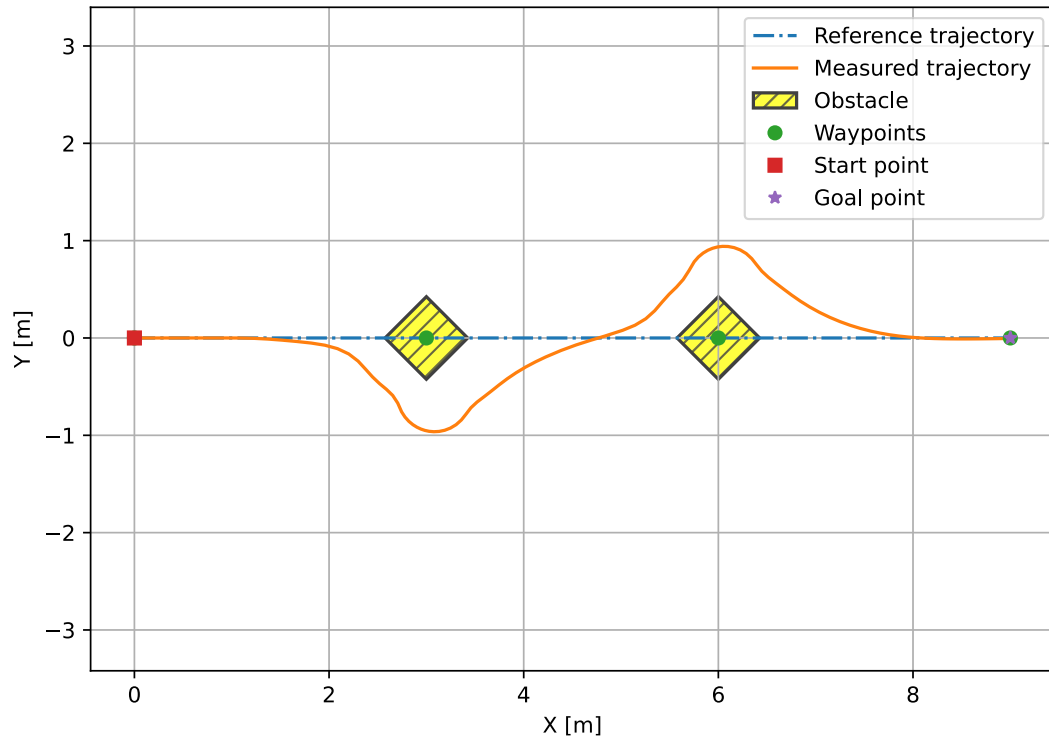


Figure A.13: OAMPC simulation trajectory tracking with obstacle avoidance.

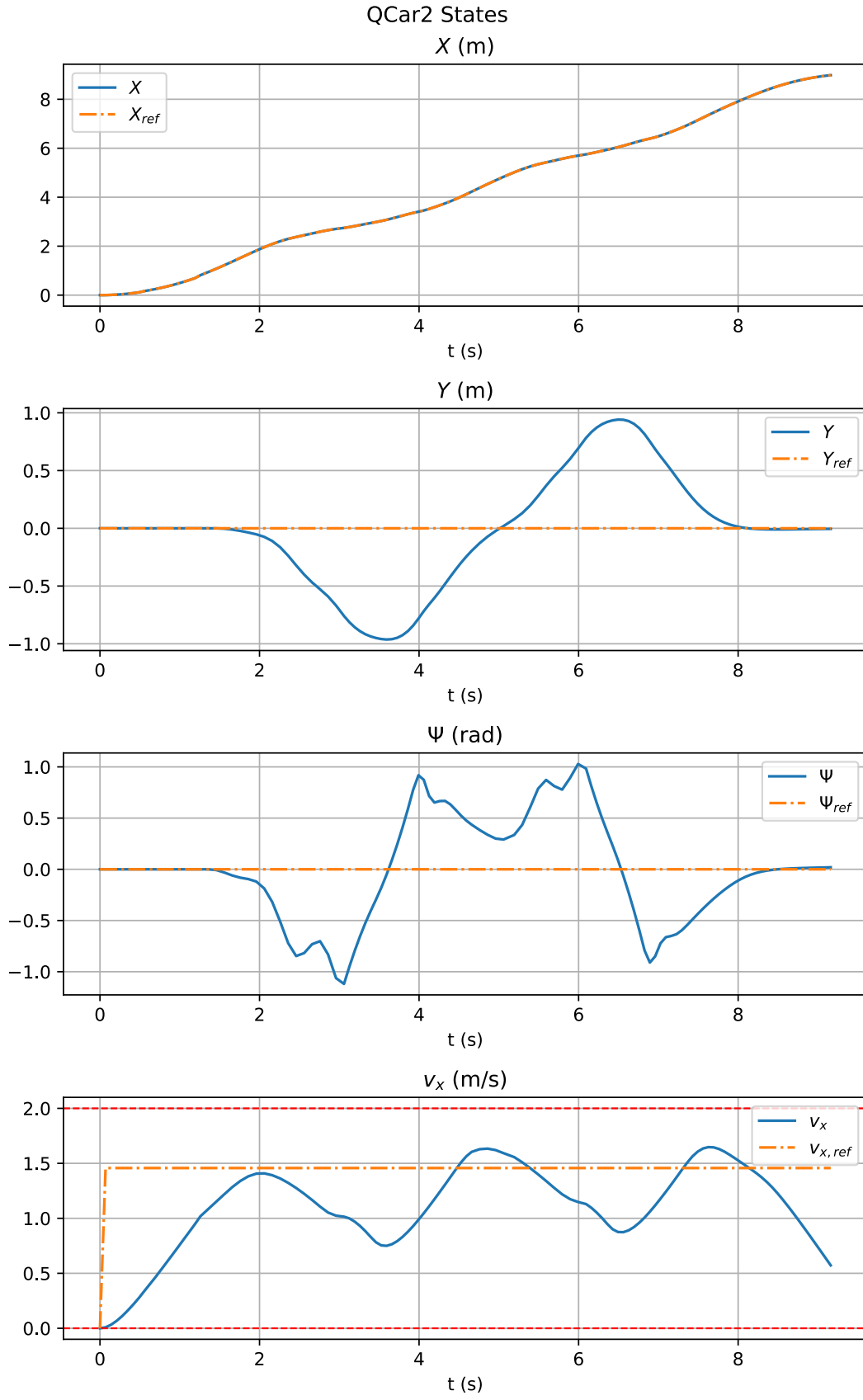


Figure A.14: OAMPC simulation state trajectories.

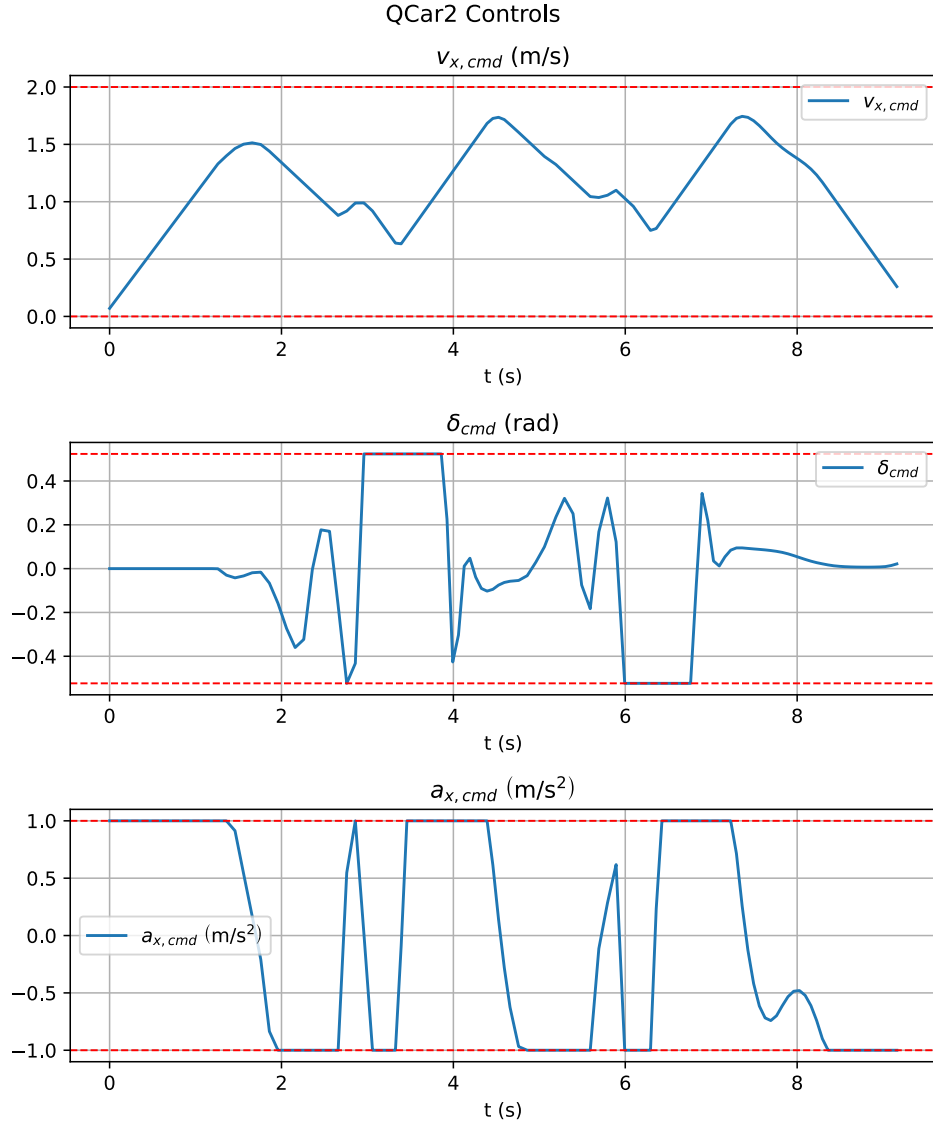


Figure A.15: OAMPC simulation control inputs.

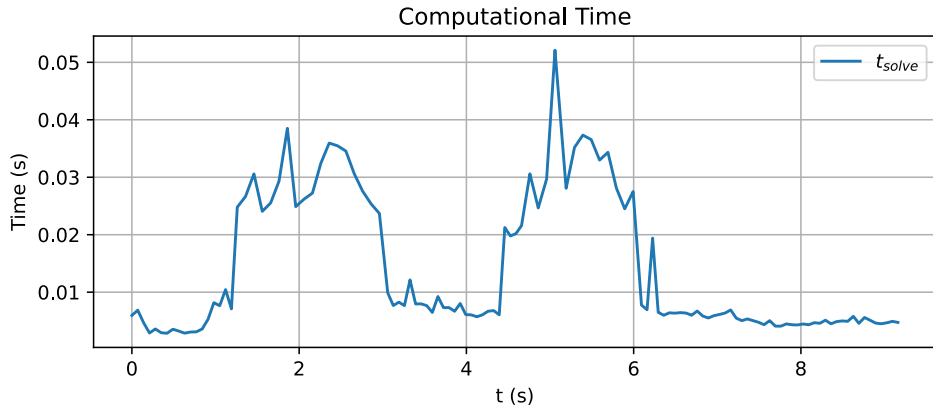


Figure A.16: OAMPC simulation computation time.

OAMPC with Obstacle Activation

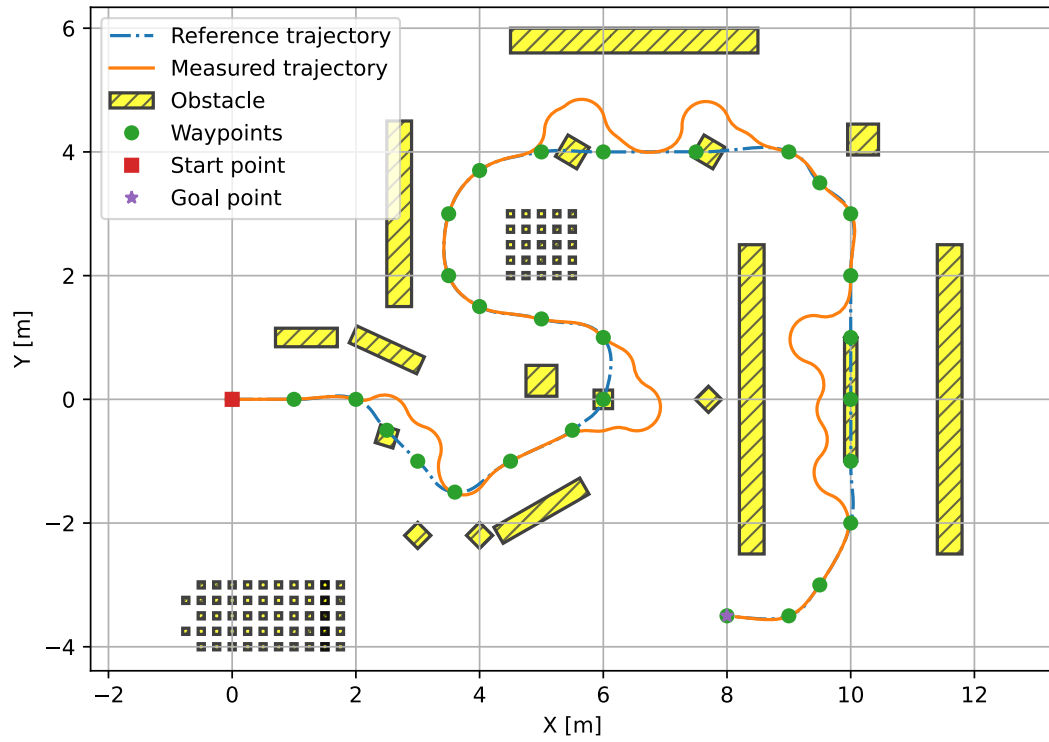


Figure A.17: OAMPC simulation trajectory tracking with obstacle activation.

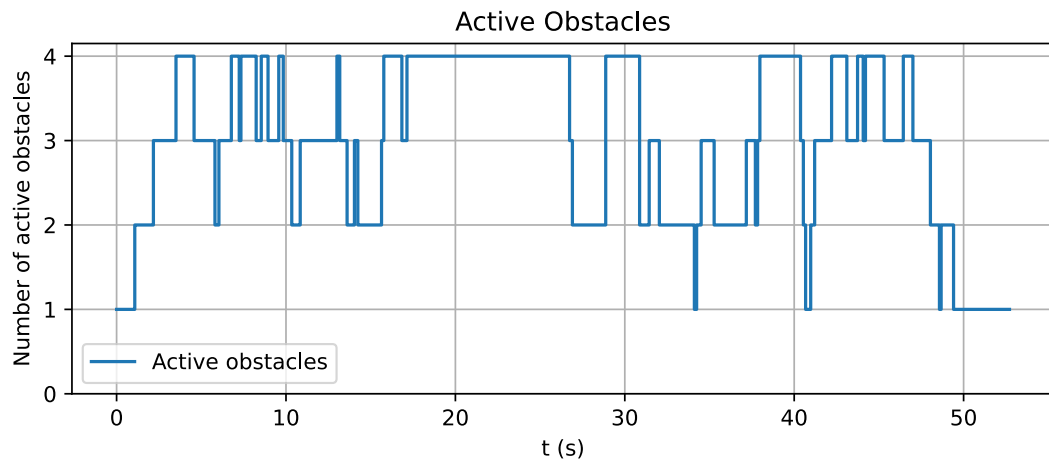


Figure A.18: Number of active obstacles during the OAMPC simulation.

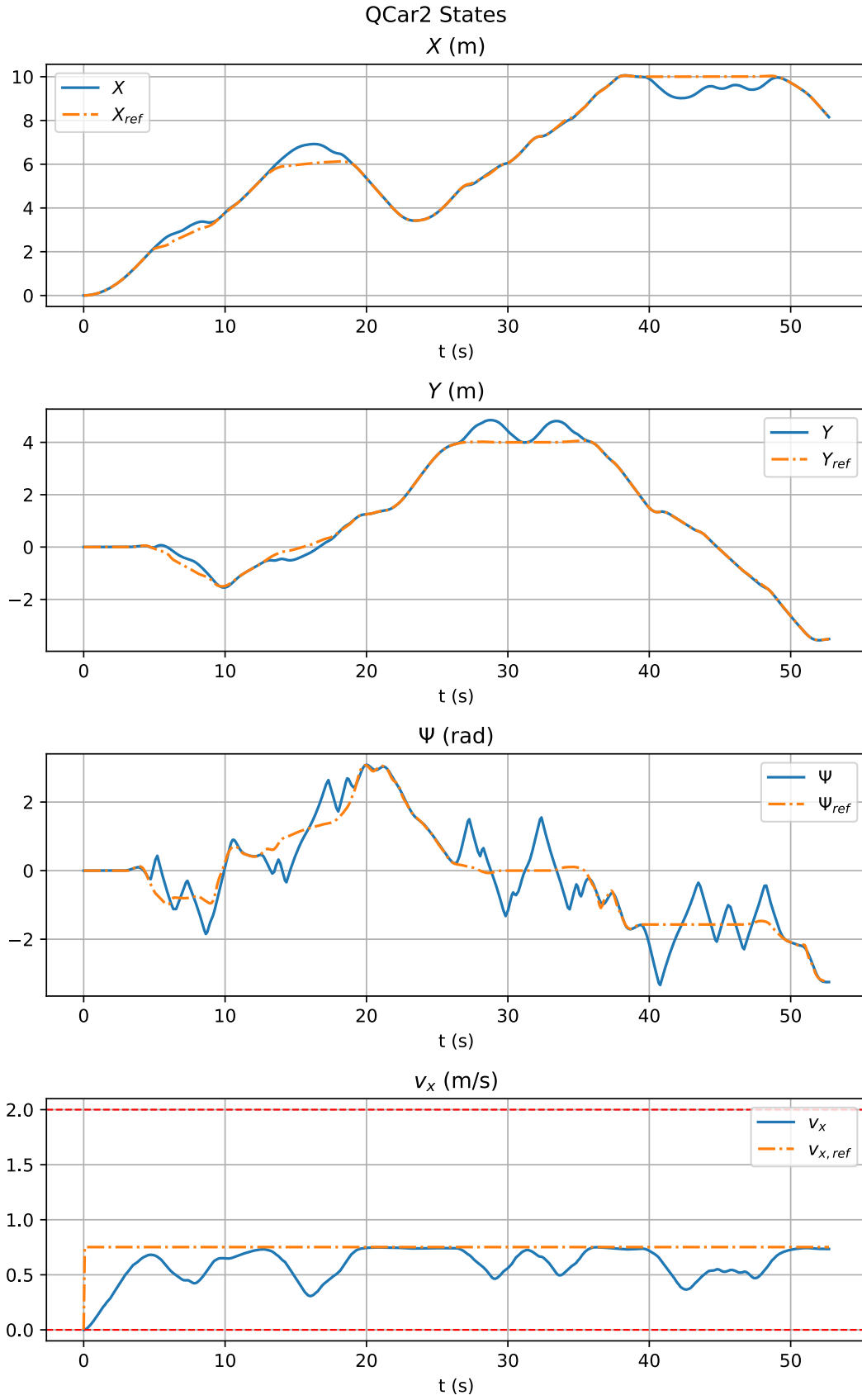


Figure A.19: OAMPC simulation vehicle states with obstacle activation.

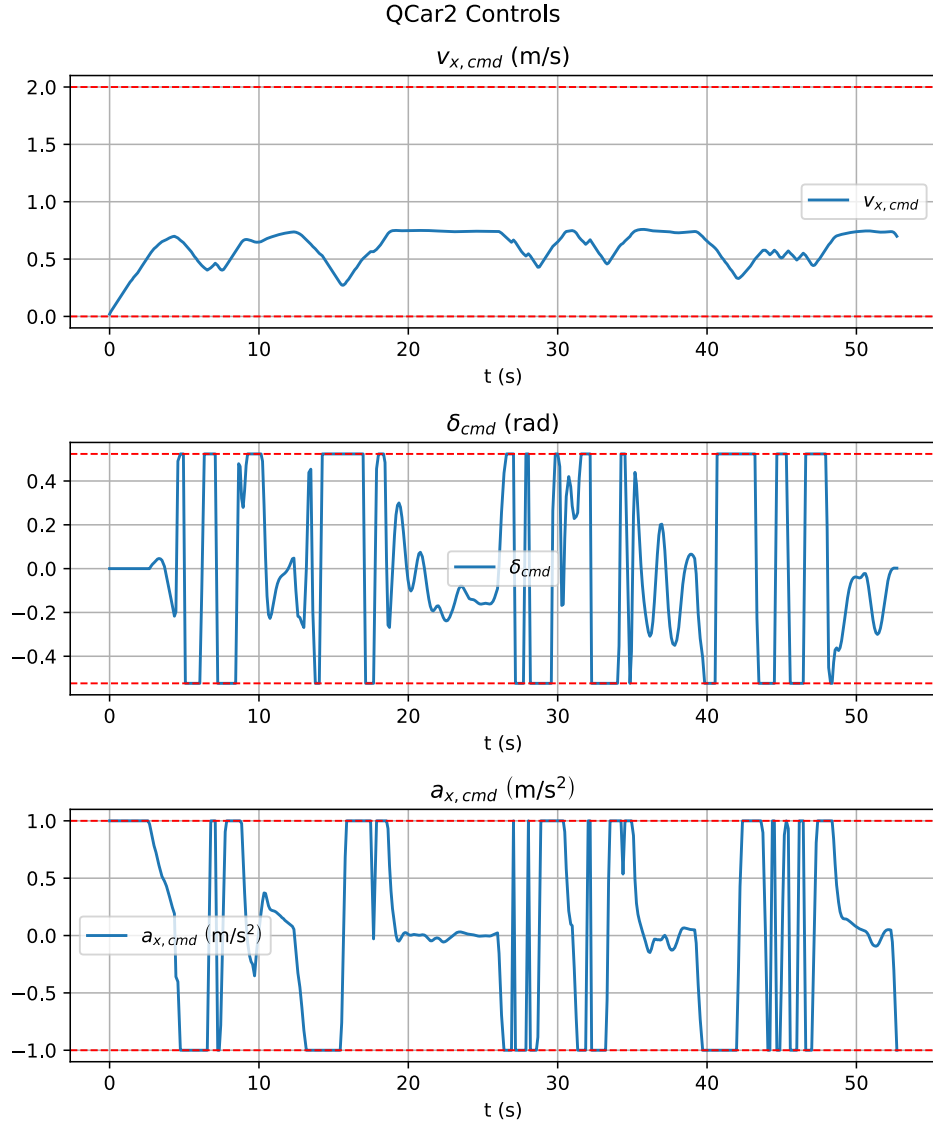


Figure A.20: OAMPC simulation control inputs with obstacle activation.

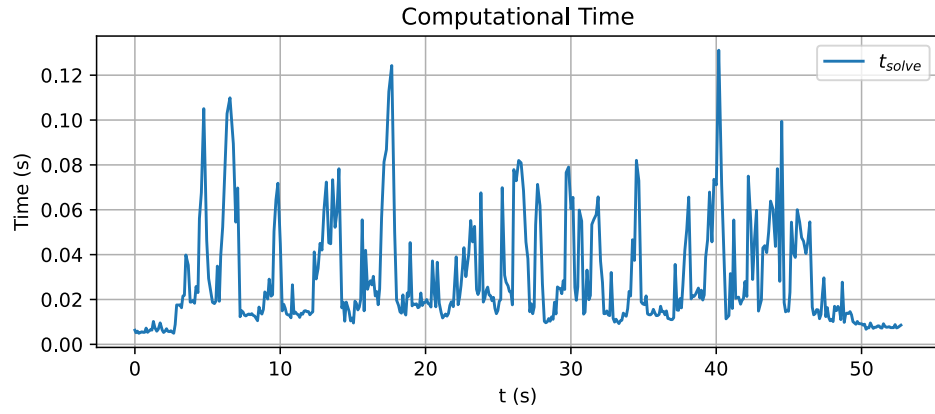


Figure A.21: OAMPC simulation computation time with obstacle activation.

Experimental Results

NMPC Reference Trajectory Tracking

Online-generated trajectories (NMPC Mk1)

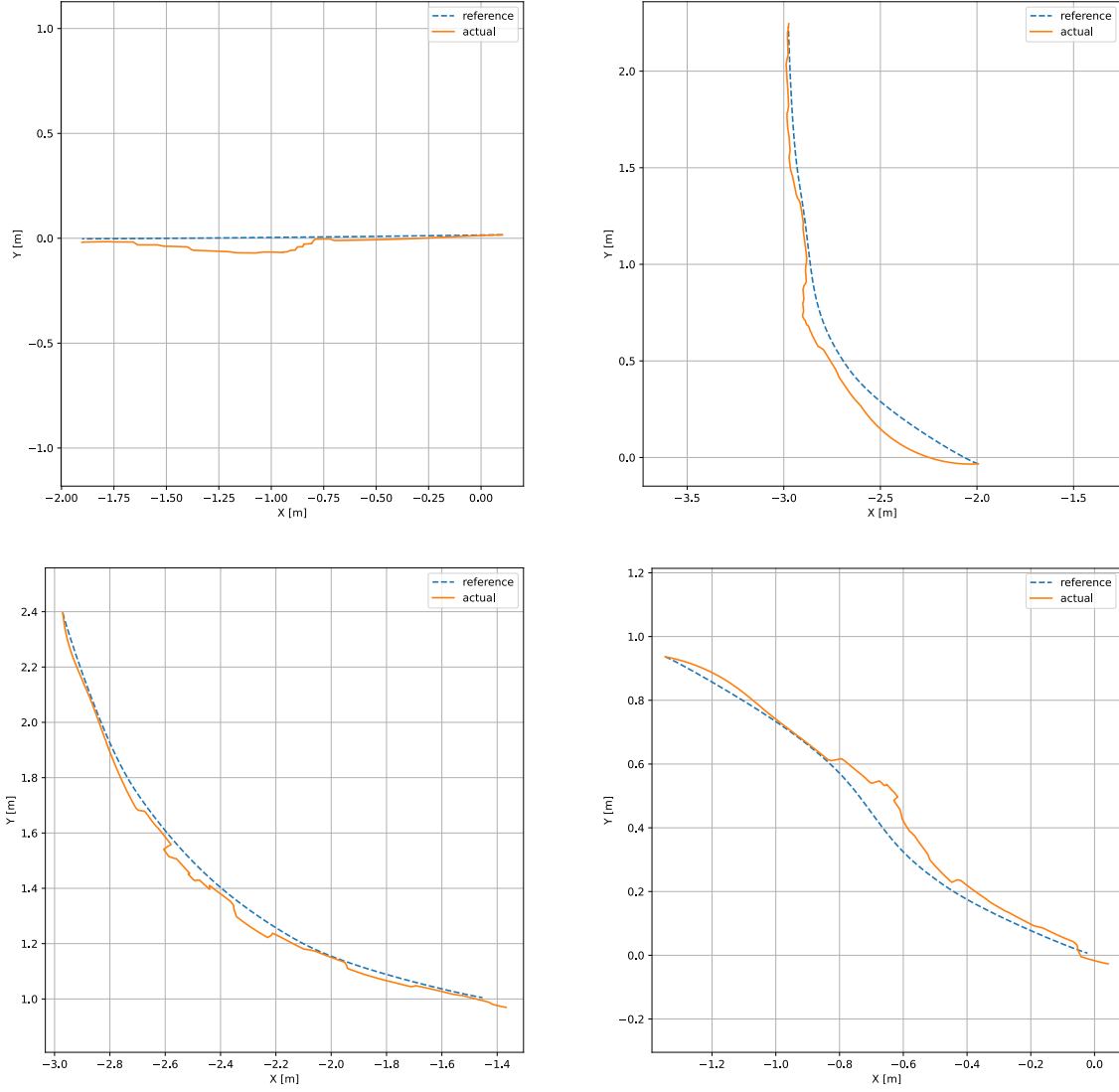


Figure A.22: NMPC experimental trajectory-tracking results over four runs.

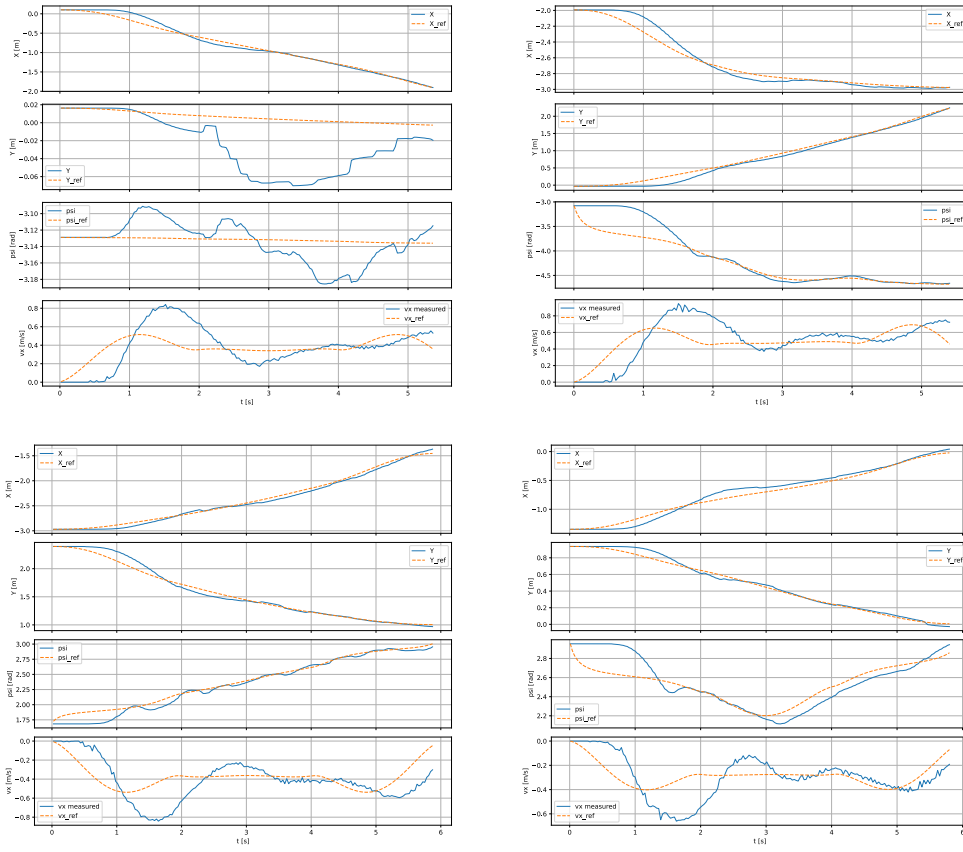


Figure A.23: NMPC experimental state-tracking results over four runs.

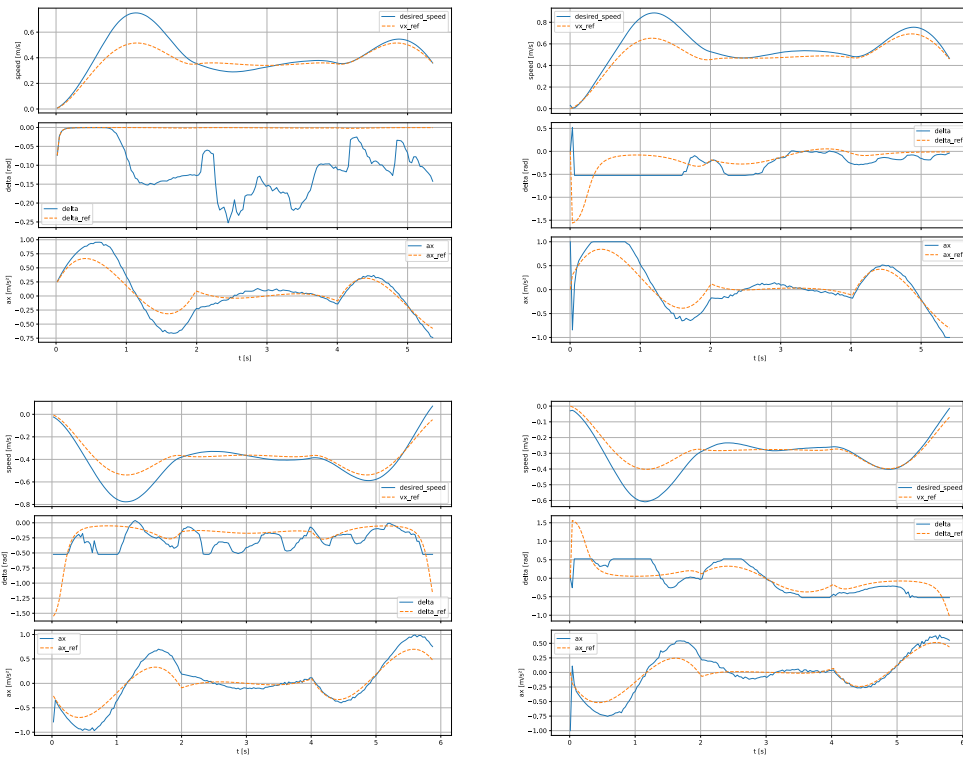


Figure A.24: NMPC experimental control-input results over four runs.

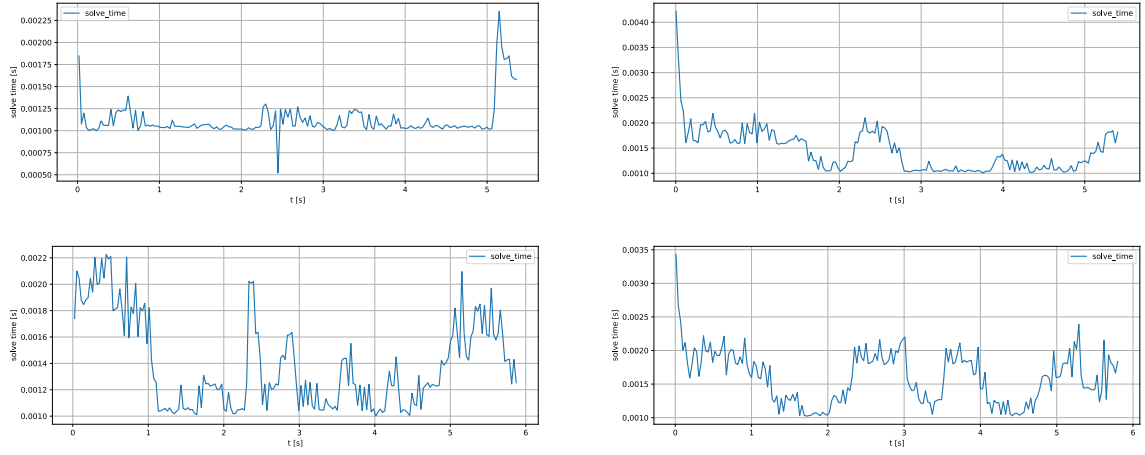


Figure A.25: NMPC experimental computation-time results over four runs.

Predefined circular trajectory (NMPC Mk2)

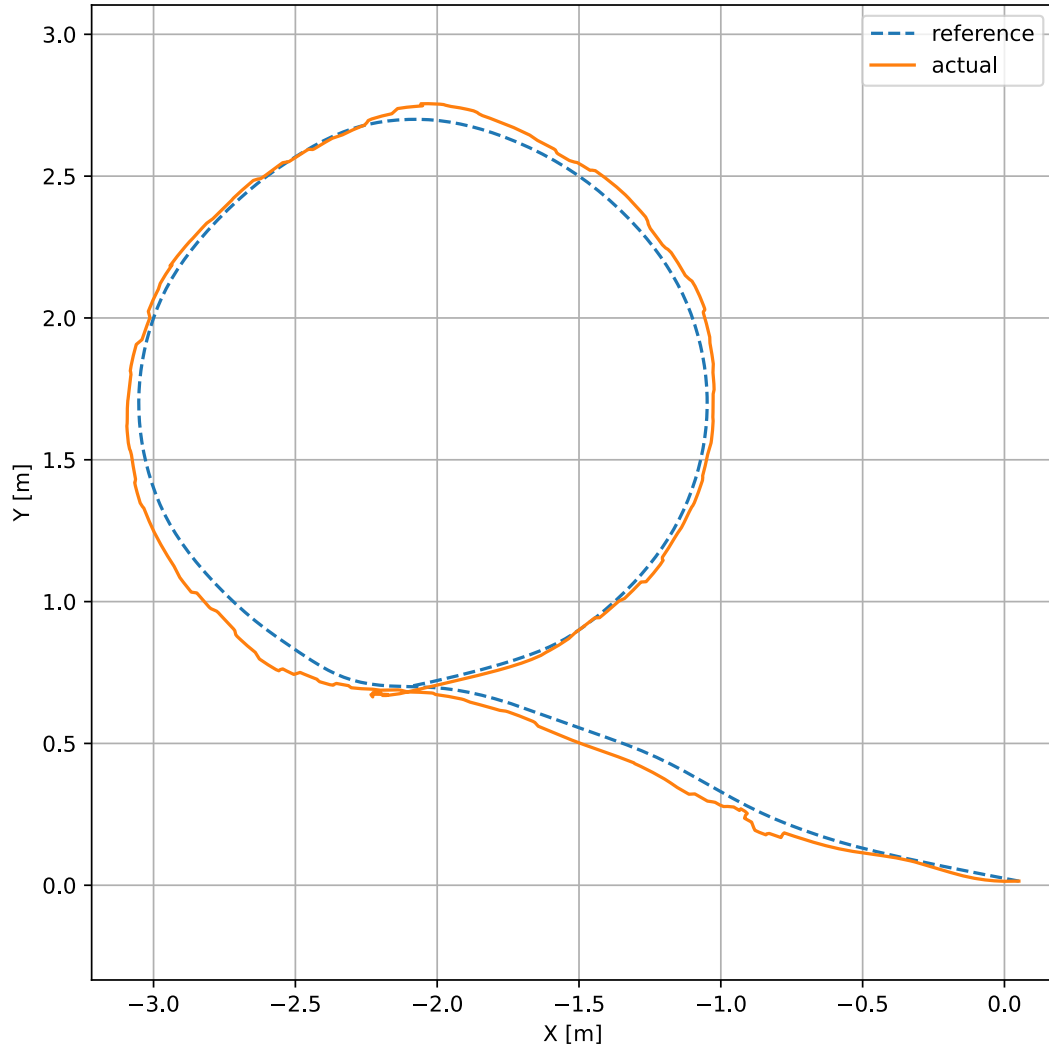


Figure A.26: NMPC Mk2 experimental trajectory tracking.

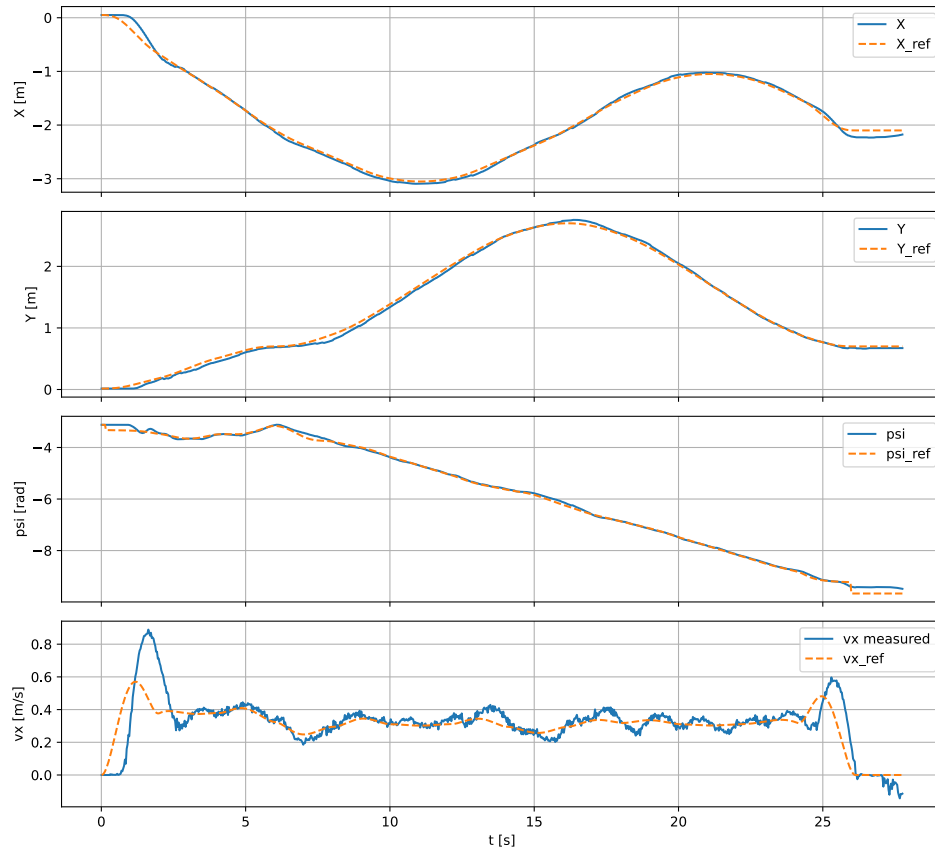


Figure A.27: NMPC Mk2 experimental state tracking.

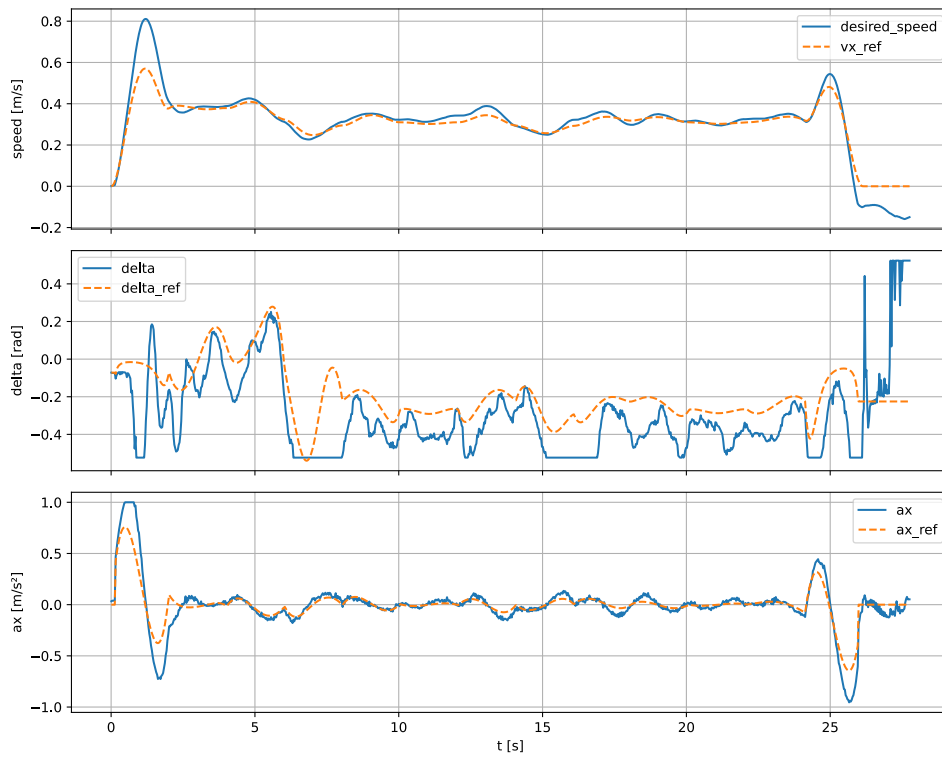


Figure A.28: NMPC Mk2 experimental control inputs.

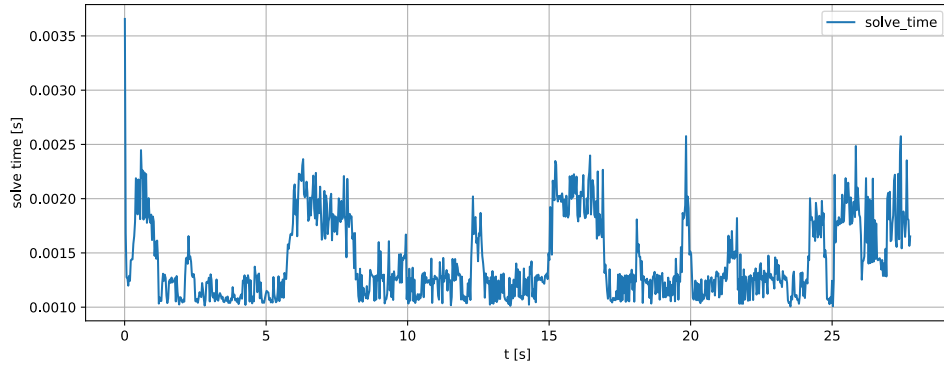


Figure A.29: NMPC Mk2 experimental computation time.

FLMPC Reference Trajectory Tracking

Online-generated trajectories (FLMPC Mk1)

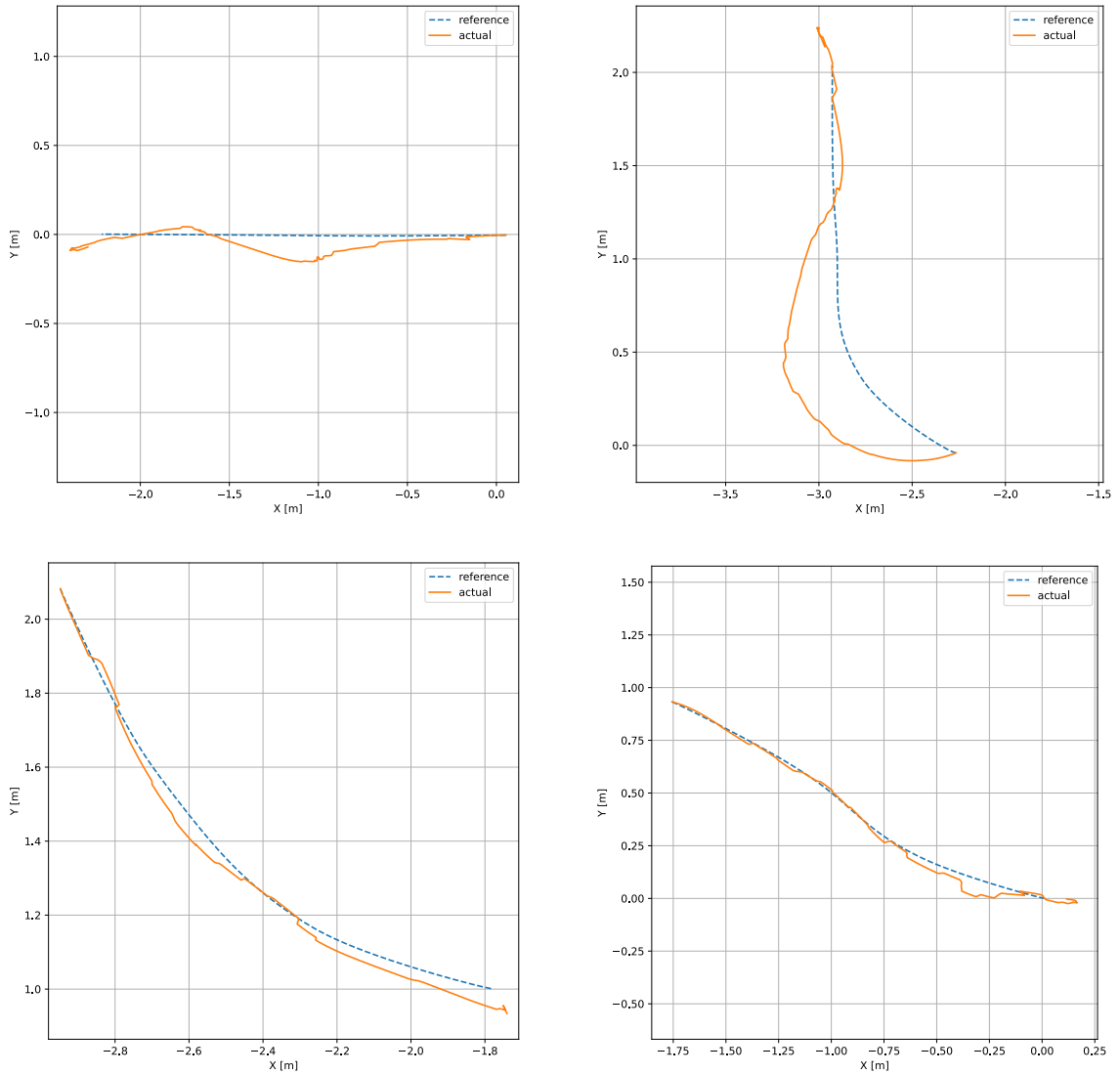


Figure A.30: FLMPC experimental trajectory-tracking results over four runs.

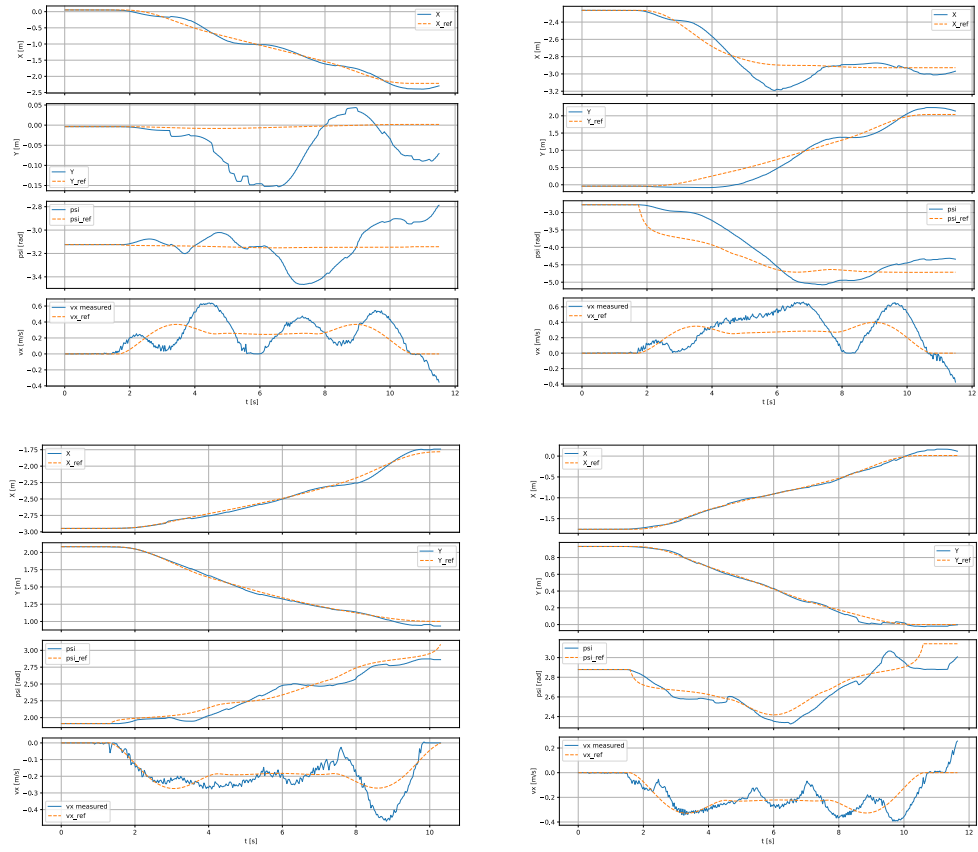


Figure A.31: FLMPC experimental state-tracking results over four runs.

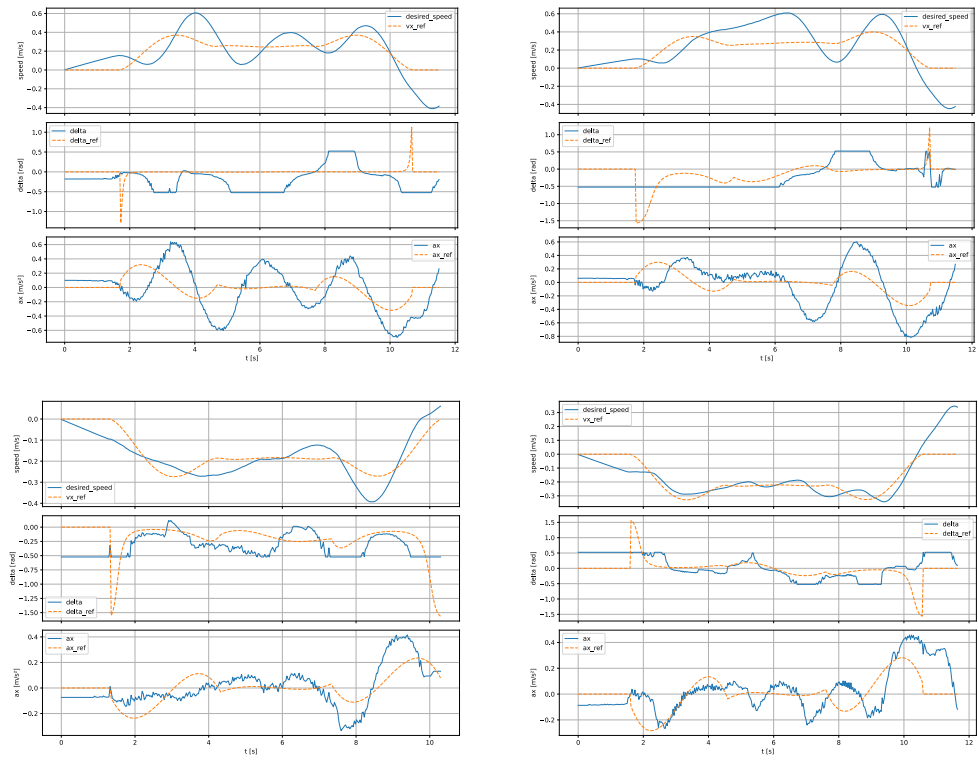


Figure A.32: FLMPC experimental control-input results over four runs.

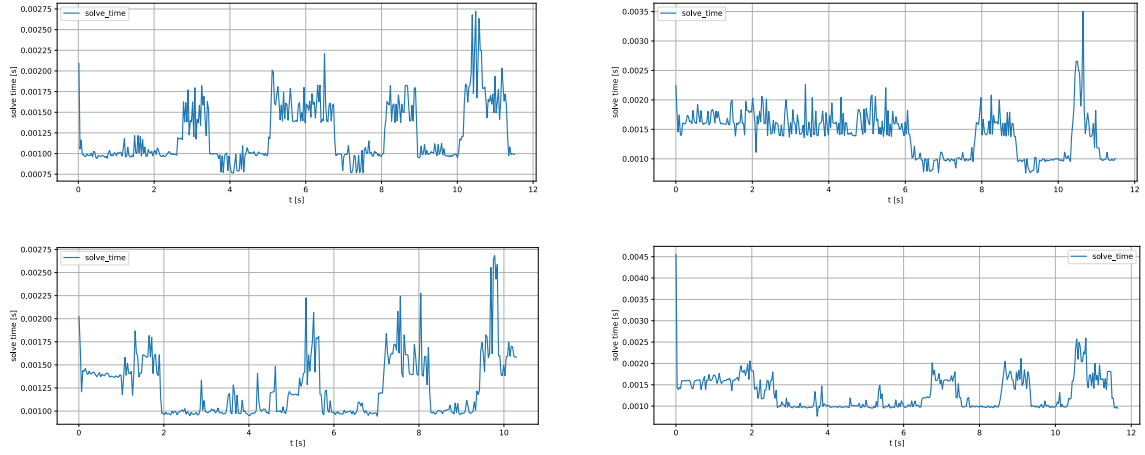


Figure A.33: FLMPC experimental computation-time results over four runs.

Predefined circular trajectory (FLMPC Mk2)

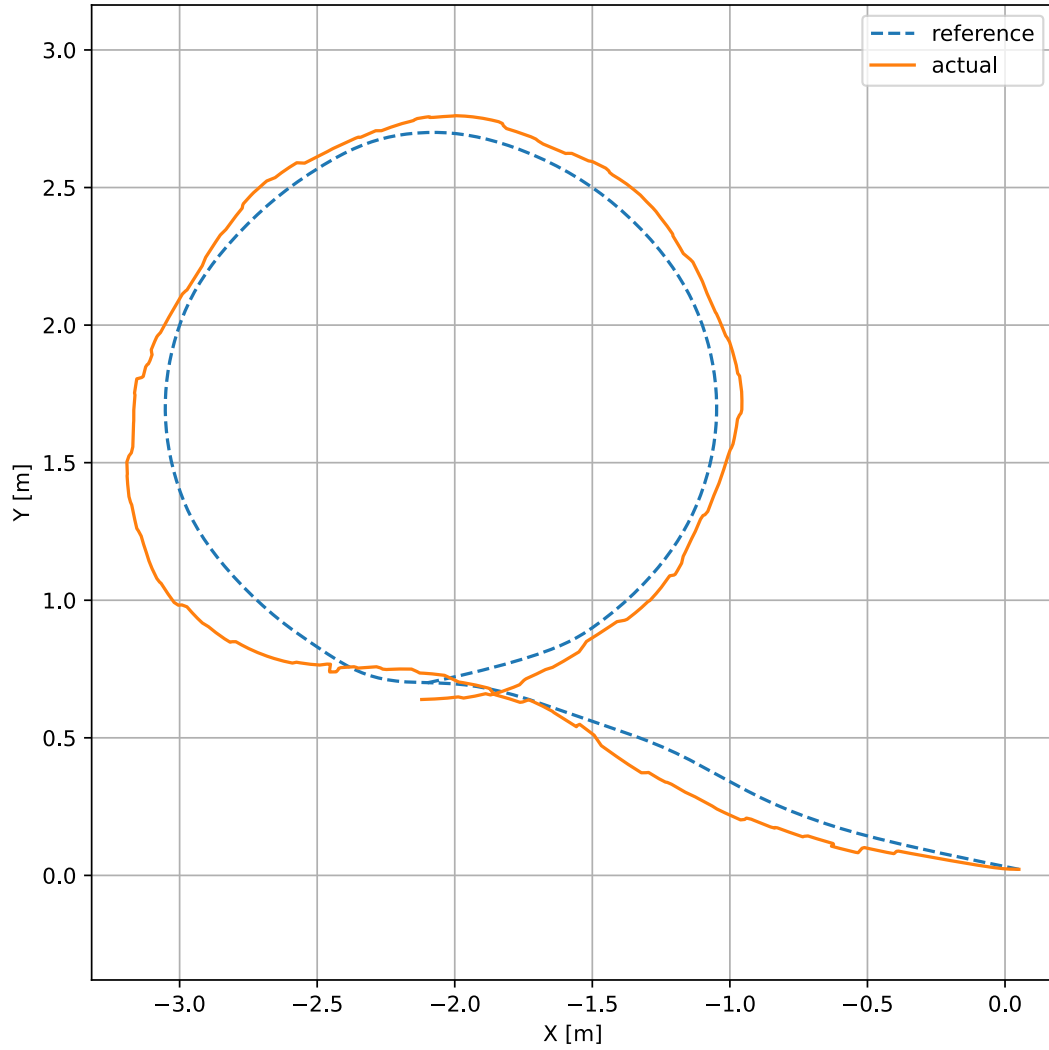


Figure A.34: FLMPC Mk2 experimental trajectory tracking.

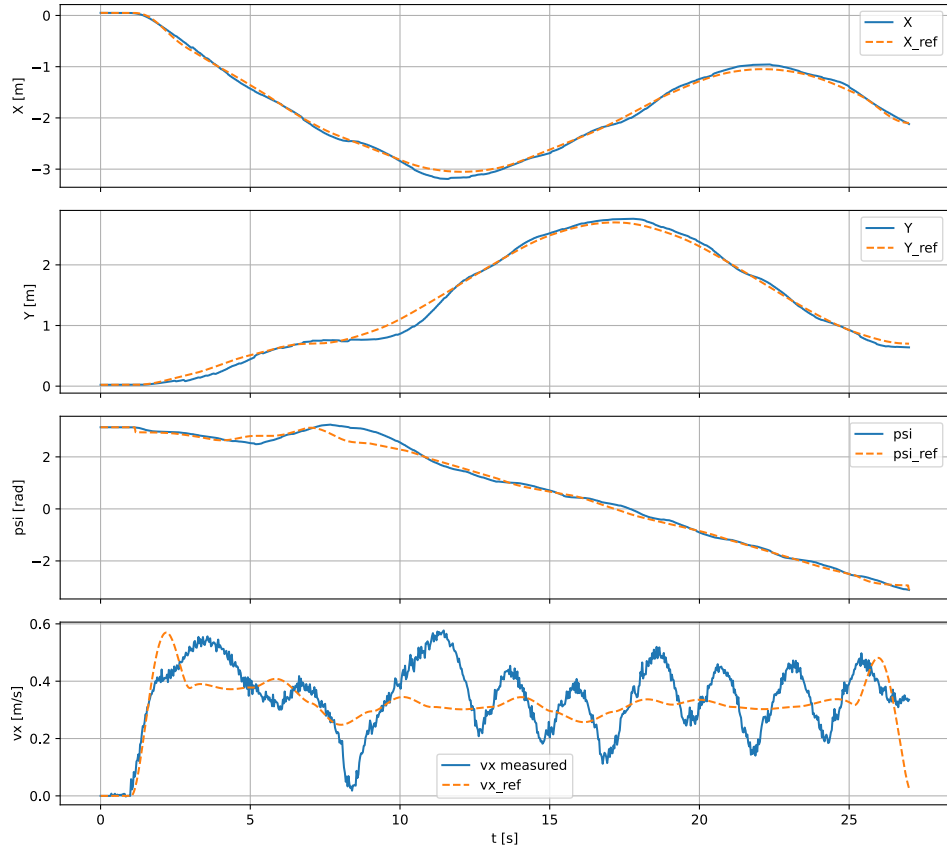


Figure A.35: FLMPC Mk2 experimental state tracking.

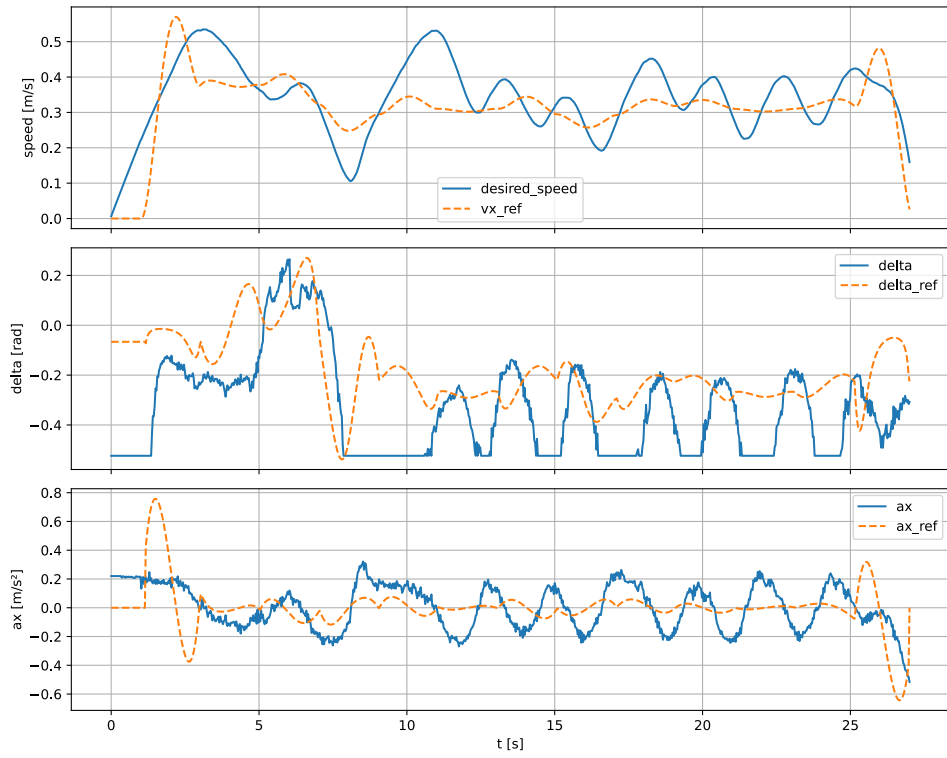


Figure A.36: FLMPC Mk2 experimental control inputs.

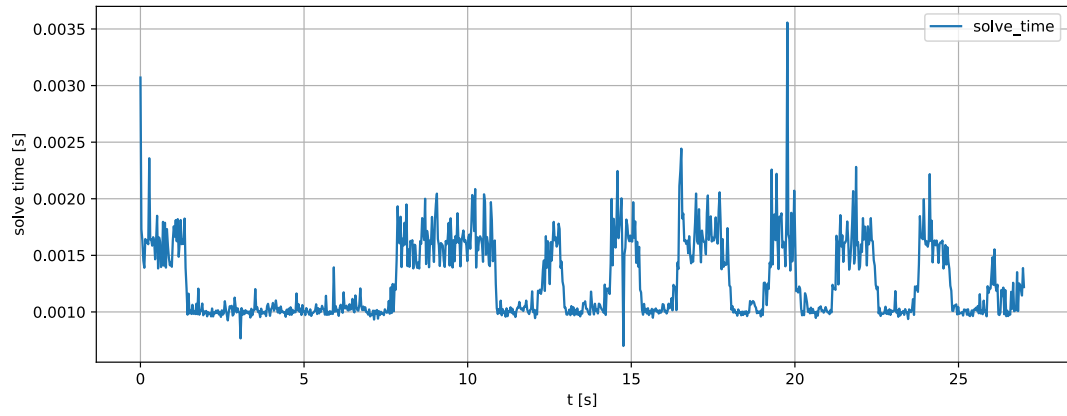


Figure A.37: FLMPC Mk2 experimental computation time.

NMPC and FLMPC Comparison

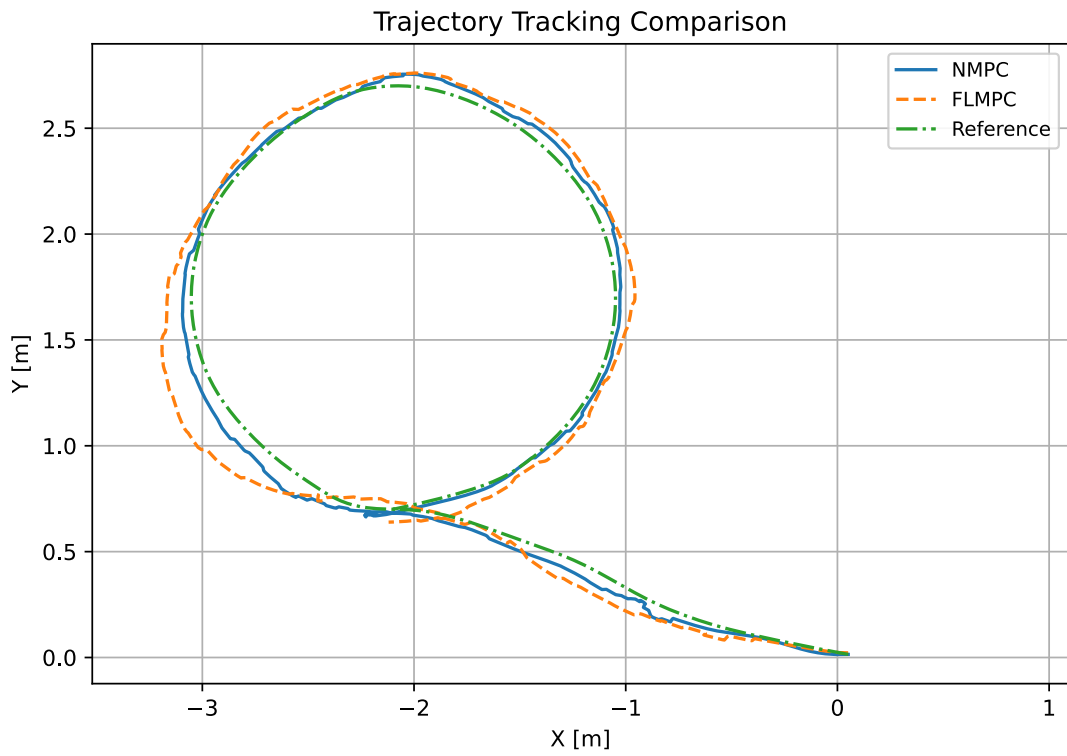


Figure A.38: NMPC and FLMPC experimental trajectory-tracking comparison.

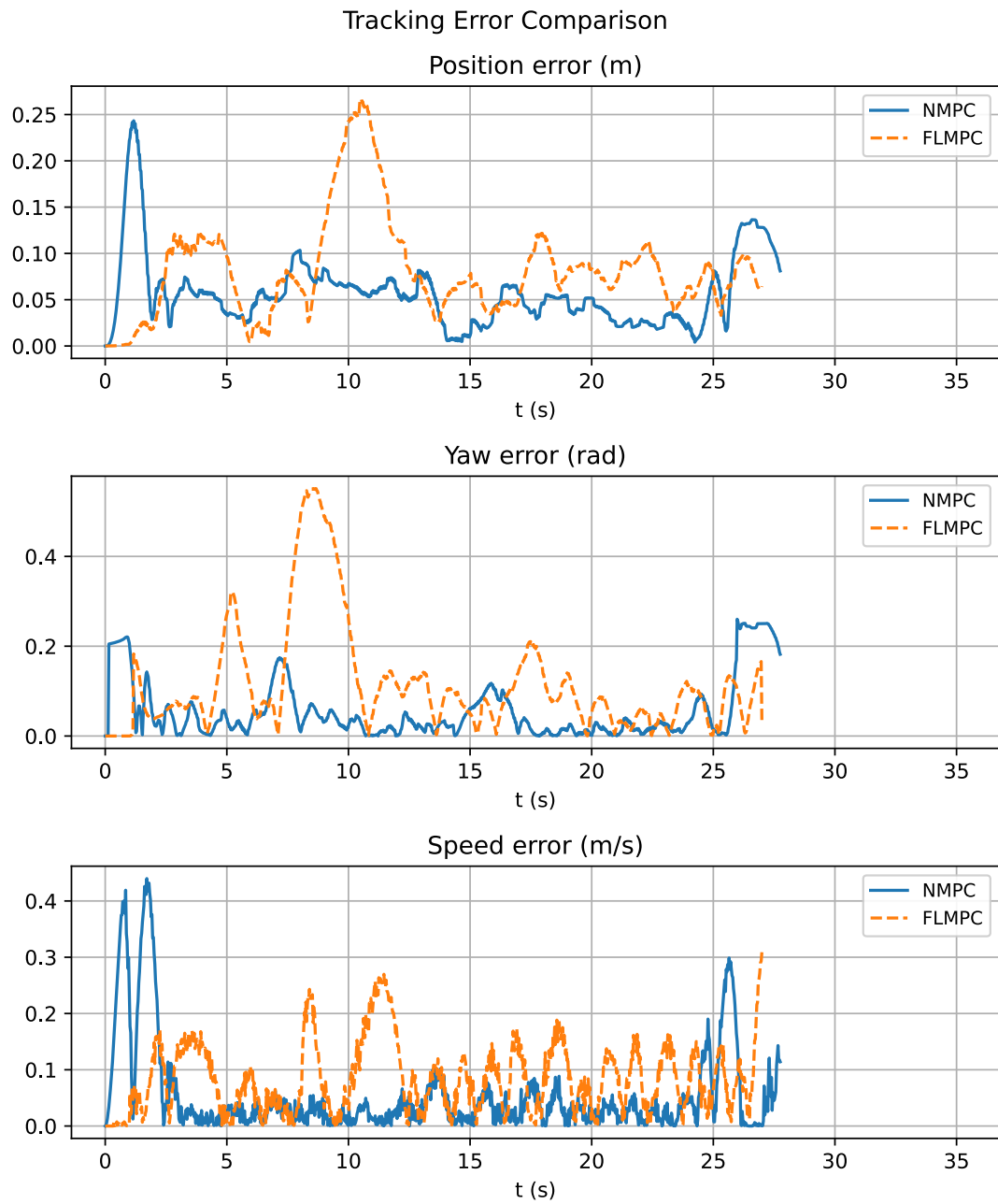


Figure A.39: NMPC and FLMP experimental tracking-error comparison.

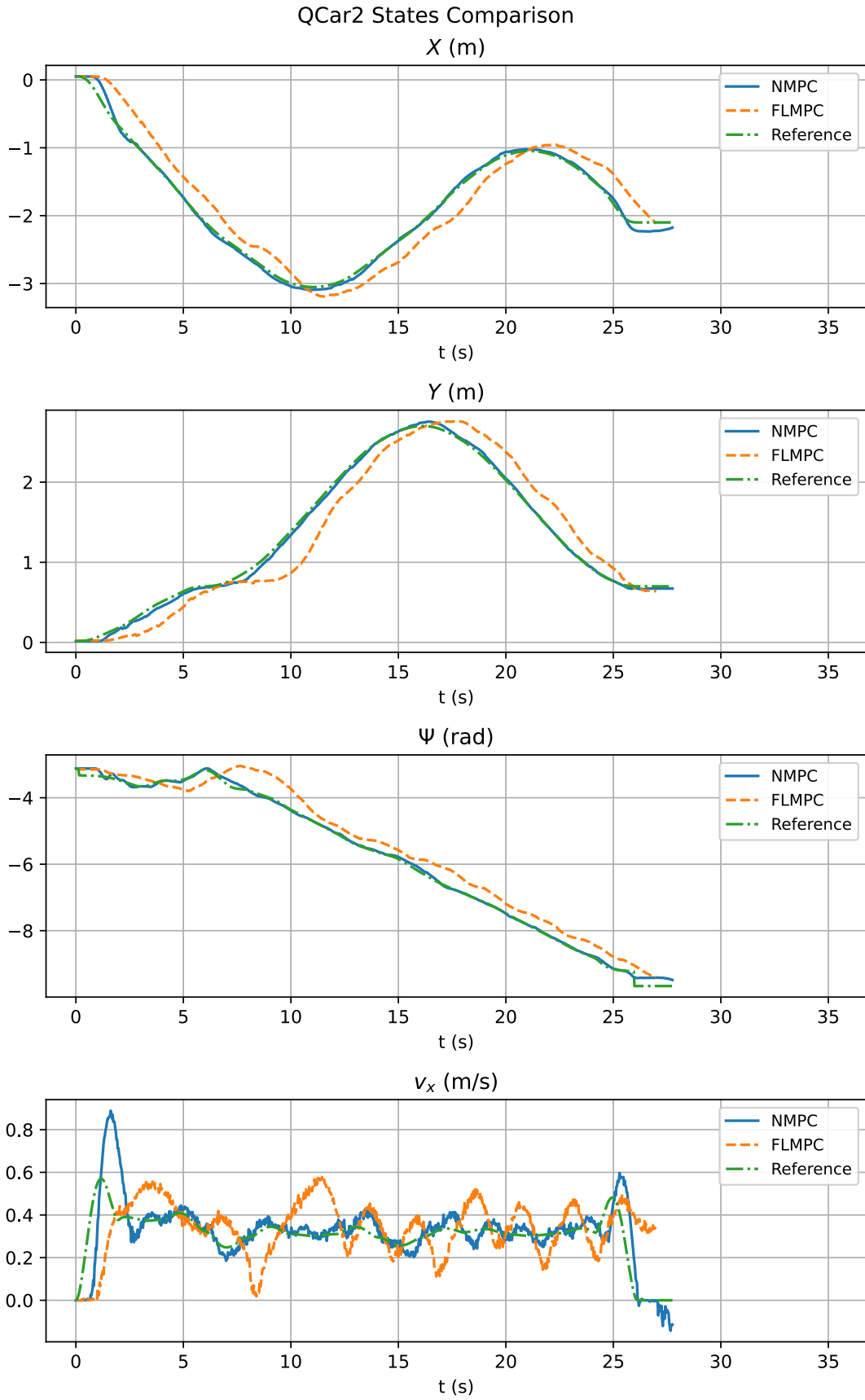


Figure A.40: NMPC and FL MPC experimental state-tracking comparison.

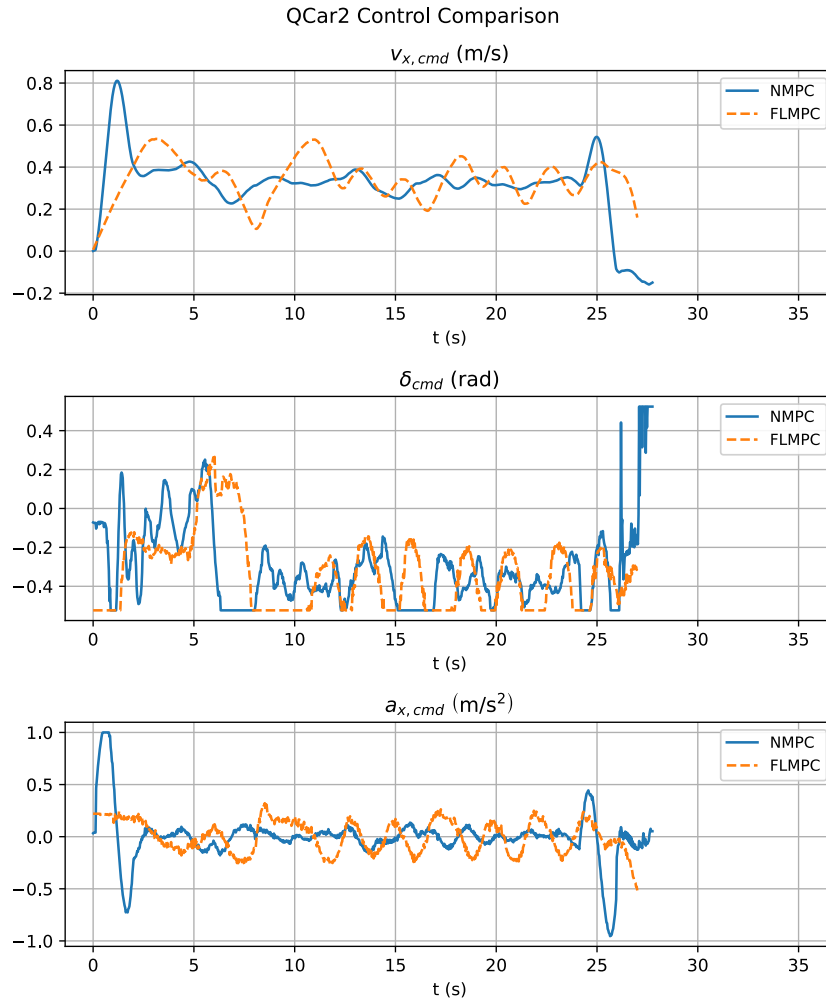


Figure A.41: NMPC and FL MPC experimental control-input comparison.

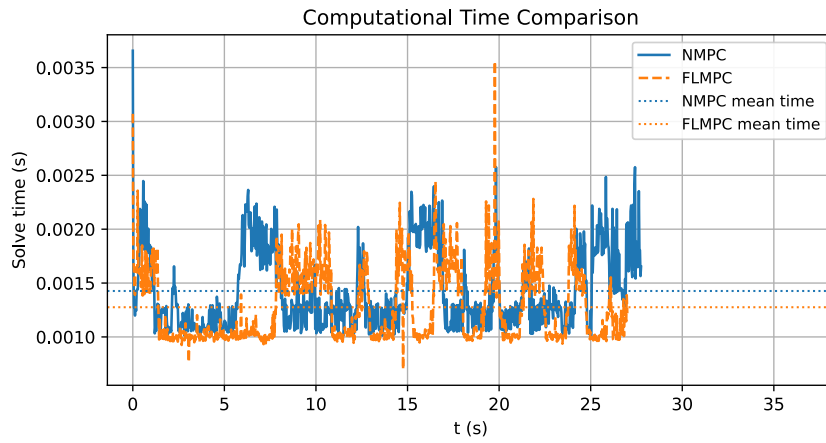


Figure A.42: NMPC and FL MPC experimental computation time comparison.